



MASTER THESIS

Comparative Analysis of Open-Source Mesh Simplification Methods for Game Development

Author:

Lukáš Gallo

Supervisors:

Ruben Tack (MA)

Robbie Storm (BSc)

*This thesis is submitted in fulfilment of the requirements
for the degree of Master of Science in Game Technology*

In the programme

Professional Master Game Technology

Academy for AI, Games and Media

Breda University of Applied Sciences

29.05.2026

Declaration of Authorship

I, Lukáš Gallo, declare that this thesis titled, “Comparative Analysis of Open-Source Mesh Simplification Methods for Game Development” and the work presented are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.
- For this thesis, I have used Claude AI for the academic writing and research. Most of the paragraphs were edited by AI to make them more concise and focused. Everything that AI produced was checked by a human.

Signed: Lukáš Gallo



Date: 29.05.2026

"This study taught me that persistence is not always about pushing harder. Sometimes the more valuable skill is stepping back and letting go of a path that is not working, and then reframing forty hours of failure as a documented finding rather than a dead end."

Lukáš Gallo

Abstract

Academy for AI, Games and Media

Master of Game Technology

Comparative Analysis of Open-Source Mesh Simplification Methods for Game Development

By Lukáš Gallo

Game developers create simplified versions of 3D models to maintain visual quality across diverse player hardware, a process typically automated through mesh simplification algorithms. Commercial tools cost tens of thousands of dollars annually, leaving independent developers to choose among free alternatives without empirical guidance on which produces better quality, runs faster, or integrates more easily. Existing research evaluates methods in isolation using geometric error metrics that poorly predict what players perceive. This thesis asked how four accessible mesh simplification methods compare in geometric accuracy, processing performance, visual quality, and usability within Blender-based game asset workflows. The comparison served independent developers, students, and studios where commercial licensing is prohibitively expensive. Four open-source implementations were benchmarked across 288 operations on eight production game assets, with scalability tested on models up to 28.1 million faces. A perceptual study collected 2,769 pairwise preference judgments from 150 participants. No single method dominated all dimensions. The method with the highest perceptual preference also delivered the fastest processing while avoiding the catastrophic failure modes present in alternatives. The method with the lowest geometric error ranked third perceptually due to shading degradation on hard-surface assets, revealing that geometric accuracy does not predict perceived quality. Geometric metrics explained only 10–15% of variance in perceptual preference. The findings provide a context-dependent selection framework mapping production scenarios to recommended methods, and demonstrated that geometric metrics alone are insufficient for production quality evaluation. The complete benchmark and analysis pipeline is published to enable replication.

Acknowledgements

I would like to thank my project supervisors, Ruben Tack and Robbie Storm, for their guidance and feedback throughout this research. I am equally grateful to Thomas Buijtenweg for his continuous advisory role across the full duration of the project, and to Jonas Pastoors, whose sharp critique of the literature review chapter in the early stages taught me how academic writing actually works. Every chapter that followed was better because of it.

This research would not have been possible without Bohemia Interactive, who provided the production game assets that form the core of the evaluation. I thank my colleagues there for their support and patience during my studies. I also thank Martin Normark for sharing his experience with neural mesh simplification replication, which further informed the neural method investigation in this study.

I want to express my gratitude to The Kellner Family Foundation, whose scholarship funding for my bachelor's degree made it possible for me to pursue this master's programme.

To my cousin and classmate Jiří Vojtěch, thank you for being by my side throughout not just this master's, but our entire academic journey. To my family, thank you for your constant support. And to the lecturers and classmates at BUas, thank you for the knowledge shared and the memories made along the way.

Contents

Declaration of Authorship	1
Abstract	3
Acknowledgements	4
Contents	5
List of Figures	9
List of Tables	11
List of Abbreviations	13
Chapter 1: Introduction	16
Chapter 2: Literature Review	19
2.1 Traditional Geometric Methods	19
2.1.1 Quadric Error Metrics	19
2.1.2 Progressive Meshes	20
2.1.3 Blender Implementation	21
2.2 Neural Alternatives to Traditional Methods	22
2.3 Evaluation Metrics and Benchmarking	23
2.3.1 Academic Geometric Metrics	24
2.3.2 Production Performance Metrics	25
2.3.3 Perceptual Quality Research	25
2.4 Production Tool Integration	26
2.4.1 Evaluated Implementations	26
2.4.2 Integration Barriers	28
2.5 Conclusion	28
Chapter 3: Methodology	31
3.1 Method Selection and Implementation	31
3.2 Performance Metrics	34
3.2.1 Processing Time	34
3.2.2 Peak Memory Usage	34

3.2.3	Scalability	35
3.2.4	Success Rate	35
3.3	Geometric Accuracy Evaluation	36
3.3.1	Hausdorff Distance	37
3.3.2	Root Mean Squared Error	37
3.4	Case Study Design	38
3.5	Perceptual Quality Assessment	39
3.5.1	Evaluation Method and Design	40
3.5.2	Visual Stimuli Preparation	40
3.5.3	Participant Recruitment and Data Collection	41
3.5.4	Pilot Testing	42
3.6	Data Analysis	43
3.7	Technical Environment	43
3.8	Ethical Considerations	44
3.9	Limitations and Mitigation Strategies	45
	Chapter 4: Data and Results	48
4.1	Processing Performance	48
4.1.1	Processing Time	49
4.1.2	Memory Usage	52
4.2	Geometric Accuracy	53
4.3	Reduction Accuracy	57
4.4	Scalability	58
4.4.1	Processing Time Scaling	58
4.4.2	Memory at Scale	60
4.5	Perceptual Quality Assessment	60
4.5.1	Overall Perceptual Rankings	61
4.5.2	Perceptual Rankings by Reduction Level	62
4.5.3	Asset-Specific Perceptual Patterns	63
4.6	Geometric-Perceptual Correlation	63

4.7	Summary of Findings	65
	Chapter 5: Discussion of Data and Results	67
5.1	Method Behavior: Visual Artifacts and Algorithmic Lineage	67
5.1.1	Fast-simplification: threshold escalation and catastrophic failure	67
5.1.2	Open3D: geometric fidelity and shading degradation	70
5.1.3	CGAL: memoryless drift and local spikes	72
5.1.4	Meshoptimizer: consistency over specialization	74
5.1.5	Summary	75
5.2	Geometric-Perceptual Disconnect	75
5.3	Production Assets as Evaluation Ground	77
5.4	Methodological Reflection	78
5.5	Ethical Considerations	80
	Chapter 6: Conclusion and Future Directions	81
6.1	Research Question Answer	81
6.2	Production Scenario Recommendations	83
6.3	Contributions	84
6.4	Future Research	85
6.5	Final Reflection	87
	Bibliography	88
	Appendix A: Method Selection and Implementation Documentation	95
A.1	Excluded Methods: Justification Summary	95
A.1.1	Probabilistic Quadrics	96
A.1.2	libigl	96
A.1.3	VTK	96
A.1.4	MeshLab	96
A.1.5	Platform-Incompatible Methods	97
A.1.6	Unvalidated Personal Projects	97
A.1.7	Component Libraries	97
A.2	Neural Methods Scalability Analysis	98

A.2.1	MeshCNN	98
A.2.2	Neural Mesh Simplification	98
A.3	Traditional Method Implementation: Key Technical Challenges	102
A.3.1	CGAL	102
A.3.2	meshoptimizer API Discovery	103
A.3.3	fast-simplification API Compatibility	103
A.4	Empirical Accessibility Barrier Measurements	104
A.4.1	Installation Complexity Quantification	104
A.4.2	Barrier Multipliers	104
A.4.3	Skill Escalation Analysis	105
A.4.4	Barrier Compounding Analysis	105
	Appendix B: Detailed Benchmark and Perceptual Data	107
B.1	Processing Performance by Reduction Level	107
B.2	Geometric Accuracy by Reduction Level	110
B.3	Per-Asset Detail Tables	113
B.4	Perceptual Quality Breakdowns	117
B.5	Scalability Data	119

List of Figures

Chapter 3: Methodology

Figure 3.1	Systematic three-stage filtering process from 15 candidate methods to 4 selected traditional QEM implementations	32
Figure 3.2	Complementary geometric error metrics: Hausdorff distance captures worst-case single-point deviation while RMSE quantifies average distributed error across all vertices	36
Figure 3.3	Production test suite of eight Bohemia Interactive game assets ranging from 17,515 to 169,087 faces	38
Figure 3.4	Scalability test suite of four Stanford 3D Scanning Repository models ranging from 69,451 to 28,055,742 faces	39
Figure 3.5	Perceptual quality assessment interface showing paired distant-close evaluation design	41

Chapter 4: Data

Figure 4.1	Processing time distribution by method	50
Figure 4.2	Processing time versus input face count at 50%, 80%, and 90% reduction levels	49
Figure 4.3	Memory usage distribution by method	53
Figure 4.4	Geometric accuracy distribution by method, aggregated across all assets and reduction levels	55
Figure 4.5	Geometric accuracy at 90% reduction level	56
Figure 4.6	Reduction accuracy: target versus actual reduction percentage for all asset-method-reduction combinations	57
Figure 4.7	Processing time scaling on log-log axes across four Stanford models at 50%, 80%, and 90% reduction levels	59
Figure 4.8	Perceptual win rates by method and viewing distance e	62
Figure 4.9	Perceptual win rate versus normalized geometric error for all method-asset-reduction combinations	64

Chapter 5: Discussion

Figure 5.1	Church at 80% reduction	68
Figure 5.2	Watermill at 80% reduction	70
Figure 5.3	Bunker at 80% reduction	73

Appendix B: Supplementary Data

Figure B.1	Processing time distribution by method at 50% reduction	108
Figure B.2	Processing time distribution by method at 80% reduction	109
Figure B.3	Processing time distribution by method at 90% reduction	110
Figure B.4	Geometric accuracy at 50% reduction	111
Figure B.5	Geometric accuracy at 80% reduction	112
Figure B.6	Perceptual win rates by method and reduction level	117
Figure B.7	Perceptual win rate heatmap by method and asset	118
Figure B.8	Geometric-perceptual correlation	119
Figure B.9	Reduction accuracy for Stanford scalability models	121

List of Tables

Chapter 2: Literature Review

Table 2.1	Overview of all mesh simplification methods identified during the literature and tool survey	27
-----------	--	----

Chapter 3: Methodology

Table 3.1	Selected methods for comparative evaluation	33
-----------	---	----

Chapter 4: Data

Table 4.1	Processing time descriptive statistics across all assets and reduction levels	49
Table 4.2	Memory usage descriptive statistics	52
Table 4.3	Geometric accuracy descriptive statistics across all assets and reduction levels	54
Table 4.4	Perceptual study parameters	61
Table 4.5	Perceptual quality overall win rates by method	61
Table 4.6	Spearman rank correlation between perceptual win rate and geometric error metrics	63

Chapter 6: Conclusions and Recommendations

Table 6.1	Production scenario recommendations derived from Chapters 4 and 5 findings	83
-----------	--	----

Appendix A: Method Selection Evidence

Table A.1	Overview of excluded methods with primary exclusion rationale	95
Table A.2	Projected GPU memory requirements for MeshCNN processing of production game assets	98
Table A.3	Installation time, dependency count, and disk space measured across all four selected methods	104

Appendix B: Supplementary Data

Table B.1	Method comparison at 50% reduction level	107
Table B.2	Method comparison at 80% reduction level	107

Table B.3	Method comparison at 90% reduction level	107
Table B.4	AK-74, 11,194 faces: per-method benchmark results at 50%, 80%, and 90% reduction	113
Table B.5	M9 pistol, 14,932 faces: per-method benchmark results at 50%, 80%, and 90% reduction	113
Table B.6	Bunker, 23,508 faces: per-method benchmark results at 50%, 80%, and 90% reduction	114
Table B.7	LAV, 41,886 faces: per-method benchmark results at 50%, 80%, and 90% reduction	114
Table B.8	Mi-8 helicopter, 118,944 faces: per-method benchmark results at 50%, 80%, and 90% reduction	115
Table B.9	Watermill, 129,124 faces: per-method benchmark results at 50%, 80%, and 90% reduction	115
Table B.10	Jeep, 141,282 faces: per-method benchmark results at 50%, 80%, and 90% reduction	116
Table B.11	Church, 169,087 faces: per-method benchmark results at 50%, 80%, and 90% reduction	116
Table B.12	Perceptual win rates by method, aggregated across all assets and viewing distances	117
Table B.13	Perceptual win rates by method and reduction level	117
Table B.14	Complete scalability results for all Stanford model-method-reduction combinations	120
Table B.15	Processing time scaling factors between consecutive Stanford models	121

List of Abbreviations

3D	Three-Dimensional
AI	Artificial Intelligence
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
CAD	Computer-Aided Design
CGAL	Computational Geometry Algorithms Library
COO	Coordinate (sparse matrix format)
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CUDA	Compute Unified Device Architecture
CV	Coefficient of Variation
DDR4	Double Data Rate 4
FBX	Filmbox File Format
FP16	Half-Precision Floating Point
GB	Gigabyte
GDC	Game Developers Conference
GMP	GNU Multiple Precision Arithmetic Library
GPU	Graphics Processing Unit
GUI	Graphical User Interface
I/O	Input/Output
JSON	JavaScript Object Notation
LAV	Light Armored Vehicle
LOD	Level of Detail
LTS	Long Term Support
MB	Megabyte
ML	Machine Learning
MPFR	GNU Multiple Precision Floating-Point Reliable Library
MSDM	Mesh Structural Distortion Measure
OBJ	Object File Format
OOM	Out of Memory
PC	Personal Computer
PLY	Polygon File Format

PR	Pull Request
QEM	Quadric Error Metrics
RAM	Random Access Memory
RMSE	Root Mean Squared Error
RSS	Resident Set Size
SD	Standard Deviation
SHREC	SHape REtrieval Contest
SSD	Solid State Drive
SVD	Singular Value Decomposition
UV	Texture Coordinate (U-V mapping coordinates)
VRAM	Video Random Access Memory
VTK	Visualization Toolkit

For my grandfather, who never stopped encouraging me to go further.

Chapter 1: Introduction

Visual fidelity in 3D games continues to increase, yet these experiences must run smoothly across diverse player hardware configurations. AAA titles and games seeking broad market reach typically need to accommodate players using everything from older hardware, comparable to modern mobile capabilities, to cutting-edge systems, a range that can span 5+ years of GPU development. Several approaches address this challenge, from runtime culling and instancing techniques to virtualized geometry systems such as Unreal Engine 5's Nanite, which can eliminate manual optimization entirely by streaming mesh detail at runtime (Karis et al., 2021). However, these advanced solutions either complement rather than replace mesh simplification, or remain tied to specific engines and do not yet cover all geometry types. The standard approach across most engines and workflows remains Level of Detail (LOD) systems, which display simpler versions of 3D models when they appear far away, reducing the workload while maintaining visual quality (Clark, 1976). Game engines employ discrete LOD systems that switch between pre-computed mesh versions at runtime based on viewing distance (Epic Games, 2025). Artists create these LOD versions during asset preprocessing, either through manual modeling or automated simplification.

Quadric Error Metrics (QEM) has served as the established standard for automated simplification since Garland and Heckbert's 1997 work. Commercial implementations in Simplygon and InstaLOD demonstrate QEM's adoption in production pipelines, with documented deployment in over 600 AAA titles including Halo Infinite (343 Industries, 2021), The Witcher 3 (CD Projekt RED, 2015), and Sonic Frontiers (Sonic Team, 2022) (InstaLOD, 2025; Microsoft Game Dev, 2024).

However, commercial automation tools remain inaccessible to most developers. Simplygon costs up to \$42,000 per year and InstaLOD up to \$19,000 per year, placing them beyond reach for independent developers and small studios (Donya Labs AB, 2025; InstaLOD GmbH, 2025). This economic barrier affects a substantial population. Blender's 20 million downloads in 2024 indicate substantial adoption among artists seeking accessible tools (Blender Foundation, 2024). These developers typically choose from accessible alternatives: Blender's built-in Decimate modifier, other open-source QEM implementations like PyMeshLab and meshoptimizer, or emerging neural network approaches such as MeshCNN and Neural Mesh Simplification (Blender Foundation, 2025; Cignoni et al., 1998; Hanocka et al., 2019; Kapoulkine, 2025; Potamias et al., 2022).

The problem is not that accessible methods do not exist, nor that no comparative research exists. Rather, existing comparisons do not address the specific needs of developers working in open-source, Blender-centered pipelines. Blender's Decimate modifier provides simplified QEM approximations designed for interactive performance rather than optimal quality (Blender Foundation, 2025). Other free QEM implementations exist with different design priorities and performance characteristics but remain unvalidated for Blender-based game workflows (Knodt, 2025). Neural approaches promise learned simplification strategies with claimed speedups of 10 times over traditional QEM (Potamias et al., 2022), yet no research tested whether they achieve practical deployment within the open-source workflows these developers rely on. Specifically, it remains unknown whether they run on consumer hardware, install within typical Python-based pipeline complexity, and deliver quality matching geometric methods. Despite these theoretical promises from both traditional and neural alternatives, systematic comparison within production contexts remains absent.

Existing research examines specific approaches in isolation rather than comparing accessible alternatives within production contexts. Limited research exists comparing multiple accessible QEM implementations and emerging neural methods within Blender-based game development workflows. Technical artists choosing between Blender's Decimate, other free QEM variants, and neural methods lack empirical evidence about which produces better quality, which runs faster, and which integrates more easily into their workflows. This knowledge gap leaves developers to either accept Decimate's known limitations or experiment with untested alternatives without guidance.

Furthermore, evaluation approaches present their own challenges. Academic research relies on geometric metrics like Hausdorff distance and Root Mean Squared Error (RMSE) to measure mathematical surface error (Cignoni et al., 1998). However, these metrics do not reliably predict visual quality. Lin et al. (2023) demonstrated that low geometric error scores can fail to indicate visible surface distortion, while Liu et al. (2024) found that 82.5% of users preferred simplification methods that geometric metrics scored poorly. This disconnect between mathematical accuracy and perceived quality means comparative evaluation requires multiple measurement types: geometric metrics for comparison with existing research, production metrics for technical feasibility, and perceptual evaluation for visual acceptability.

This research addressed these gaps through systematic comparison of accessible mesh simplification methods within Blender-based game workflows. The investigation directly benefits independent developers working with limited budgets, students and

educators in game programs, and developers in regions where commercial licensing is prohibitively expensive. To ensure ecological validity through applicability to real-world production contexts, this study evaluated methods under real production constraints rather than ideal laboratory conditions, testing performance against the actual hardware limits, workflow requirements, and quality standards that determine practical viability for resource-constrained developers.

The research question asked: How do accessible mesh simplification methods compare in geometric accuracy, processing speed, visual quality, and ease of use when evaluated within Blender-based game workflows for developers with limited resources?

This study compared four open-source QEM implementations selected for accessibility within Blender-based workflows. The evaluation used three measurement types: geometric metrics, specifically Hausdorff distance and RMSE, enabling comparison with previous research, production metrics including processing speed and memory use testing practical feasibility, and perceptual evaluation measuring visual acceptability during gameplay. This multi-metric approach addressed the documented disconnect between mathematical accuracy and perceived quality (Lin et al., 2023; Liu et al., 2024). Chapter 3 details these evaluation approaches: geometric accuracy measurement, performance benchmarking, and perceptual assessment via pairwise comparison.

The research focused on static mesh simplification for real-time 3D game rendering in Blender 4.5, targeting independent developers and small studios. The comparative evaluation examined traditional geometric methods selected for practical deployment feasibility, with neural approaches investigated separately and excluded after systematic evaluation revealed fundamental accessibility barriers. Neither candidate neural method produced functional output on production-scale game assets: MeshCNN's fixed architecture exceeded available GPU memory by two orders of magnitude, and Neural Mesh Simplification exhibited unresolvable training failures independently confirmed by a third-party researcher (Appendix A). The scope excluded animated meshes, procedural generation workflows, mobile-specific optimization, and large studio contexts with specialized infrastructure. Users were assumed to have basic Python knowledge and access to mid-range gaming computers typical of independent development.

The following chapter reviews the development of mesh simplification from foundational QEM research through neural alternatives, examines how existing evaluation approaches fall short in production contexts, and identifies the specific gaps in accessible tool comparison that this research was designed to address.

Chapter 2: Literature Review

This literature review examines mesh simplification methods accessible to Blender-based game developers. It traces traditional geometric approaches from foundational Quadric Error Metrics to current implementations, explores emerging neural alternatives, and investigates how these methods are evaluated through geometric, production, and perceptual metrics. The review then surveys available open-source implementations and their integration barriers, ultimately identifying critical gaps in systematic comparison, multi-metric evaluation, and production validation that motivate this research. This review examines four key areas: traditional geometric methods and their evolution, emerging neural alternatives, evaluation approaches across geometric, production and perceptual dimensions, and the landscape of available implementations. Together, these reveal critical gaps in systematic comparison that this research addresses.

2.1 Traditional Geometric Methods

Traditional geometric methods use algorithmic approaches to simplify meshes by minimizing mathematical surface error. This section examines Quadric Error Metrics as the established industry standard, Progressive Meshes as a theoretically superior approach that game development struggled to adopt due to runtime constraints, and Blender's Decimate modifier as the primary accessible implementation. These methods establish the performance and quality baseline for evaluating emerging alternatives.

2.1.1 Quadric Error Metrics

QEM has dominated mesh simplification since Garland and Heckbert's seminal 1997 work. Current industry implementations in Simplygon (Donya Labs AB, 2025), InstaLOD (InstaLOD GmbH, 2024), and Unreal Engine (UE) since UE1 version 224 (Unreal Wiki, 2025) demonstrate persistent adoption across production pipelines (Knodt, 2025). QEM therefore represents the baseline against which all accessible alternatives in this research were compared.

QEM represents surface geometry through quadric error matrices, mathematical structures storing plane equations that describe the local surface around each vertex. Each vertex stores plane equations from its neighborhood. During edge collapse operations, which merge two connected vertices into one and remove triangles, these quadrics combine to compute optimal vertex placement (Garland & Heckbert, 1997).

The algorithm repeatedly merges vertex pairs using a priority queue to minimize geometric error, preserving surface features better than earlier methods that considered only local geometry (Cignoni et al., 1998; Garland & Heckbert, 1997). Hoppe (1999) extended the method to preserve appearance attributes including texture coordinates and vertex colors.

Garland & Heckbert (1997) developed QEM at Carnegie Mellon University with a specific design choice: the algorithm minimizes mathematical surface error without considering what humans notice. The method treats all parts of a mesh as equally important, measuring error purely by geometric surface deviation from the original, the mathematical distance between the simplified surface and the original geometry. As Lee et al. (2005) observe, "a purely curvature-based metric may not necessarily be a good metric of perceptual importance." QEM can preserve high-curvature but visually monotonous repeated textures while simplifying geometrically simpler features that players would notice, such as distinctive silhouette edges or facial features. This disconnect between geometric accuracy and visual quality is precisely why this research evaluates QEM-based methods using multiple evaluation approaches: traditional geometric metrics such as Hausdorff distance and RMSE enable comparison with existing academic research, but production workflows targeting player perception require additional evaluation methods beyond pure geometric error.

2.1.2 Progressive Meshes

Hoppe's (1996) Progressive Meshes stored a base mesh plus incremental refinement operations, enabling smooth detail transitions across arbitrary resolution levels. However, game development struggled to adopt Progressive Meshes due to runtime overhead. Progressive meshes require runtime mesh generation through successive vertex split operations, consuming CPU cycles during gameplay (Chen et al., 2006). This runtime mesh reconstruction proved computationally prohibitive for interactive applications where frame budgets must accommodate not only rendering but also physics simulation, collision detection, and game logic.

Most modern game engines adopted discrete LOD systems employing pre-computed detail levels that minimize runtime processing requirements (Chen et al., 2006). Industry implementations commonly generate multiple fixed detail levels during asset preprocessing, switching between pre-computed meshes at runtime based on projected screen size rather than absolute distance, ensuring consistent visual quality regardless of mesh scale (Epic Games, 2025). Specialized applications such as GPU-accelerated crowd rendering demonstrate that progressive techniques remain viable

when computational costs can be offloaded to parallel hardware (Peng et al., 2011), though discrete LOD dominates general game production due to its minimal CPU overhead.

This historical divergence between academic elegance and production adoption directly motivates this research's evaluation approach. Progressive Meshes demonstrated that mathematically sophisticated methods can fail in production contexts due to practical constraints invisible in academic benchmarks. Runtime overhead made the technique impractical despite its theoretical advantages for smooth LOD transitions (Chen et al., 2006). QEM succeeded precisely because it operates during asset preprocessing rather than gameplay, avoiding runtime costs entirely. Emerging neural simplification methods face the same critical test: whether they can achieve practical deployment in production workflows, or whether they will remain academically interesting but production-impractical like Progressive Meshes.

This research tested that question by subjecting neural methods to the same accessibility criteria applied to traditional alternatives, with the investigation outcomes documented in Appendix A.

2.1.3 Blender Implementation

Blender's Decimate modifier implements simplified QEM in "Collapse" mode, representing the primary accessible simplification tool for independent developers and small studios working in open-source workflows. Unlike commercial tools optimized for batch processing and production-scale automation (Simplygon, 2019), Blender prioritizes interactive performance for manual iteration (Blender Foundation, 2025). This design philosophy requires algorithmic approximations that reduce computation time at the cost of optimal quality, a deliberate tradeoff favoring iteration speed over mathematical precision.

While the modifier offers three operational modes, only "Collapse" provides QEM-based edge collapse simplification relevant to automated LOD generation (Blender Foundation, 2025). The remaining modes serve different purposes: reversing subdivision operations and removing coplanar vertices, neither of which addresses the polygon reduction workflow evaluated in this research.

While comparative studies evaluate specific simplification approaches in isolation, such as commercial tools for watermarking applications (Uccheddu et al., 2018), quad-preservation methods within Blender (Knodt, 2025), or neural techniques against single traditional baselines (Potamias et al., 2022). To the best of our knowledge, no systematic research compares multiple accessible QEM implementations within

Blender-based game development workflows or establishes whether emerging neural methods meet the accessibility threshold for inclusion in such a comparison. Technical artists choosing among free QEM variants lack empirical evidence about quality-performance trade-offs evaluated within their actual working environment. For independent developers and small studios who cannot afford commercial tool licenses, this absence of systematic comparison within their production context represents a significant barrier to informed technical decision-making.

2.2 Neural Alternatives to Traditional Methods

Recently, neural approaches have emerged as learnable alternatives to traditional geometric algorithms (Potamias et al., 2022). Unlike QEM's training-free operation on arbitrary meshes (Garland & Heckbert, 1997), neural methods learn simplification strategies from data (Potamias et al., 2022), enabling integration into differentiable pipelines but requiring training datasets.

MeshCNN (Hanocka et al., 2019) introduced edge-based convolutional networks for task-driven simplification. The network learns which edges to collapse based on the objective, with "the deleted edges being the ones whose features contribute the least to the used objective" (Hanocka et al., 2019). Classification networks preserve class-distinguishing features while segmentation networks preserve part boundaries. This task-driven adaptivity (Hanocka et al., 2019) contrasts with QEM's fixed geometric error metric. Potamias et al. (2022) proposed a one-pass approach decomposing simplification into learned vertex sampling, edge prediction via attention networks, and triangle classification, achieving 10× faster processing than QEM (Potamias et al., 2022).

Training data availability has constrained neural approaches. Early methods trained on small datasets (Hanocka et al., 2019; Potamias et al., 2022), orders of magnitude fewer samples than ImageNet's 1.2 million training images (Russakovsky et al., 2015). Researchers adopted unsupervised training with geometric losses (Sharp & Ovsjanikov, 2020) to avoid the expense of manual expert annotation (Koch et al., 2018) and the circularity of training networks to replicate existing algorithms.

This thesis investigated both MeshCNN (Hanocka et al., 2019) and Neural Mesh Simplification (Potamias et al., 2022) for inclusion in the comparative evaluation alongside traditional QEM implementations. Whether these methods met the accessibility threshold for practical deployment in Blender workflows was treated as an empirical question rather than an assumption, with the investigation and its outcomes documented in Appendix A.

Neural approaches also introduce practical and ethical considerations relevant to the target developer population. GPU hardware prerequisites for both training and inference may recreate accessibility barriers through infrastructure costs, potentially disadvantaging developers in regions with limited computing resources. The black box nature of learned strategies complicates diagnostic interpretation when simplification produces unexpected results, as Zhang et al. (2021) observe that deep neural networks "are often found to exhibit unexpected behaviors" with mechanisms that remain poorly understood. Environmental costs, while substantially smaller than the headline figures associated with large language model training (Patterson et al., 2021; Strubell et al., 2019), and authorship questions around learning from training data (Lemley & Casey, 2020) warrant acknowledgment, though their practical impact on mesh simplification at current scales is limited. These considerations are examined further in Sections 3.8 and 5.5.

These neural approaches represent the first significant challengers to QEM-based polygon simplification since Garland and Heckbert's 1997 work achieved industry adoption. The emergence of methods demonstrating competitive quality and dramatic speedups of 10× faster than QEM (Potamias et al., 2022) raises the question of whether neural simplification can achieve practical deployment in Blender workflows for resource-constrained developers. The alternative is that computational requirements and integration complexity render them production-impractical despite theoretical promise, as Progressive Meshes proved to be a generation earlier, as discussed in Section 2.1.2.

2.3 Evaluation Metrics and Benchmarking

Effective evaluation of mesh simplification methods requires multiple measurement approaches. This section examines academic geometric metrics that enable research comparison, production performance metrics that determine practical feasibility, and perceptual quality research that evaluates visual perception. These three categories address different questions of mathematical accuracy, technical viability, and visual quality, demonstrating why no single metric type suffices for comprehensive evaluation. The methodology chapter operationalizes these categories through systematic implementation detailed in Section 3.2 for performance, Section 3.3 for geometric accuracy, and Section 3.5 for perceptual evaluation.

2.3.1 Academic Geometric Metrics

Academic mesh simplification relies on standardized geometric metrics enabling cross-study comparison. Hausdorff distance measures worst-case error: the maximum deviation between meshes (Cignoni et al., 1998). RMSE quantifies average surface deviation by computing squared distances from sampled points on the simplified mesh to their nearest points on the original, providing a complementary measure where isolated outlier vertices do not dominate the score (Cignoni et al., 1998). Additional metrics include QEM as quadric error summation and F-score balancing precision and recall (Knodt, 2025).

These metrics enable direct comparison with existing research, creating common evaluation language (Luebke, 2001). However, they measure mathematical properties rather than perceptual quality. Lin et al. (2023) demonstrate that geometric error metrics "may not faithfully reflect the visual quality" of meshes, with low numerical scores failing to predict visible surface distortion. Liu et al. (2024) conducted user studies in game contexts finding that 82.5% of participants preferred simplification methods preserving semantically significant features like buildings over methods minimizing geometric error that removed recognizable landmarks. This disconnect between geometric accuracy and perceptual quality means geometric metrics alone cannot determine which simplification method a developer should use, despite remaining essential for cross-study comparison.

Standard benchmarks differ substantially from production realities. (Liu et al., 2024) document that 98.9% of ShapeNet meshes contain non-manifold geometry, despite this dataset being widely used for academic research. Comparative evaluations of simplification tools rely on external geometric measurement utilities like Metro rather than built-in tool metrics (Uccheddu et al., 2018), indicating limited integration between production workflows and academic validation approaches.

Despite these limitations, geometric metrics remain essential for comparison. Hausdorff distance and RMSE enable direct comparison with existing QEM implementations and neural method benchmarks, as nearly all recent comparative studies use these metrics (Knodt, 2025; Potamias et al., 2022; Uccheddu et al., 2018). Using these standard metrics allows this research to compare directly against prior studies and adopt their proven evaluation approaches.

2.3.2 Production Performance Metrics

Game production prioritizes practical constraints over mathematical rigor. Processing time matters because LOD generation is typically embedded in iterative asset workflows where even small per-operation delays compound: Okeremi (2025) demonstrated that two-minute reductions in build iteration time save over an hour daily at typical development velocities, a pattern directly applicable to batch simplification of large asset libraries. Studios therefore track performance metrics alongside quality metrics when evaluating production tools (Activision GDC, 2018).

Production tools focus on workflow integration rather than comprehensive geometric analysis. Comparative studies measuring tool performance use external academic utilities for precise geometric error quantification (Uccheddu et al., 2018), suggesting production tools prioritize artist-facing features and performance metrics over detailed geometric validation.

Production metrics address practical questions geometric error cannot answer. A method achieving low Hausdorff distance becomes irrelevant if processing times make batch LOD generation impractical or frame rates suffer during deployment. For Blender-based workflows targeting independent developers with limited hardware and tight schedules, performance metrics covering processing speed, memory consumption, and interactive responsiveness determine whether methods work in practice. However, production metrics measure only technical feasibility, not whether simplified meshes meet player-facing quality standards during gameplay.

2.3.3 Perceptual Quality Research

User studies reveal substantial disconnects between geometric metrics and visual perception. Liu et al. (2024) demonstrated through gameplay-context user testing that participants strongly preferred simplification preserving semantically important features, with their method receiving 82.5% preference over competing approaches including probabilistic quadric error methods. Kim et al. (2010) validated through eye-tracking that computational mesh saliency models correlate significantly with human visual attention at $p < 0.0001$ across all tested models, with fixations clustering near high-saliency regions rather than distributing uniformly.

Both Liu et al. (2024) and Kim et al. (2010) identified that humans prioritize semantically meaningful and geometrically salient features, reinforcing the critique of purely geometric simplification approaches discussed earlier. This perceptual research provides empirical support for incorporating user studies into simplification evaluation.

Perceptual evaluation methodology remains inconsistent. Neither Liu et al. nor Kim et al. address dynamic gameplay involving motion blur, camera movement, or divided attention. Liu et al. used simplified game scenarios while Kim et al. employed controlled static viewing. Whether findings generalize to actual gameplay remains unvalidated. Perceptual studies remain limited with varying methodologies and no standardized evaluation framework (Yildiz & Capin, 2017).

Perceptual quality ultimately matters because players judge appearance, not algorithms. A simplified mesh with excellent geometric scores fails if it shows visible texture stretching or broken silhouettes during gameplay. This is critical for developers without resources for manual cleanup, so automated simplification must produce visually acceptable results directly. This research therefore included perceptual evaluation alongside geometric and production metrics, operationalized through the pairwise comparison methodology detailed in Section 3.5.

2.4 Production Tool Integration

Understanding which simplification methods are available and how easily they integrate into Blender workflows is essential for evaluating their practical viability. This section first surveys the landscape of accessible implementations, documenting fifteen open-source alternatives spanning traditional geometric and neural approaches. It then examines integration barriers that affect both reproducibility and practical adoption, from zero-setup built-in modifiers to complex deep learning frameworks requiring GPU hardware. These practical considerations directly influence which methods can realistically serve independent developers working with limited technical resources and constrained development timelines and inform which algorithms this research will ultimately test in its comparative evaluation.

2.4.1 Evaluated Implementations

This section surveys 15 open-source mesh simplification implementations across traditional geometric and neural approaches, examining their algorithmic foundations, performance characteristics, and integration accessibility relevant to Blender-based game asset workflows. Table 2.1 summarizes all 15 methods identified during the literature and tool survey.

Method	Type	Algorithm	Language	Key Feature
Blender Decimate	Baseline	QEM approx.	Python	Zero setup
meshoptimizer	Traditional	Attribute-aware QEM	C++	20M tri/sec, production-proven
fast-simplification	Traditional	Fast-Quadric QEM	Python/Cython	4-5x faster than VTK
Open3D	Traditional	Standard QEM	Python	ML framework compatible
CGAL	Traditional	Lindstrom-Turk + variants	C++	30yr validation, Test of Time Award
Probabilistic Quadrics	Traditional	Probabilistic QEM	C++	Primitives only, incomplete
libigl	Traditional	QSlm QEM	C++	Functional redundancy
VTK	Traditional	Garland-Heckbert	C++	4-5x slower than alternatives
MeshLab	Traditional	VCGLib QEM	C++	No programmatic API
simplifyjs	Traditional	QEM	JavaScript	Platform incompatible
naver/mesh-simplifier	Traditional	QEM	TypeScript	Web-focused, no Python
astaka-pe/ mesh_simplification	Traditional	Valence-aware QEM	Pure Python	Unvalidated personal project
sp4cerat/Fast-Quadric	Traditional	Fast-Quadric	C++	Component library (wrapped by fast-simplification)
MeshCNN	Neural	Edge-based CNN	Python/PyTorch	750-edge limit, 267x scaling barrier
Neural Mesh Simplification	Neural	Three-stage GNN	Python/PyTorch	Invalid output, memory issues

Table 2.1. Overview of all mesh simplification methods identified during the literature and tool survey, categorized by algorithmic family, underlying algorithm variant, implementation language, and distinguishing feature.

Two patterns emerge from this survey. First, the landscape is concentrated: most implementations are variants of Garland-Heckbert QEM, with only two neural

alternatives identified. Despite this shared algorithmic foundation, installation complexity ranges from zero-setup built-in modifiers to multi-hour C++ compilation or GPU-dependent deep learning frameworks. Second, no study compares these options within a shared evaluation framework, leaving developers to choose based on documentation claims rather than empirical evidence. Section 3.1 details how this list was narrowed to four methods representing different points along the accessibility spectrum.

2.4.2 Integration Barriers

Blender integration barriers vary significantly across mesh simplification approaches. Blender Decimate needs no setup, as users can access it through a simple graphical interface, making it accessible enough for high school students (Shapiro, 2013). Python libraries like PyVista and Open3D require additional installation and configuration steps (Open3D, 2023; Sullivan & Kaszynski, 2019). C++ libraries such as Computational Geometry Algorithms Library (CGAL) and meshoptimizer need build tools and cross-platform compilation, adding complexity when used in Python workflows. Neural methods require deep learning frameworks with specific version requirements, plus GPU hardware (Hanocka et al., 2019).

These barriers directly affect reproducibility. Bonneel et al. (2020) found that only a small portion of published SIGGRAPH graphics methods include working code, and Hernández & Colom (2023) note that when setup becomes too time-consuming, software becomes "de facto non-reproducible." Documenting these barriers therefore served two purposes in this research: enabling reproduction by other researchers and helping developers evaluate whether implementation effort justifies potential improvements over Blender's built-in tools. Appendix A provides empirical measurements including installation time quantification, dependency analysis, and barrier multiplier calculations demonstrating CGAL's 38× time requirement versus pip-installable alternatives.

2.5 Conclusion

This review reveals three critical gaps in mesh simplification research. First, while commercial QEM tools dominate industry pipelines, their prohibitive costs exclude most developers. As established in Section 2.1.3, no research systematically compares multiple accessible QEM implementations and neural methods within Blender workflows, leaving developers without empirical evidence for choosing between free alternatives.

Second, existing evaluation methodologies prove insufficient for production contexts. Lin et al. (2023) demonstrate that standard geometric metrics "may not faithfully reflect visual quality," while Liu et al. (2024) found 82.5% of users preferred results that geometric metrics scored poorly. This disconnect means comparative studies using only geometric metrics, only performance metrics, or only perceptual assessment provide incomplete evidence. No research systematically evaluates simplification methods across all three dimensions simultaneously, leading this investigation to develop multi-metric evaluation addressing geometric accuracy, production feasibility, and perceptual acceptability.

Third, neural approaches remain production-unvalidated. Potamias et al. (2022) report 10× speedups versus QEM, yet no research tests whether neural methods achieve practical Blender deployment. Progressive Meshes demonstrated that sophisticated methods can fail production adoption due to constraints invisible in academic benchmarks. Game engines rejected them despite theoretical elegance due to runtime overhead (Chen et al., 2006). Neural methods face identical tests regarding GPU dependencies, framework overhead, and integration complexity.

These gaps motivate the research question: How do accessible mesh simplification methods compare in geometric accuracy, processing performance, visual perceptual quality, and practical usability when evaluated within Blender-based game asset workflows targeting resource-constrained developers? This investigation compares four open-source QEM implementations selected for practical deployment feasibility, following systematic exclusion of neural approaches that did not meet the accessibility criteria detailed in Appendix A.

Based on the literature, the initial hypothesis was that neural methods would process meshes faster than traditional approaches (Potamias et al., 2022) but would be more difficult to install and configure. Pilot investigation confirmed the latter: neural methods proved inaccessible for the target developer population due to hardware requirements, training instability, and reproducibility failures documented in Appendix A, leading to their exclusion from the quantitative comparison. For the four methods that met the accessibility threshold, the expectation was that standard quality metrics measuring geometric surface error will fail to predict how good simplified meshes look to viewers (Lin et al., 2023; Liu et al., 2024), meaning this research must evaluate methods using multiple criteria to understand their real-world value. The four open-source QEM implementations were further expected to exhibit different quality-performance characteristics from one another, though whether these differences represent improvements or trade-offs remains to be determined through empirical

testing. The following methodology chapter details the systematic evaluation design: method selection with explicit criteria, performance benchmarking, geometric accuracy measurement, production-scale case studies, and perceptual assessment.

Chapter 3: Methodology

This chapter discusses the methodology used to answer the research question:
How do accessible mesh simplification methods compare in geometric accuracy, processing performance, visual perceptual quality, and practical usability when evaluated within Blender-based game asset workflows targeting resource-constrained developers?

To answer this question, this research employed a comparative mixed-methods evaluation testing each method on identical assets under identical reduction parameters. The design balanced internal validity through standardized conditions with ecological validity through production game assets rather than synthetic academic models. Neural approaches were investigated during pilot testing but excluded based on empirical accessibility barriers documented in Appendix A, narrowing the comparison to four traditional QEM implementations. The mixed-methods design reflected the documented disconnect between geometric metrics and perceptual quality (Lin et al., 2023; Liu et al., 2024): no single metric type could answer the research question alone. The evaluation therefore combined performance benchmarking as detailed in Section 3.2, geometric accuracy measurement in Section 3.3, and perceptual assessment via pairwise comparison in Section 3.5.

3.1 Method Selection and Implementation

Python accessibility targets the skill profile of indie developers working in Blender workflows. Stack Overflow Developer Survey (2024) reported Python usage among 51% of all developers compared to 23% for C++, though these figures represent general proficiency rather than game-specific expertise. Pip-installable Python packages require only basic package management, while C++ build systems involving template metaprogramming represent advanced specialization accessible to a smaller developer subset. The accessibility gap between these approaches, while unquantified by industry surveys, is substantial, as documented in Appendix A.4.3. Academic validation establishes baseline quality expectations necessary for comparative evaluation. Methods lacking validated performance benchmarks cannot meaningfully distinguish algorithmic differences from implementation quality issues. The complexity range encompassed production game assets from low-complexity props of 10K triangles to high-complexity models exceeding 250K triangles, while the 10-minute threshold ensured methods

completed within iterative art direction workflows rather than requiring overnight batch processing incompatible with resource-constrained development.

From the literature review, 15 candidate methods were identified. These underwent systematic three-stage filtering based on Python accessibility without C++ compilation, academic validation through peer review or production deployment, and production relevance for handling 10,000 to 100,000+ triangle assets within 10 minutes, as illustrated in Figure 3.1.

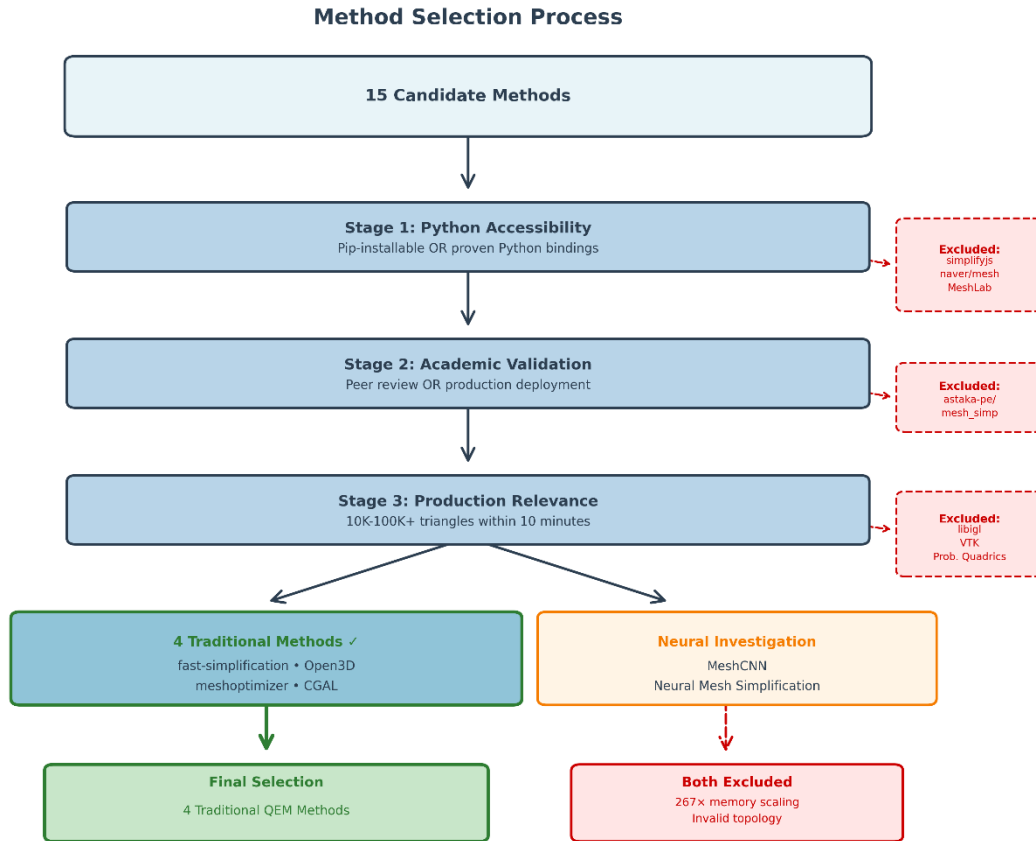


Figure 3.1. Systematic three-stage filtering process from 15 candidate methods to 4 selected traditional QEM implementations, showing exclusions at each stage and neural method investigation results.

This progressively narrowed candidates to four traditional geometric methods, summarized in Table 3.1, and excluded two neural approaches after systematic investigation. CGAL was reconsidered after initial exclusion due to its academic significance as a 30-year development effort and 2023 Test of Time Award recipient (SoCG, 2023).

The three traditional methods, fast-simplification, Open3D, and meshoptimizer, installed via pip in 2-3 minutes with minor API issues resolved through systematic debugging. meshoptimizer pre-release bindings required buffer-based calling

convention discovered through `inspect.signature()` introspection, while fast-simplification compatibility was maintained across PyVista versions via conditional property access. CGAL required C++ subprocess architecture due to incomplete Python bindings, with 62-minute installation of 109 packages and OBJ format standardization after PLY incompatibility was discovered during pilot testing. Pilot validation on two test assets of 69k and 6k triangles confirmed all four methods achieved 100% success rate with stable performance, with Coefficient of Variation (CV) below 15% for 91.7% of tests. Open3D was the exception, exhibiting systematic instability at 50% reduction with CV exceeding 15% across both assets.

Method	QEM Variant	Primary Strength	Installation
fast-simplification	Fast-Quadric (Forstmann, 2014)	Speed and ease of use	Pip, fast
Open3D	Standard QEM (Garland & Heckbert, 1997)	Boundary preservation	Pip, fast
meshoptimizer	Attribute-aware QEM (Kapoulkine, 2025)	Production performance (~20M tri/sec)	Pip, fast
CGAL	Alternative QEM variant (Lindstrom & Turk, 1998)	Comprehensive algorithm library, 30+ year validation	C++ compilation, slow

Table 3.1. Selected methods for comparative evaluation. QEM variant column identifies the underlying algorithm with original authorship. All four methods are accessible through Python, either natively or via compiled bindings. Fast-simplification (Kaszynski, 2025) wraps Forstmann's (2014) C++ implementation.

Neural method investigation revealed fundamental barriers. MeshCNN implemented successfully on SHape REtrieval Contest (SHREC-16) dataset but exhibited fixed 750-edge architectural constraint incompatible with production assets containing 100,000-500,000 edges. Scaling analysis calculated a 1,600 GB GPU memory requirement, 267× the available 6 GB VRAM. This would require a hardware configuration outside of the scope of this study, and typically outside of the scope of professional practice and context of the study.

Neural Mesh Simplification investigation resolved five technical barriers including version incompatibilities, CUDA mismatches, and dense matrix allocation bugs requiring sparse operation rewrite, before exclusion. The empirical scalability limit of 8,500 vertices at 0.6 GB per 1,000 vertices excluded 32% of training dataset. Single-epoch training produced topologically invalid output with holes and 37 disconnected components, preventing geometric accuracy measurement. Hardware requirements of 12-60 GB VRAM for production assets contradicted accessibility objectives and were therefore disqualified as a method for this study.

Four methods were selected for quantitative comparison, achieving comprehensive coverage across QEM variants, performance characteristics, and accessibility levels. Complete technical specifications, barrier measurements, and exclusion rationale are documented in Appendix A.

3.2 Performance Metrics

Performance metrics enabled objective, reproducible comparison of computational efficiency across methods. Four key performance indicators captured computational requirements relevant to resource-constrained developers: processing time, peak memory usage, scalability limits, and success rate. Each metric addressed a specific practical constraint faced by indie developers working with limited hardware and tight schedules.

3.2.1 Processing Time

Processing time was defined as wall-clock duration from method invocation to simplified mesh output, capturing algorithm execution and data structure overhead. Measurement used Python's `time.perf_counter()` for high-resolution monotonic timing. Each asset-method-reduction combination underwent three test repetitions, with mean and standard deviation computed to assess measurement stability. Control variables included identical hardware configuration consisting of an RTX 3060 with 6 GB VRAM, a Ryzen 7 5800H processor, and 32 GB RAM, absence of background processes, and cold cache clearing between runs via garbage collection. Results were reported in seconds for values exceeding one second or milliseconds for values below. Pilot testing on two assets of 69,451 and 6,320 triangles validated measurement stability with coefficient of variation below 15% for 66 of 72 tests, confirming three repetitions provided sufficient precision.

3.2.2 Peak Memory Usage

Peak memory usage was defined as maximum RAM or VRAM allocated during simplification, measured from initialization through completion. Measurement employed hybrid profiling combining `tracemalloc` for Python heap allocations and `psutil` for total process memory, addressing the limitation that pure Python profilers miss C++ library allocations in wrapped implementations. Memory sampling occurred at 0.1-second intervals, capturing peak value across execution. For GPU-accelerated methods, VRAM usage is monitored via CUDA runtime queries. Results were reported in

megabytes for values below 1 GB or gigabytes for values at or above 1 GB. The 32 GB RAM represented the hardware constraint of this research, selected as representative of consumer-grade development hardware accessible to indie developers and students. While professional studio workstations may exceed 64GB, Steam Hardware Survey data indicated consumer gaming hardware clustered around 16-32GB (Valve Corporation, 2025), suggesting indie developers working on consumer hardware likely operate within similar constraints. Methods requiring substantially more memory create accessibility barriers beyond software cost. Pilot testing confirmed `tracemalloc + psutil` hybrid profiling successfully captured memory usage across all four method implementations, though Open3D exhibited a near-zero peak memory delta of 0.001 MB suggesting its C++ allocations may bypass Python profiling. This limitation does not affect comparative conclusions as relative memory requirements remain observable through total process memory measurements.

3.2.3 Scalability

Scalability evaluation employed two testing tiers characterizing both production viability and theoretical limits. Production-scale testing validated methods across typical game asset complexity using Bohemia Interactive assets detailed in Section 3.4, with comprehensive evaluation including performance metrics, geometric accuracy, and perceptual quality. Scalability testing extended evaluation to four Stanford 3D Scanning Repository models spanning 69,451 to 28,055,742 faces, described in Section 3.4 and shown in Figure 3.4, to characterize scaling behavior beyond production requirements (Stanford University, n.d.). Scalability testing measured processing time, peak memory usage, and success rate only, with each configuration executed once rather than three times, reflecting its purpose of establishing feasibility and scaling trends rather than measurement stability. Failure modes including application crash, timeout exceeding 10 minutes, and geometric invalidity were documented to distinguish between memory exhaustion, numerical instability, or algorithmic limitations. This dual-tier approach enabled both practical workflow validation and theoretical capacity characterization, allowing developers to assess method suitability for their specific complexity requirements.

3.2.4 Success Rate

Success rate was defined as the percentage of test attempts producing valid manifold output without crashes, timeouts, or manual intervention. Calculation encompassed complete test matrix: $N_{\text{assets}} \times M_{\text{reduction_levels}} \times 3_{\text{runs}} =$

`total_attempts`. Success criteria required valid mesh with manifold topology, meaning a watertight surface without holes or self-intersections, absence of self-intersections verified via validation libraries, and correct connectivity matching expected reduction ratio. Failure categories were tracked separately: application crashes, invalid non-manifold geometry, processing timeouts, and cases requiring manual parameter adjustment. $\text{Success_rate} = (\text{successful_completions} / \text{total_attempts}) \times 100$. Success rate captured robustness across diverse geometry without manual intervention or special-case handling. Pilot testing demonstrated 100% success rate across all 72 operations spanning 4 methods, 2 assets, 3 reduction levels, and 3 repetitions, with zero crashes or geometric invalidity, validating method robustness on consumer hardware.

3.3 Geometric Accuracy Evaluation

Geometric accuracy was quantified using two complementary metrics: Hausdorff distance capturing worst-case deviation and RMSE capturing average surface error. Hausdorff distance alone is overly sensitive to single outlier vertices, while RMSE alone masks localized failures. Figure 3.2 illustrates how these metrics capture different error characteristics. This dual-metric approach represented industry standard practice (Garland & Heckbert, 1997; Hoppe, 1996; Lindstrom & Turk, 1998). Pilot testing confirmed `trimesh` library successfully computed both metrics across all methods, though `rtree` spatial indexing library installation was required for proximity queries.

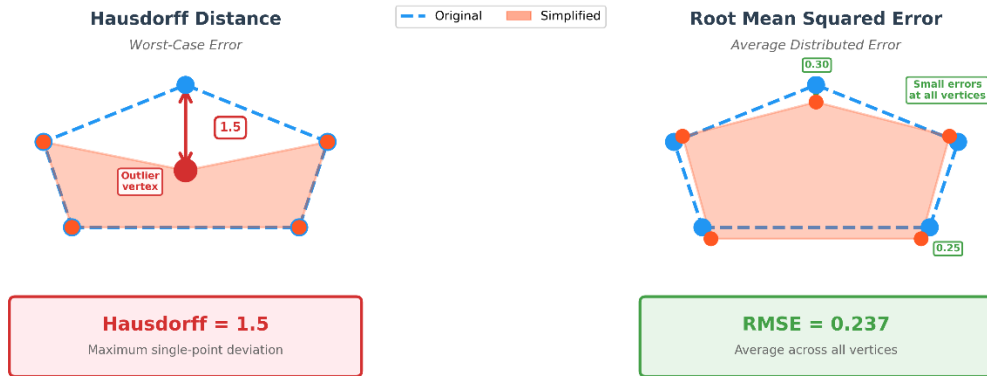


Figure 3.2. Complementary geometric error metrics: Hausdorff distance captures worst-case single-point deviation while RMSE quantifies average distributed error across all vertices.

3.3.1 Hausdorff Distance

Hausdorff distance measures maximum geometric deviation between meshes, capturing the worst-case error at any single point on the surface. Two-way Hausdorff is computed as: $h(A, B) = \max_{a \in A} \min_{b \in B} d(a, b)$, where $H(A, B) = \max(h(A, B), h(B, A))$. This captured catastrophic failures including collapsed geometry, protruding vertices, and topological artifacts.

Measurement used trimesh's `proximity.hausdorff()` function. Point clouds were sampled from both meshes using uniform surface sampling at 10,000 points per mesh. Two-way distance was computed by evaluating directed distances in both directions and taking the maximum. Results were normalized by original mesh bounding box diagonal (Cignoni et al., 1998) and expressed as percentage.

Literature deliberately avoids prescribing universal quality thresholds for geometric error metrics, with Sorgente et al. (2023) confirming that mesh quality assessment is "inherently problem-dependent." Game LOD quality depends on perceptual factors at viewing distances rather than absolute geometric deviation, motivating the multi-metric evaluation approach combining geometric accuracy described in Section 3.3, performance characteristics in Section 3.2, and perceptual quality assessment in Section 3.5.

3.3.2 Root Mean Squared Error

RMSE quantifies overall shape preservation through average squared distances: $RMSE = \sqrt{((1/N) \times \sum d_i^2)}$, where d_i is distance from point i on simplified mesh to nearest point on original mesh. Unlike Hausdorff distance focusing on worst-case deviation, RMSE captured cumulative effect of distributed errors, revealing whether methods preserved overall silhouette and surface characteristics.

Measurement protocol followed identical sampling and normalization procedures as Hausdorff distance. For each point on simplified mesh, distance to nearest point on original mesh was computed using `k-d trees`, spatial data structures enabling efficient nearest-neighbor searches. Results were normalized by bounding box diagonal and expressed as percentage. Together, Hausdorff and RMSE provide complementary geometric accuracy characterization: Hausdorff captured catastrophic local failures while RMSE captured overall shape fidelity.

3.4 Case Study Design

The evaluation employed two test suites serving distinct purposes: a production suite for comprehensive multi-metric evaluation and an academic suite for scalability characterization and reproducibility validation.

The production suite comprised eight game assets from Bohemia Interactive, spanning production-relevant complexity from the 17,515-face M9 Pistol to the 169,087-face Church. Figure 3.3 presents all eight assets with their face counts and geometry categories. The suite covered four geometry types: weapons, specifically the M9 Pistol and AK-74, vehicles including the Jeep, Mi-8, and Light Armored Vehicle (LAV), architectural structures such as the Bunker and Church, and one industrial object, the Watermill. This diversity prevented geometry-specific bias in method comparison while maintaining the hard-surface focus representative of common game asset categories. All eight assets underwent comprehensive evaluation across all three measurement dimensions: performance metrics detailed in Section 3.2, geometric accuracy in Section 3.3, and perceptual quality assessment in Section 3.5. Production assets are provided under non-disclosure agreement restricting use to academic research, which limited full external replication but ensured ecological validity through real production geometry rather than synthetic test models.

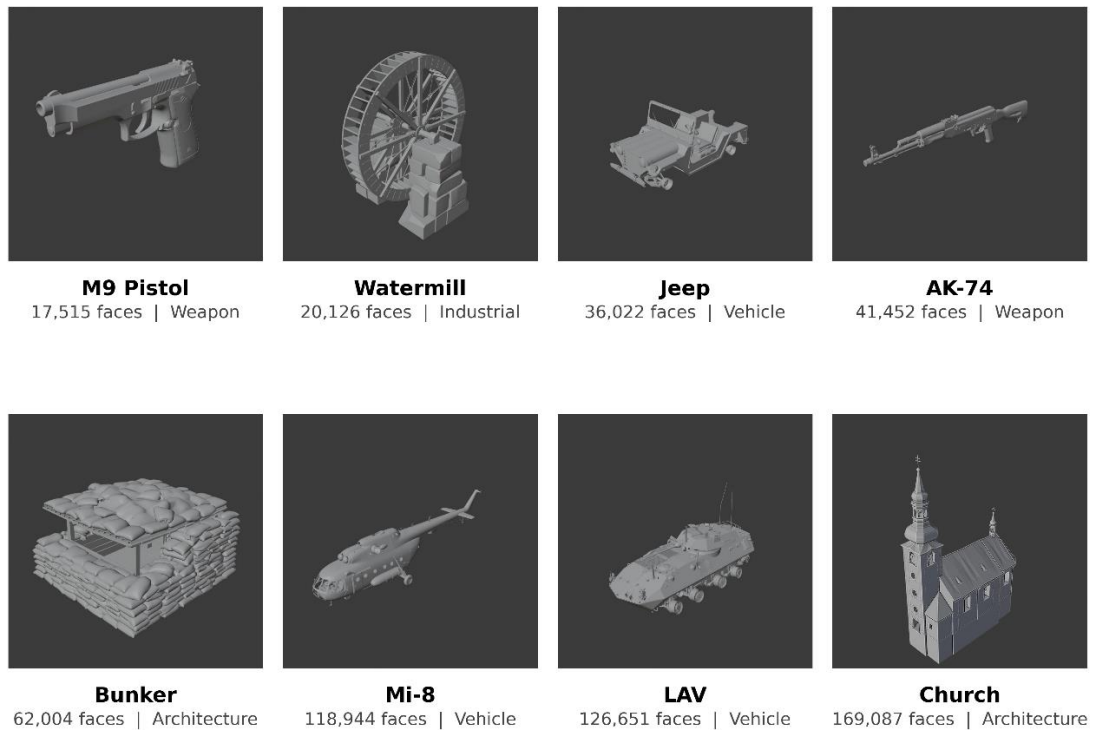


Figure 3.3. Production test suite of eight Bohemia Interactive game assets ranging from 17,515 to 169,087 faces across weapons, vehicles, architectural structures, and industrial objects. Assets are ordered by ascending face count.

The academic suite comprised four Stanford 3D Scanning Repository models (Stanford University, n.d.): the 69,451-face Bunny, the 345,944-face Armadillo, the 871,414-face Dragon, and the 28,055,742-face Lucy, spanning a 404x complexity range. Figure 3.4 presents these models. This suite served two purposes. First, scalability testing extended evaluation beyond production complexity to establish empirical breaking points, measuring only processing time, peak memory usage, and success rate. Geometric accuracy computation and perceptual evaluation are not conducted on these models. Second, reproducibility validation enabled external verification of performance and geometric accuracy results, since the Stanford models are freely available while the proprietary production assets cannot be shared for replication studies.



Figure 3.4. Scalability test suite of four Stanford 3D Scanning Repository models ranging from 69,451 to 28,055,742 faces.

Hard-surface asset prioritization in the production suite provided methodological advantages: well-defined features enable clear feature preservation assessment, regular topology reduces confounding variables compared to organic meshes, and well-defined material boundaries test appearance preservation capabilities. This focus provided a consistent evaluation baseline while acknowledging that organic assets may produce different performance characteristics, noted as a limitation in Section 3.9.

3.5 Perceptual Quality Assessment

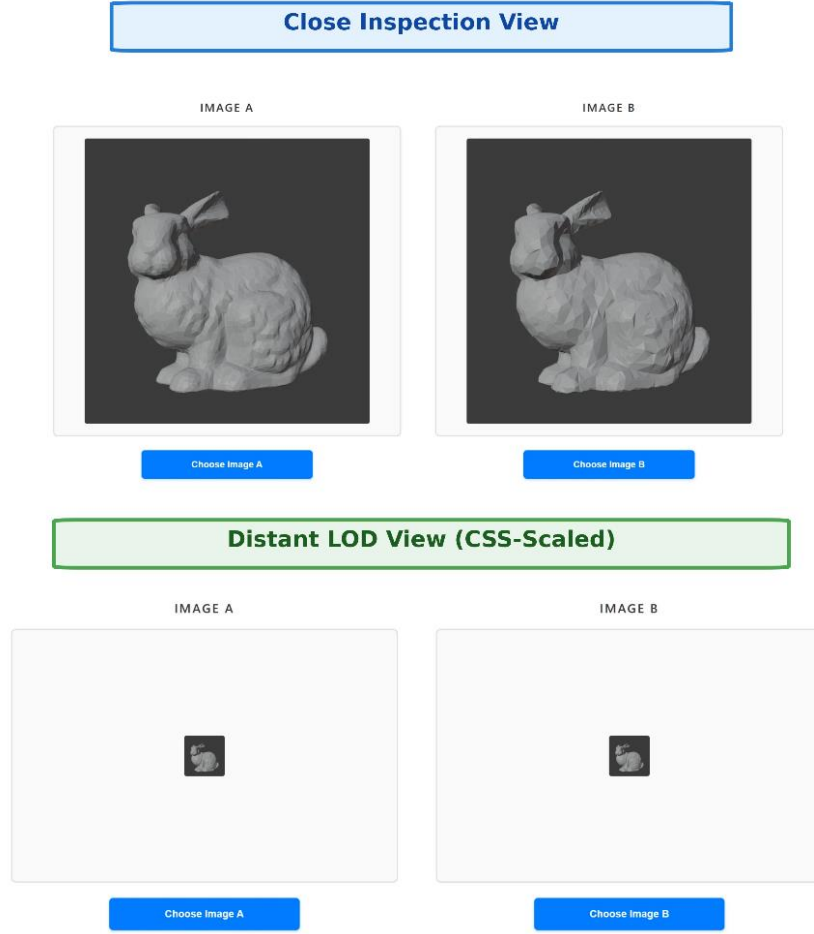
Geometric accuracy metrics measure mathematical surface deviation, but their relationship to human visual perception remains unclear. Prior research suggests potential disconnects between numerical error and perceived quality (Lin et al., 2023; Liu et al., 2024), motivating empirical investigation through perceptual evaluation. This assessment tested whether geometric metrics reliably predict visual quality and identified potential gaps between mathematical accuracy and human perception.

3.5.1 Evaluation Method and Design

Perceptual quality was assessed through pairwise forced-choice comparison using a custom web-based system implementing paired distant-close evaluation. Each method pair was shown twice: once at simulated LOD viewing distance using CSS-scaled images and once at close inspection using full-size images, with view order randomized 50/50 to prevent order effects. This design tested whether method performance differed between gameplay viewing distances and close inspection. A custom implementation was developed to enable paired comparison logic and session-based adaptive sampling unavailable in commercial platforms, while maintaining the thesis accessibility focus through exclusively free technologies: Flask, Python, and PythonAnywhere. Pairwise comparison was selected over Likert scales because forced-choice judgments reduce cognitive load and prevent rating inflation (Mantiuk et al., 2012). The system implemented two-level balanced adaptive sampling prioritizing underrepresented model-reduction combinations and method pairs, maintaining balanced data coverage despite incomplete participant sessions. Perceptual evaluation was conducted exclusively on the eight production game assets to maintain ecological validity. Academic standard models serve only scalability and reproducibility purposes documented through automated metrics.

3.5.2 Visual Stimuli Preparation

Visual stimuli consisted of side-by-side rendered images showing identical assets simplified by different methods at identical reduction levels, as shown in Figure 3.5. Rendering used Blender 4.5 LTS with Eevee engine, perspective camera with per-asset adjusted focal length, three-point lighting at fixed intensities, and neutral 50% gray background. Distance simulation employed CSS scaling determined by reduction level: 90% reduction displays at 8% of frame simulating far LOD3 distance, 80% reduction at 15% simulating LOD2, and 50% reduction at 25% simulating medium LOD1. Close-up view presented full resolution. Stimuli showed only neutral "Image A" and "Image B" labels without method identification or technical context to prevent bias.



Each method pair evaluated at both viewing distances per session • View order randomized • Forced-choice selection

Figure 3.5. Perceptual quality assessment interface showing paired distant-close evaluation design where participants evaluate each method comparison at both simulated LOD viewing distance, shown in the bottom row, and full-resolution close inspection, shown in the top row.

3.5.3 Participant Recruitment and Data Collection

Target sample size was N=30-50 participants recruited from the game development industry, requiring participants to be aged 18 or older with English proficiency and basic 3D graphics familiarity. This sample size was determined through coverage analysis: 30 participants completing 10 pairs each yielded 300 pair selections distributed across 36 unique pairs, providing approximately 8-9 observations per pair for each viewing distance. This coverage aligned with established perceptual quality studies in computer graphics, where Kim et al. (2010) used 24 participants and Liu et al. (2024) used 30, while remaining feasible within thesis constraints. Additionally, the participants were prompted to complete the survey more than once over multiple days time span, which compensates for the increased number of assets in the actual test. Each

session presented 10 pairs shown at both viewing distances for 20 total comparisons taking approximately 4-5 minutes, with sessions terminating after 10 pairs as pilot testing indicated this length maintained participant engagement with 90% completion rate. The paired distant-close design introduced additional cognitive load compared to single-view evaluation, supporting this session length despite technical feasibility of longer sessions. Data collection implemented privacy-preserving design collecting only pairwise choices, reaction times, timestamps, and view type. No personally identifiable information, IP addresses, or cookies were recorded. Session identifiers were randomly generated per page load with no cross-session tracking.

Participants were recruited through direct outreach within the researcher's academic and professional networks, including students and staff at Breda University of Applied Sciences, Howest University of Applied Sciences and industry contacts at Bohemia Interactive. The study link was distributed with a brief description of the research purpose and estimated completion time, with no compensation offered for participation.

3.5.4 Pilot Testing

The evaluation system underwent pilot testing with 5 participants prior to full deployment. Pilot testing confirmed three design objectives: adaptive sampling maintained balanced coverage across method pairs and reduction levels, the paired distant-close evaluation logic presented comparisons correctly at both viewing distances, and session length of 10 pairs totalling 20 comparisons maintained participant engagement with a 90% completion rate. Mean session duration of 4.2 minutes with Standard Deviation (SD) of 0.8 confirmed the time estimates established during design.

One technical issue was identified and resolved during pilot deployment: the Flask backend on PythonAnywhere required CORS header configuration to allow cross-origin image loading in participant browsers, without which rendered comparison images failed to display. Additionally, a pilot participant suggested refining the CSS-based image scaling for the distant viewing condition to better simulate gameplay LOD distances across the three reduction levels. This refinement was validated with the project supervisor and implemented before data collection began. No pilot data was included in the final dataset, and no usability barriers were reported.

3.6 Data Analysis

Data analysis employed descriptive statistical methods to characterize performance patterns and relationships between evaluation dimensions. Descriptive statistics including mean, median, standard deviation, range, and interquartile range were computed for all performance metrics and geometric accuracy measures across the complete test matrix. Analysis distinguished between comprehensive production asset evaluation covering all metrics and limited scalability characterization covering processing time, memory usage, and success rate only for Stanford models.

Results were organized to enable multi-level analysis: per-method performance aggregated across all assets and reduction levels, per-reduction-level trends showing quality degradation with increased simplification, and per-asset-complexity patterns revealing scaling characteristics. Boxplots visualize performance distributions with boxes indicating interquartile ranges, medians as central lines, and whiskers extending to $1.5\times$ interquartile range, with outliers plotted individually. This visualization exposed distribution shape, skewness, and reliability patterns that bar charts with error bars would mask, while tabulated descriptive statistics provide numerical comparison.

Perceptual evaluation data tested whether geometric accuracy predicted visual quality through separate analyses for distant and close viewing conditions. Spearman's rank correlation ρ between perceptual rankings and the two geometric metrics, Hausdorff distance and RMSE, was computed for each viewing distance. Strong correlation with $|\rho|$ exceeding 0.7 would indicate geometric metrics predict perceptual quality, while weak correlation would reveal gaps between mathematical accuracy and visual perception.

3.7 Technical Environment

The study employed Python-based tools prioritizing accessibility and reproducibility. Three isolated conda environments managed version compatibility across different simplification implementations, with complete dependency specifications preserved in `environment.yml` files enabling exact reproduction.

Primary testing environment integrated four mesh simplification methods: Fast-simplification accessed through the PyVista library, alongside Open3D, meshoptimizer, and CGAL. PyVista provided unified mesh I/O and visualization capabilities across methods. Supporting libraries included NumPy for numerical operations, trimesh for geometric accuracy computation of Hausdorff distance and RMSE, pandas for results organization, psutil for memory profiling, and matplotlib for statistical visualization.

CGAL required standalone C++ executable compiled with CMake and Boost libraries due to incomplete Python bindings for its Surface Mesh Simplification module, as documented in Section 3.1 and Appendix A.3.1.

Neural method environments supported investigation of MeshCNN and Neural Mesh Simplification approaches, requiring separate PyTorch installations with CUDA support. These methods were ultimately excluded from comparison due to accessibility barriers documented in Appendix A.2, though environments remain reproducible via preserved configuration files.

Perceptual quality assessment stimuli were generated using Blender 4.5 LTS with Eevee rendering engine selected for efficiency while maintaining sufficient visual quality for comparative evaluation. Rendering parameters included standardized camera positioning, three-point lighting configuration, and neutral background as detailed in Section 3.5.

The hardware configuration consisted of an NVIDIA GeForce RTX 3060 Laptop GPU with 6GB VRAM, AMD Ryzen 7 5800H 8-Core processor, 32GB DDR4 RAM, and 2TB Samsung 990 EVO Plus NVMe SSD storage running Windows 10 operating system. This configuration represented consumer-grade hardware typical of resource-constrained independent developers, aligning with research objectives targeting accessible workflows rather than professional studio infrastructure.

Beyond technical specifications, several ethical considerations shape the research design to ensure integrity and participant protection.

3.8 Ethical Considerations

Ethical considerations were addressed throughout the study to ensure research integrity and participant protection. Asset usage respected intellectual property rights through proper licensing, where production assets from Bohemia Interactive are secured via non-disclosure agreement restricting use to academic research. Academic standard models from Stanford 3D Scanning Repository were used under their attribution license for scalability testing and reproducibility validation. This dual-pathway approach balances ecological validity with research transparency, though proprietary asset use limits full replication by external researchers.

Perceptual quality study implemented privacy-preserving data collection recording only task-relevant data: pairwise choices, reaction times, timestamps, and view type, whether distant or close. No personally identifiable information, IP addresses, browser fingerprints, or session cookies were collected. Session identifiers followed the format `p_[timestamp]_[random]` and regenerated with each page load, preventing

cross-session participant tracking. Data was stored on PythonAnywhere secure hosting with access restricted to the researcher and was archived.

Asset content was evaluated for potential participant impact and cultural sensitivity. Production assets from military simulation contexts included realistic depictions of weapons, armored vehicles, and military equipment alongside neutral architectural and mechanical elements. While the perceptual study focused exclusively on geometric simplification quality through decontextualized side-by-side comparison without narrative framing, participants encountered military subject matter during evaluation. Asset selection prioritized mechanical and architectural diversity over specific equipment types where possible. Participants were informed via study introduction that assets originate from military simulation games, enabling informed consent. The study design presented assets as technical geometry evaluation tasks rather than contextual scenarios, mitigating potential discomfort while acknowledging that individual participants may hold varied perspectives on military imagery. No assets depicting violence, casualties, or combat scenarios were included.

Environmental impact from the 216-288 computational testing executions totalled approximately 15-20 kWh. Testing was conducted on local hardware for cost-efficiency and practical accessibility rather than cloud infrastructure. The full scripting repository developed during implementation was uploaded to GitHub, allowing easier future reproducibility.

This research utilized large language model assistance from Anthropic's Claude for iterative writing refinement, data analysis script development, and visualization generation throughout the thesis process. LLM usage introduced a cost consideration: subscription-based AI tools represent a recurring expense that may not be accessible to all researchers. At approximately \$20 per month over the research period, this cost is modest relative to commercial mesh simplification licenses discussed in this thesis, but warrants acknowledgment as a resource dependency. All analytical decisions, methodological choices, and interpretations remain the researcher's own. LLM-generated code was validated through manual review and empirical testing before use in data collection or analysis.

3.9 Limitations and Mitigation Strategies

Several potential limitations affect the methodology's generalizability. Testing was conducted on consumer-grade laptop hardware consisting of an RTX 3060 with 6 GB VRAM, a Ryzen 7 5800H, and 32 GB RAM, representing the research hardware

constraint and accessibility focus targeting resource-constrained indie developers and students. Results remain specific to this hardware configuration.

Open3D exhibited performance instability during pilot testing, with 50% reduction showing high variance with CV of 15-19% across both test assets despite 100% success rate. This pattern may reflect internal adaptive algorithms or caching behavior not documented in the API. The instability was isolated to 50% reduction with other reduction levels of 80% and 90% demonstrating stable performance, suggesting reduction-specific rather than fundamental robustness issues.

Production asset evaluation focused on 6-8 hard-surface game models representing a limited sample of asset types. Asset selection criteria prioritized diversity across triangle count ranges of 10K to 150K, geometry types, and topology characteristics. The hard-surface focus may not reflect organic character or foliage asset performance. Academic standard models such as the Stanford Bunny, Armadillo, and stress testing meshes provide reproducibility validation and scalability characterization but differ geometrically from game production assets. These organic sculptural models test algorithmic robustness rather than production workflow applicability. Each asset is documented with comprehensive metadata enabling future extension to additional types.

Neural mesh simplification methods were excluded from quantitative comparison, limiting analysis to traditional QEM approaches. Extended investigation totaling approximately 40-50 hours across both methods provided empirical evidence justifying exclusion: MeshCNN's fixed 750-edge architectural constraint required 267x available VRAM for production assets, while Neural Mesh Simplification exhibited quadratic memory scaling of 0.6 GB per 1,000 vertices, a gradient flow obstruction preventing face classifier training, and topologically invalid inference output. Independent corroboration from the third-party implementation author confirmed identical reproducibility failures documented in Appendix A. These findings themselves constitute a research contribution, empirically validating the thesis premise about accessibility barriers in neural geometry processing. Rapid advances in neural mesh processing may render these specific barriers obsolete, and Section 6.4 identifies re-evaluation as a future research direction.

Perceptual study participants recruited from game development industry may exhibit different sensitivities than general players due to professional exposure to visual artifacts. Pairwise comparison methodology reduces rating bias compared to absolute scales. Correlation analysis tested whether developer preferences aligned with geometric accuracy. The perceptual evaluation focused on method comparisons at

identical reduction levels, controlling reduction ratio as a variable to isolate method quality differences. Cross-level comparisons, for example comparing 50% reduction of one method versus 80% reduction of another, could reveal whether superior methods at higher reduction levels produce quality comparable to inferior methods at lower reduction levels, though such analysis would substantially increase required comparison types from 72 to 168 beyond Block B scope. The evaluation design prioritized within-level method comparison to directly address the research question of which accessible method produced best quality at equivalent reduction ratios.

Software versions tested will receive updates potentially invalidating reproduction or benchmark conclusions. Exact versions are documented comprehensively with conda environment exports preserving complete dependency configurations enabling recreation of identical software environments. Testing framework employed Python with potential overhead not representative of direct C++ library performance. Profiling isolated algorithm execution from I/O operations, and framework overhead remained consistent across methods enabling valid relative comparison. The approach reflected real Blender-based workflows typical of target audience, improving ecological validity despite potential differences from C++ engine integration.

Chapter 4: Data and Results

This chapter presents empirical results from benchmark testing and perceptual evaluation conducted across four mesh simplification methods. The research procedure followed the methodology described in Chapter 3 without deviation. One measurement limitation emerged during execution: Resident Set Size (RSS)-based memory profiling produced artifacts on fast-executing methods, documented with contextual annotations in Section 4.1.2.

Benchmark testing executed 288 simplification operations across the production suite shown in Figure 3.3: eight game assets processed by four methods, namely fast-simplification, Open3D, meshoptimizer, and CGAL, at three reduction levels of 50%, 80%, and 90% with three repetitions per configuration. All 288 operations completed successfully with zero crashes, timeouts, or geometric invalidity, achieving 100% success rate. Timing measurements exhibited low coefficient of variation across repetitions with mean CV of 2.2-5.1% per method, with only 8 of 96 configurations exceeding 10% CV, confirming that three repetitions provide stable performance estimates. Perceptual evaluation collected 2,769 pairwise preference judgments from 150 participants across two viewing conditions. This chapter reports data from both evaluation streams without interpretation. Discussion of findings, implications, and comparison with existing research follows in Chapter 5.

All measurement tools operated as specified in Sections 3.2 and 3.7. Data collection, analysis, and visualization were conducted through custom Python scripting, with benchmark automation via the batch testing framework and geometric accuracy computed using trimesh. The complete scripting repository will be published on GitHub following thesis defense to enable external reproduction.

Results are organized by measurement dimension. The following sections discuss processing performance, geometric accuracy, reduction accuracy, scalability characterization, perceptual quality, and the relationship between geometric metrics and perceptual judgments. Summary statistics tables and aggregate figures appear in this chapter. Per-asset detail tables and per-reduction breakdowns are provided in Appendix B for reference.

4.1 Processing Performance

Processing performance encompassed execution time and memory consumption across the complete test matrix. These metrics address practical feasibility for resource-

constrained developers: whether methods complete within iterative workflow timeframes and operate within consumer hardware memory limits.

4.1.1 Processing Time

Processing time varied by two orders of magnitude across methods. Table 4.1 summarizes descriptive statistics aggregated across all assets and reduction levels. Meshoptimizer was the fastest method with a mean processing time of 23.82 ms, while CGAL was the slowest at 1,479.60 ms, confirming the two-order-of-magnitude spread. Fast-simplification and Open3D occupied intermediate positions as shown in Table 4.1. Fast-simplification exhibited the highest variability, with a standard deviation exceeding twice its mean, driven by the 169,087-face church asset where processing time reached 1,253.59 ms at 80% and 90% reduction levels.

	Mean (ms)	Median (ms)	Std (ms)	IQR (ms)	Min (ms)	Max (ms)
Fast-Simplification	159.04	54.08	334.15	93.84	10.88	1253.59
Open3D	469.35	304.62	424.09	746.94	50.37	1502.07
Meshoptimizer	23.82	16.02	19.06	31.49	4.62	62.61
CGAL	1479.6	992.85	1083.14	2041.01	323.66	3816.52

Table 4.1. Processing time descriptive statistics across all assets and reduction levels. Values represent wall-clock duration from method invocation to simplified mesh output.

Figure 4.1 shows the distribution of processing times across methods. The top panel displays the full range, where CGAL dominates the vertical scale. The bottom panel provides a zoomed view of the three faster methods, revealing that meshoptimizer and fast-simplification operate below 200 ms for most configurations while Open3D spans a wider range from approximately 50 ms to 1,500 ms.

Processing time scales with input complexity across all methods, though at different rates. Figure 4.2 shows time plotted against input face count at each reduction level. CGAL exhibits the steepest scaling gradient, rising from approximately 330 ms at 17,515 faces to 3,816 ms at 169,087 faces. Open3D follows a similar but more gradual trajectory, reaching approximately 900 ms at the highest complexity under 50% reduction and scaling further at higher reduction levels. Meshoptimizer maintains near-flat scaling below 100 ms across all tested complexities and reduction levels. Fast-simplification remains below 200 ms for most configurations but exhibits a sharp increase at the highest complexity on the church asset, particularly at 80% and 90%

reduction levels where it reaches approximately 1,250 ms. This acceleration coincides with the simplification ceiling documented in Section 4.3.

Per-reduction breakdowns confirm that processing time increases with reduction level for all methods except meshoptimizer, which maintains consistent performance regardless of target reduction. Detailed per-reduction comparison tables are provided in Appendix B.

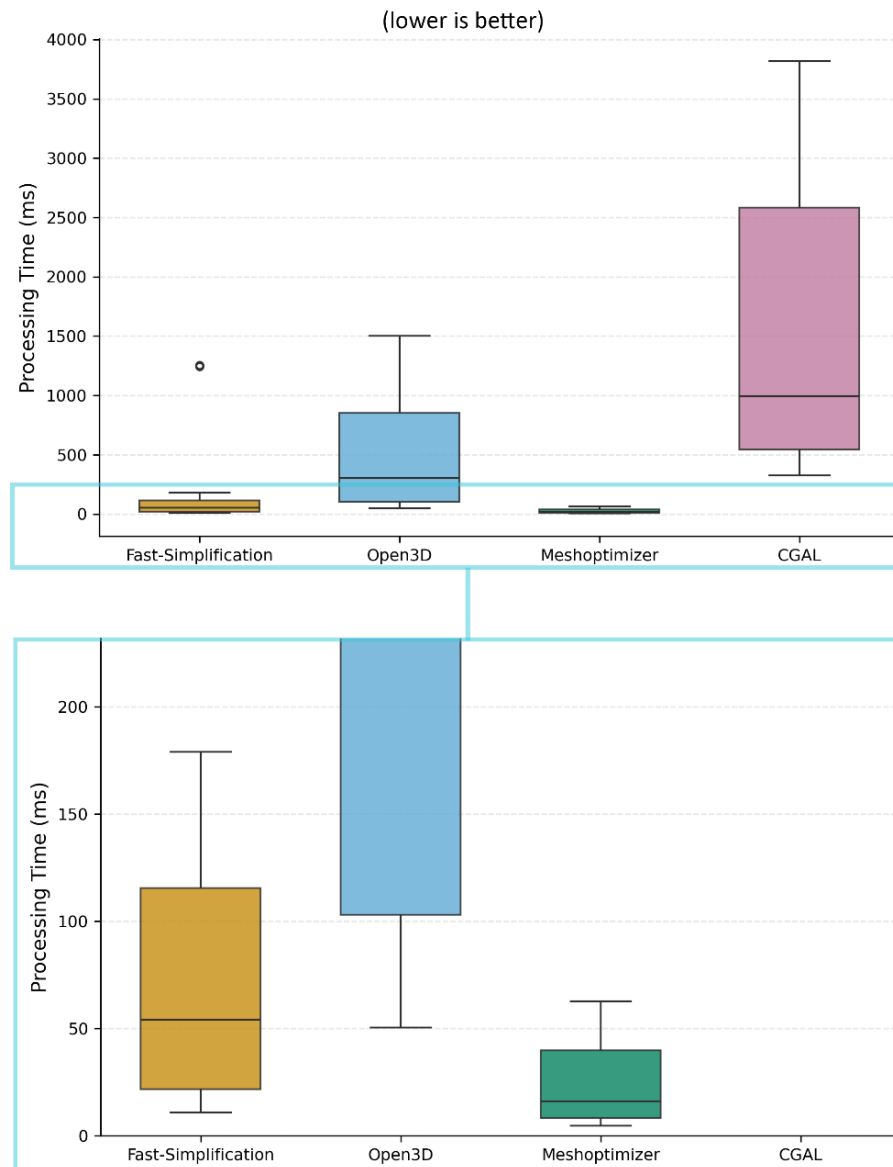


Figure 4.1. Processing time distribution by method. Top: full range showing all four methods. Bottom: zoomed view of fast-simplification, Open3D, and meshoptimizer with CGAL excluded for scale clarity.

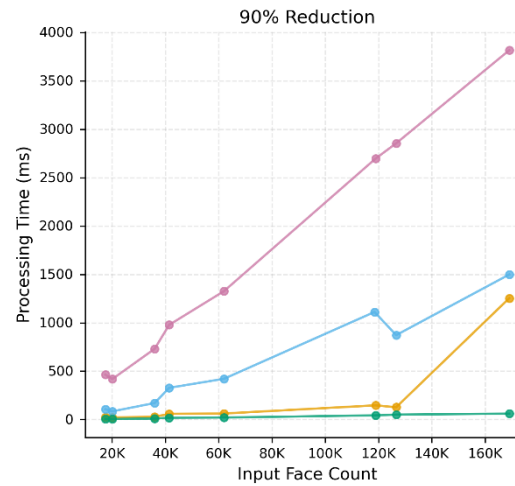
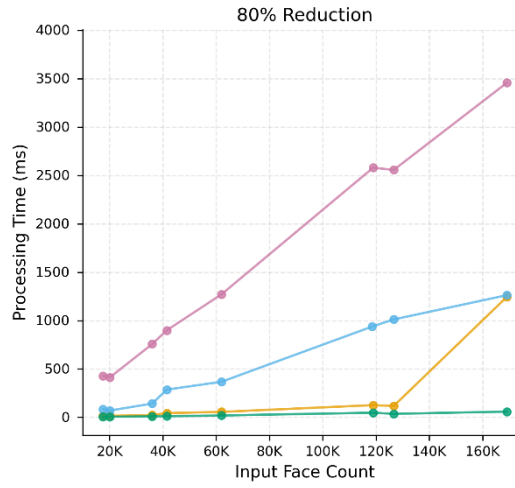
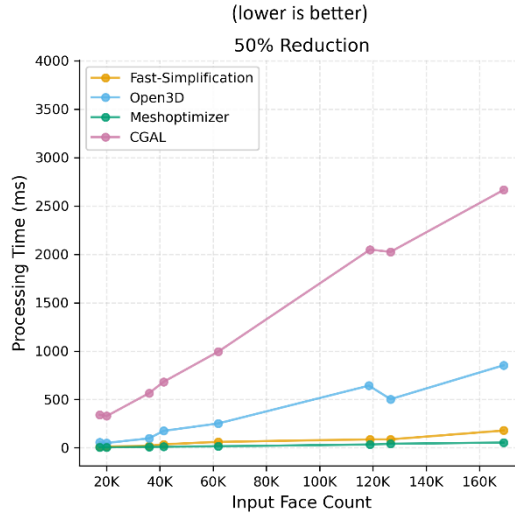


Figure 4.2. Processing time versus input face count at 50%, 80%, and 90% reduction levels. Each point represents the mean of three repetitions for one asset-method combination.

4.1.2 Memory Usage

Memory consumption was measured as the change in process RSS during simplification. Table 4.2 presents descriptive statistics. Meshoptimizer exhibited the lowest memory footprint with a mean delta of 1.57 MB and the highest measurement stability at 75.0% of configurations. CGAL required the most memory at 16.38 MB mean, roughly 10× higher, but achieved 100% measurement stability, consistent with its longer execution times providing more RSS polling samples. Fast-simplification and Open3D occupied intermediate positions as shown in Table 4.2, though fast-simplification produced one outlier reaching 38.64 MB on the church asset. Open3D exhibited the lowest measurement stability at 37.5%.

	Mean (MB)	Median (MB)	Std (MB)	Min (MB)	Max (MB)	Stability (%)
Fast-Simplification	3.46	2.67	5.18	-2.25	38.64	66.7
Open3D	2.11	0.41	4.21	-8.35	15.13	37.5
Meshoptimizer	1.57	0.99	1.67	0.0	5.9	75.0
CGAL	16.38	12.89	7.47	8.18	29.95	100.0

Table 4.2. Memory usage descriptive statistics. Values represent RSS delta in megabytes during simplification. Stability percentage indicates the proportion of configurations where coefficient of variation across three repetitions remained below the instability threshold.

Three measurement considerations affect interpretation of memory data. First, Open3D recorded negative memory deltas on multiple configurations with a minimum of -8.35 MB, indicating that RSS decreased during simplification. This likely occurs when the operating system reclaims memory from other allocations between the pre-measurement and post-measurement sampling points, a known limitation of RSS-based profiling on fast-executing operations. Second, CGAL memory values reflect Python-side subprocess overhead from process communication and result parsing rather than the C++ algorithm's actual consumption, as documented in Section 3.1. CGAL's memory figures are therefore not directly comparable to the other three methods on an algorithmic basis. Third, low measurement stability for fast-executing methods at 37.5% for Open3D and 66.7% for fast-simplification likely reflects RSS polling granularity rather than genuine behavioral variance: methods that complete in under 100 ms provide fewer polling samples, which can produce higher coefficient of variation across repetitions. All data is retained with these contextual considerations rather than excluded.

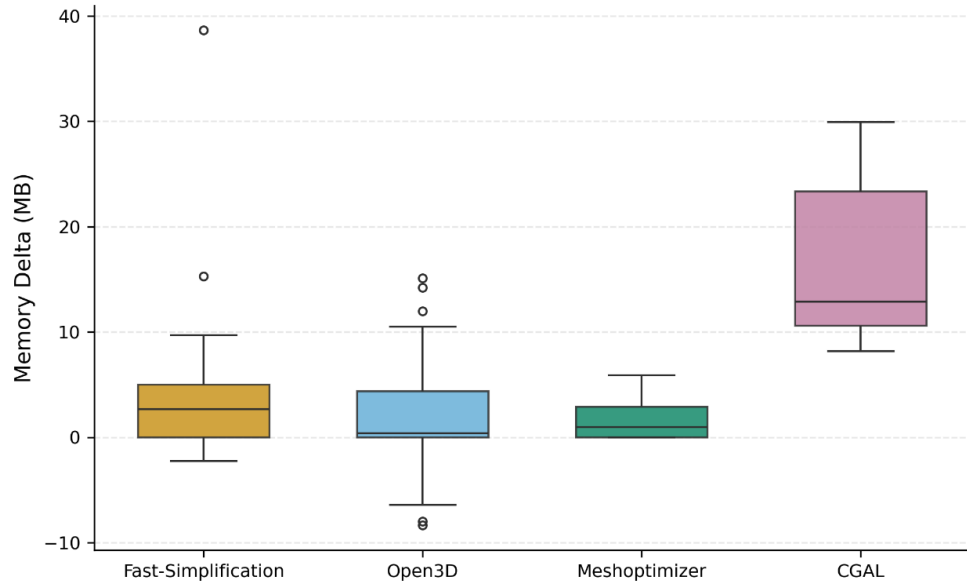


Figure 4.3. Memory usage distribution by method. CGAL shows consistently higher consumption. Open3D exhibits negative deltas on several configurations, an artifact of RSS polling granularity on fast-executing operations.

4.2 Geometric Accuracy

Geometric accuracy captured how faithfully simplified meshes preserve the original surface geometry. Two complementary metrics are reported: Hausdorff distance measuring worst-case deviation at any single point, and RMSE measuring average surface error across all sampled points. Both metrics are normalized to the original mesh bounding box diagonal and expressed as percentages, enabling comparison across assets of different physical scales.

Table 4.3 summarizes geometric accuracy across all assets and reduction levels. Open3D achieved the lowest error on both metrics, with mean Hausdorff distance of 0.88% and mean RMSE of 0.045%. CGAL and meshoptimizer produced similar accuracy, with mean Hausdorff values of 3.07% and 2.70% respectively. Fast-simplification exhibited the highest error on both metrics, and notably the smallest gap between its mean and median Hausdorff values of 2.88% versus 2.85%, indicating consistently elevated error across assets rather than outlier-driven inflation. By contrast, Open3D's median Hausdorff of 0.43% sits well below its mean of 0.88%, suggesting that a few difficult assets pull its average upward while typical performance is stronger.

	Hausdorff Mean (%)	Hausdorff Median (%)	Hausdorff Std (%)	RMSE Mean (%)	RMSE Median (%)	RMSE Std (%)
Fast-Simplification	2.884	2.8523	1.561	0.2742	0.2264	0.2221
Open3D	0.8823	0.4329	1.1517	0.0451	0.0214	0.0729
Meshoptimizer	2.6984	0.6248	4.9453	0.0783	0.0603	0.0743
CGAL	3.0732	0.5964	4.8851	0.0917	0.0249	0.1679

Table 4.3. Geometric accuracy descriptive statistics across all assets and reduction levels. Values are normalized to bounding box diagonal and expressed as percentage. Lower values indicate better surface preservation.

Figure 4.4 shows the aggregate accuracy distributions. Fast-simplification consistently produced the highest error with wide spread, indicating variable performance across assets. Open3D achieved the tightest distributions with the lowest medians on both metrics. CGAL and meshoptimizer occupy intermediate positions, though both exhibit high-value outliers visible in the Hausdorff panel.

Error magnitude increases with reduction level across all methods. Figure 4.5 presents the 90% reduction results, where method differences are most pronounced. At this aggressive reduction level, Open3D maintains the lowest median Hausdorff at 1.66% and RMSE at 0.099%. CGAL produces the widest Hausdorff spread with SD of 5.67%, driven by extreme values on the LAV and bunker assets where it reaches 15.98% and 13.24% respectively. Meshoptimizer shows similarly extreme outliers on the LAV at 19.48% and Mi-8 at 4.99%. Fast-simplification Hausdorff values remain relatively consistent across assets at a mean of 3.06% with SD of 1.46% but at a higher baseline than Open3D. Equivalent figures for 50% and 80% reduction levels are provided in Appendix B.

A pattern emerges in the per-asset data: methods that achieve low average RMSE error can simultaneously produce high worst-case Hausdorff error on specific assets. For example, meshoptimizer achieves the second-lowest mean RMSE overall but produces the single highest Hausdorff value in the entire dataset at 19.48% on the LAV at 90% reduction. This pattern reinforces the rationale for reporting both metrics. Complete per-asset results are tabulated in Appendix B.

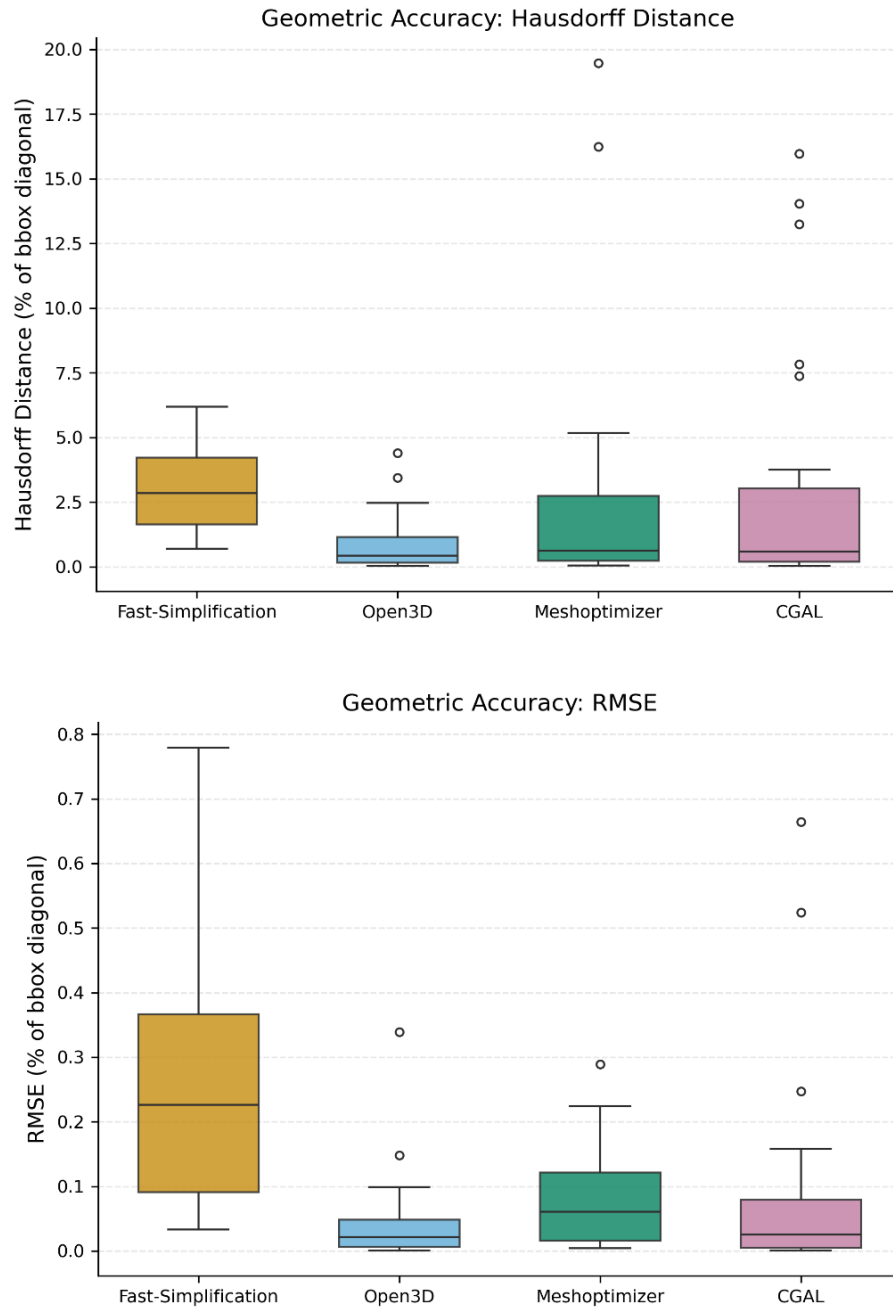


Figure 4.4. Geometric accuracy distribution by method, aggregated across all assets and reduction levels. Top: Hausdorff distance measuring worst-case error. Bottom: RMSE measuring average error. Both metrics normalized to bounding box diagonal.

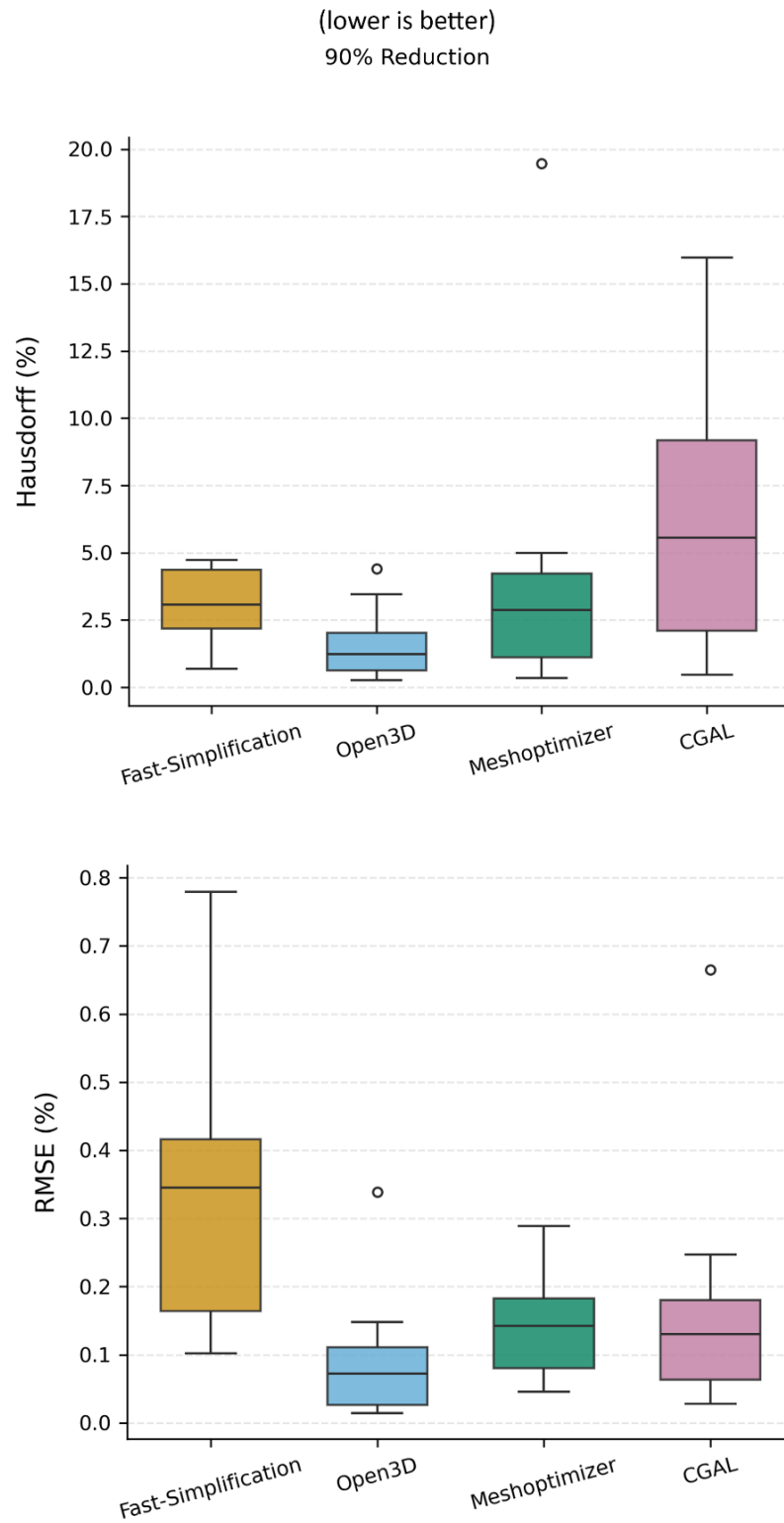


Figure 4.5. Geometric accuracy at 90% reduction level.

Top: Hausdorff distance. Bottom: RMSE.

4.3 Reduction Accuracy

Reduction accuracy measured whether methods achieved the requested polygon reduction target. Figure 4.6 plots actual reduction percentage against target reduction for all 96 asset-method-reduction combinations. Most data points cluster near the diagonal representing perfect accuracy, indicating that all four methods generally achieve their targets.

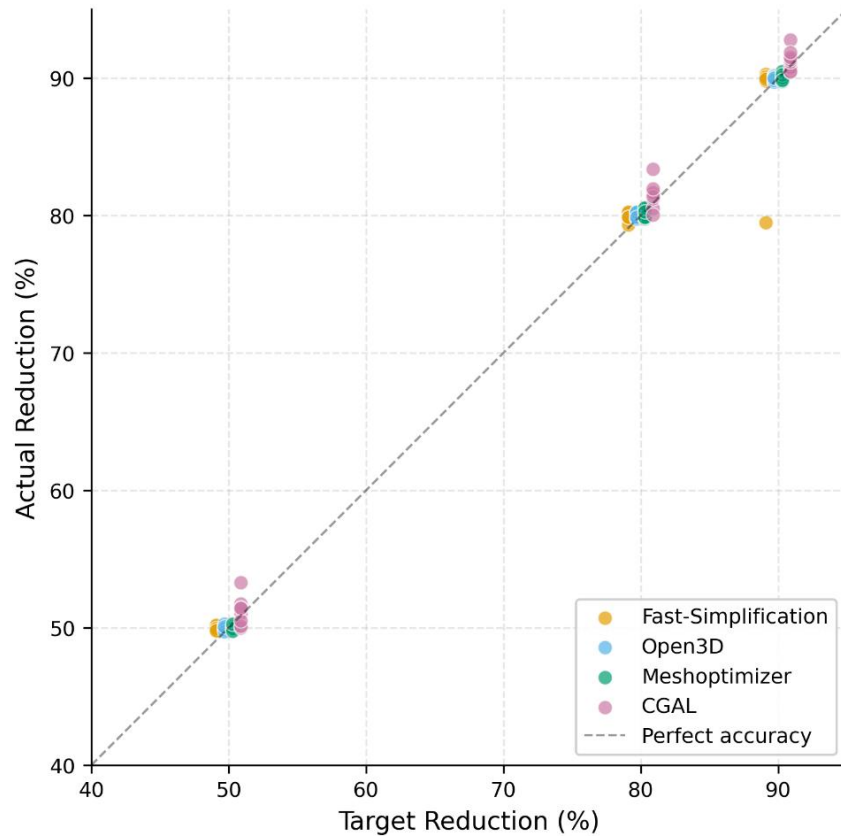


Figure 4.6. Reduction accuracy: target versus actual reduction percentage for all asset-method-reduction combinations. Dashed diagonal represents perfect accuracy. Points above the line indicate over-reduction, while points below indicate under-reduction.

Two systematic deviations appear. First, CGAL consistently over-reduces across all assets and reduction levels. Most assets show 0.2–1.9 percentage point overshooting, but the Mi-8 helicopter exhibits the largest deviations: 53.2% actual at 50% target by +3.2 percentage points, 83.5% at 80% target by +3.5 percentage points, and 93.0% at 90% target by +3.0 percentage points. This over-reduction is consistent and predictable rather than random, occurring across every CGAL configuration in the test matrix.

Second, fast-simplification hits a simplification ceiling on the church asset at 169,087 faces. At both 80% and 90% target reduction, fast-simplification achieves only

79.5% actual reduction, visible in Figure 4.6 as the isolated point well below the diagonal at the 90% target with a -10.5 percentage point deviation. This ceiling does not occur on any other asset or with any other method, suggesting geometry-specific interaction between the fast-quadric algorithm and the church asset's dense architectural topology. The church is the most complex asset in the test suite.

4.4 Scalability

Scalability testing extended evaluation beyond production-relevant complexity using the four Stanford 3D Scanning Repository models presented in Section 3.4, Figure 3.4, spanning a $404\times$ complexity range from the 69,451-face Bunny to the 28,055,742-face Lucy. Testing measured processing time, peak memory usage, and success rate only. Geometric accuracy computation was disabled for this tier, and each configuration was executed once rather than three times. Single-repetition design reflects the purpose of scalability testing: establishing feasibility and scaling trends rather than measurement stability. This deviation from the three-repetition protocol used for production assets in Section 4.1, is documented here for replicability.

All 48 operations spanning 4 models, 4 methods, and 3 reduction levels completed successfully, achieving 100% success rate. No method crashed, timed out, or produced invalid output at any complexity level, including the 28.1-million-face Lucy model.

4.4.1 Processing Time Scaling

Figure 4.7 presents processing time on log-log axes across the full complexity range at each reduction level. Log-log scaling prevents the $404\times$ range in face count from compressing sub-million data to the origin, and reveals each method's scaling class through slope.

Meshoptimizer maintained the fastest processing at every complexity level, scaling from 17 ms on the 69K-face Bunny to 21 seconds on the 28.1M-face Lucy at 90% reduction. Fast-simplification followed a similar trajectory, reaching 31 seconds on Lucy. Both methods demonstrate near-linear scaling across the full range. Open3D and CGAL exhibit steeper scaling gradients. On Lucy, Open3D required 4.7–10.8 minutes depending on reduction level, and CGAL required 8.3–13.4 minutes. At the 90% reduction level, the four methods converge more closely at the highest complexity: the gap between the fastest method, meshoptimizer at 21 seconds, and the slowest, CGAL at 13.4 minutes, narrows to approximately $38\times$, compared to $68\times$ at 50% reduction on the same model.

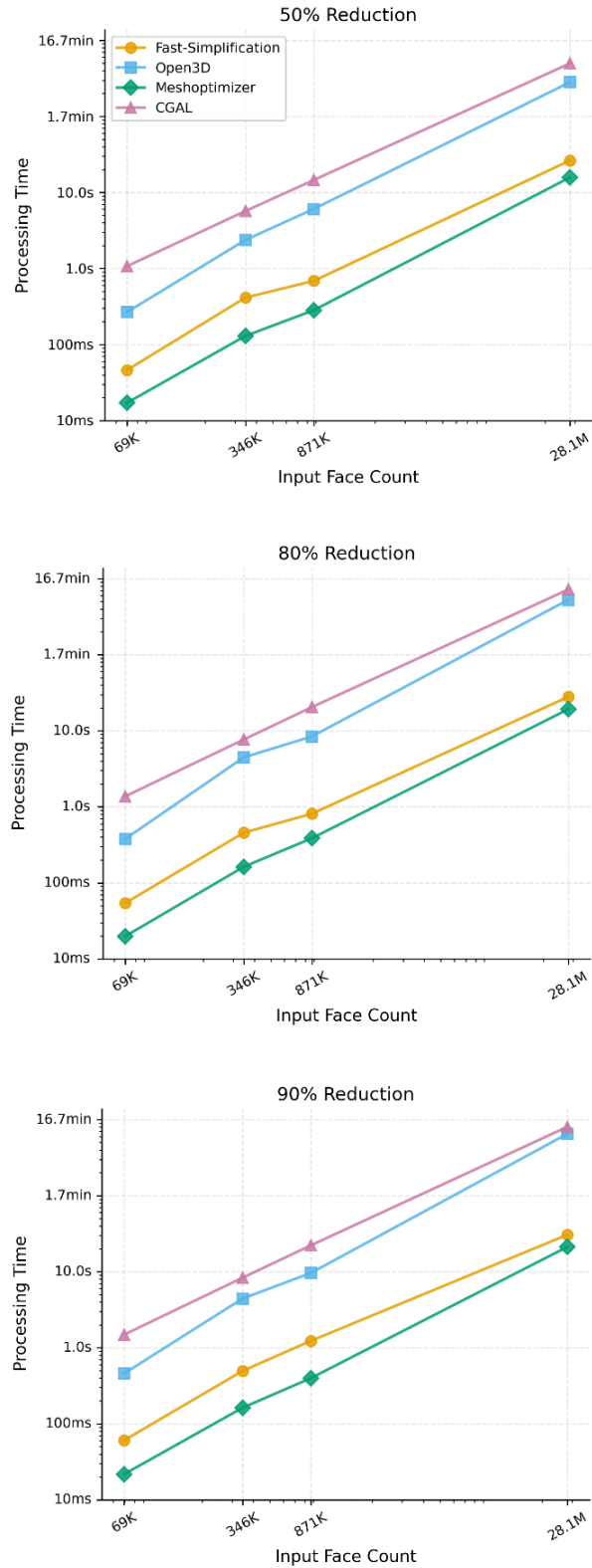


Figure 4.7. Processing time scaling on log-log axes across four Stanford models at 50%, 80%, and 90% reduction levels. Each point represents a single execution. The vertical axis spans from 10 ms to approximately 17 minutes.

4.4.2 Memory at Scale

Memory consumption at the 28.1-million-face scale reveals method-specific resource profiles. Fast-simplification required the highest peak memory at 9.1 GB for 50% reduction, reflecting its in-memory data structure expansion for large meshes. CGAL consumed 4.2 GB peak, consistent across reduction levels. Meshoptimizer required 3.0 GB. Open3D memory could not be reliably measured at this scale: all three Lucy configurations recorded negative memory deltas of 1,139–4,362 MB, indicating that RSS-based profiling breaks down entirely when the operating system manages large memory-mapped allocations during extended operations.

At sub-million complexity from Bunny through Dragon, all methods operated within 200 MB peak memory, well within consumer hardware constraints. The scaling from production-range complexity of 69K to 871K faces to extreme complexity of 28.1M faces produced memory increases of 15× to 46× depending on method, confirming that 28-million-face models require workstation-class memory allocation but remain processable on systems with 16 GB or more available RAM.

4.5 Perceptual Quality Assessment

Perceptual quality was evaluated through pairwise forced-choice comparison, where participants selected which of two simplified meshes appeared closer to the original. The study collected responses across two viewing conditions: distant view simulating gameplay LOD distances, and close inspection at full resolution. Table 4.4 summarizes the study parameters.

The study recruited 150 participants, of whom 133 completed full sessions of 20 comparisons each and 17 completed partial sessions. All 2,769 responses are included in the analysis. responses cover all 144 unique pairwise comparisons across 8 assets, 3 reduction levels, 6 method pairs, with a median of 10 evaluations per unique pair.

Median response time was 6.0 seconds per comparison, with 5.2 seconds for distant and 6.8 seconds for close viewing, consistent with deliberate visual evaluation rather than random selection. Twelve responses, representing 0.4% of the total, were recorded in under one second, and 44 responses, representing 1.6%, exceeded 60 seconds. Both extremes are retained in the dataset, as exclusion would require assumptions about participant intent that cannot be verified from response timing alone.

Parameter	Value
Total participants	150
Complete sessions (20 comparisons)	133
Partial sessions	17
Total responses	2769
Unique pairwise comparisons	144
Assets evaluated	8
Reduction levels	50%, 80%, 90%
Methods compared	4
Viewing conditions	Distant, Close
Evaluations per pair (median)	10

Table 4.4. Perceptual study parameters. Complete and partial sessions are included in analysis.

4.5.1 Overall Perceptual Rankings

Table 4.5 presents overall win rates aggregated across all assets, reduction levels, and viewing distances. Win rate represents the percentage of comparisons in which a method was selected as the preferred result. Meshoptimizer achieved the highest overall win rate at 57.6%, followed by CGAL at 55.9% and Open3D at 54.2% as shown in Table 4.5. The top three methods are separated by only 3.4 percentage points, with all three performing above the 50% chance-level threshold. Fast-simplification was selected least frequently at 32.3%, substantially below the other methods and consistently below chance level.

Method	Overall Win Rate	Overall Wins	Distant Win Rate	Close Win Rate	Δ (Close – Distant)
Fast-Simplification	32.3%	448/1385	34.7%	30.0%	-4.8pp
Open3D	54.2%	762/1406	53.4%	55.0%	+1.7pp
Meshoptimizer	57.6%	791/1373	58.6%	56.6%	-2.0pp
CGAL	55.9%	768/1374	53.4%	58.4%	+5.1pp

Table 4.5. Perceptual quality overall win rates by method. Win rate represents percentage of pairwise comparisons won. Delta column shows the difference between close and distant viewing conditions.

Figure 4.8 breaks down win rates by viewing distance. The three higher-performing methods, Open3D, meshoptimizer, and CGAL, show minimal difference between distant and close conditions, with deltas ranging from -2.0 to $+5.1$ percentage points. Fast-simplification performs slightly better at distant viewing, reaching 34.7%, than at close inspection, where it drops to 30.0%, consistent with higher geometric error being less visible at simulated gameplay distances. Across all methods, participants selected the same method at both viewing distances in 66.6% of paired evaluations, covering 920 of 1,381 pairs where the same comparison appeared at both distances. The remaining 33.4% of cases produced different selections at distant versus close viewing, indicating that viewing distance influences approximately one-third of perceptual judgments.

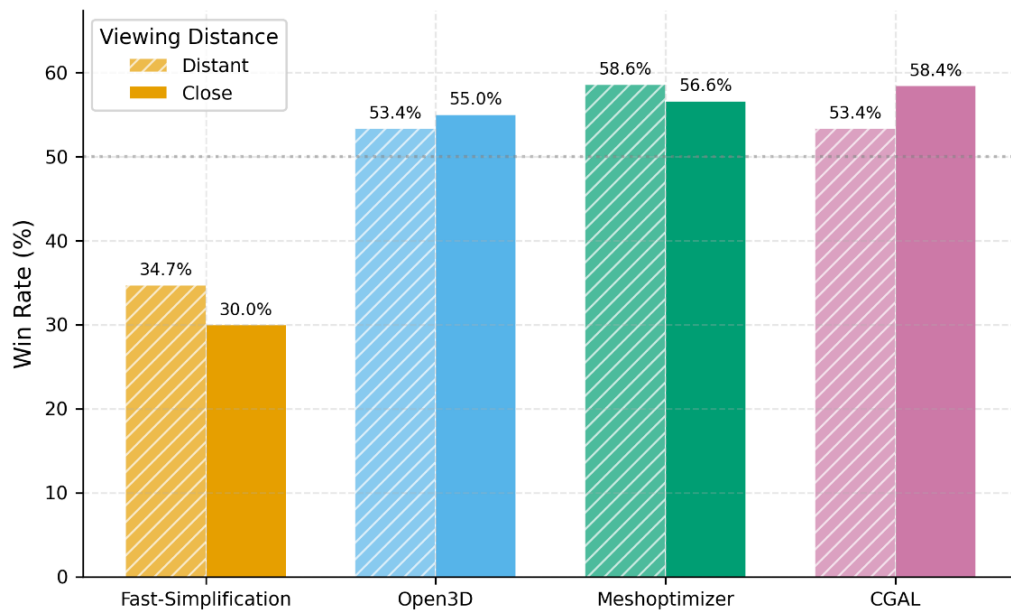


Figure 4.8. Perceptual win rates by method and viewing distance. Hatched bars indicate distant, LOD-simulated viewing. Solid bars indicate close inspection. Dashed line at 50% represents chance-level performance in pairwise comparison.

4.5.2 Perceptual Rankings by Reduction Level

Method rankings shift across reduction levels. At 50% reduction, CGAL achieves the highest win rate of 63.5%, followed by meshoptimizer at 55.4%, Open3D at 50.1%, and fast-simplification at 30.6%. At 80% reduction, the top three methods converge: CGAL reaches 56.5%, meshoptimizer 56.0%, and Open3D 55.3%, with fast-simplification at 32.6%. At 90% reduction, meshoptimizer takes the lead with 61.5%, followed by Open3D at 57.1%, while CGAL drops below 50% to 47.9%. Fast-simplification remains

below 34% across all reduction levels. Per-reduction bar charts and numerical tables are provided in Appendix B.

4.5.3 Asset-Specific Perceptual Patterns

No single method achieves the highest perceptual win rate on all assets. CGAL leads on three assets: AK-74 at 64.9%, M9 Pistol at 73.6%, and Jeep at 69.7%. Open3D leads on two: Bunker at 71.3% and Mi-8 at 72.7%. Meshoptimizer leads on two: Church at 70.8% and Watermill at 79.9%. Fast-simplification leads on one asset only, the LAV at 55.8%, where it is the sole asset on which fast-simplification outperforms all other methods. Win rates range from 8.7% for fast-simplification on the Watermill to 79.9% for meshoptimizer on the Watermill, demonstrating that asset geometry substantially influences method-specific perceptual quality. The complete heatmap of per-asset win rates is provided in Appendix B.

4.6 Geometric-Perceptual Correlation

Spearman rank correlation tested whether geometric accuracy metrics predicted perceptual preference. Table 4.6 reports correlations between method win rates and both geometric error metrics across viewing conditions. All correlations are statistically significant at $p < 0.01$ and negative, confirming the expected direction: lower geometric error is associated with higher perceptual win rates.

View	n	Hausdorff ρ	Hausdorff p	RMSE ρ	RMSE p
Combined	96	-0.309	0.002	-0.389	<0.001
Distant	96	-0.311	0.002	-0.369	<0.001
Close	96	-0.308	0.002	-0.397	<0.001

Table 4.6. Spearman rank correlation between perceptual win rate and geometric error metrics. All correlations are statistically significant. $n = 96$ data points spanning 4 methods, 8 assets, and 3 reduction levels.

However, correlation strength is weak across all conditions. RMSE shows the strongest association at $\rho = -0.389$ with $p < 0.001$ combined, while Hausdorff distance correlates slightly lower at $\rho = -0.309$ with $p = 0.002$. Neither metric accounts for more than 15% of variance in perceptual preference with $R^2 \approx 0.10$ – 0.15 . Correlations remain consistent between distant and close viewing conditions, with differences of only 0.003–0.028 in magnitude between conditions.

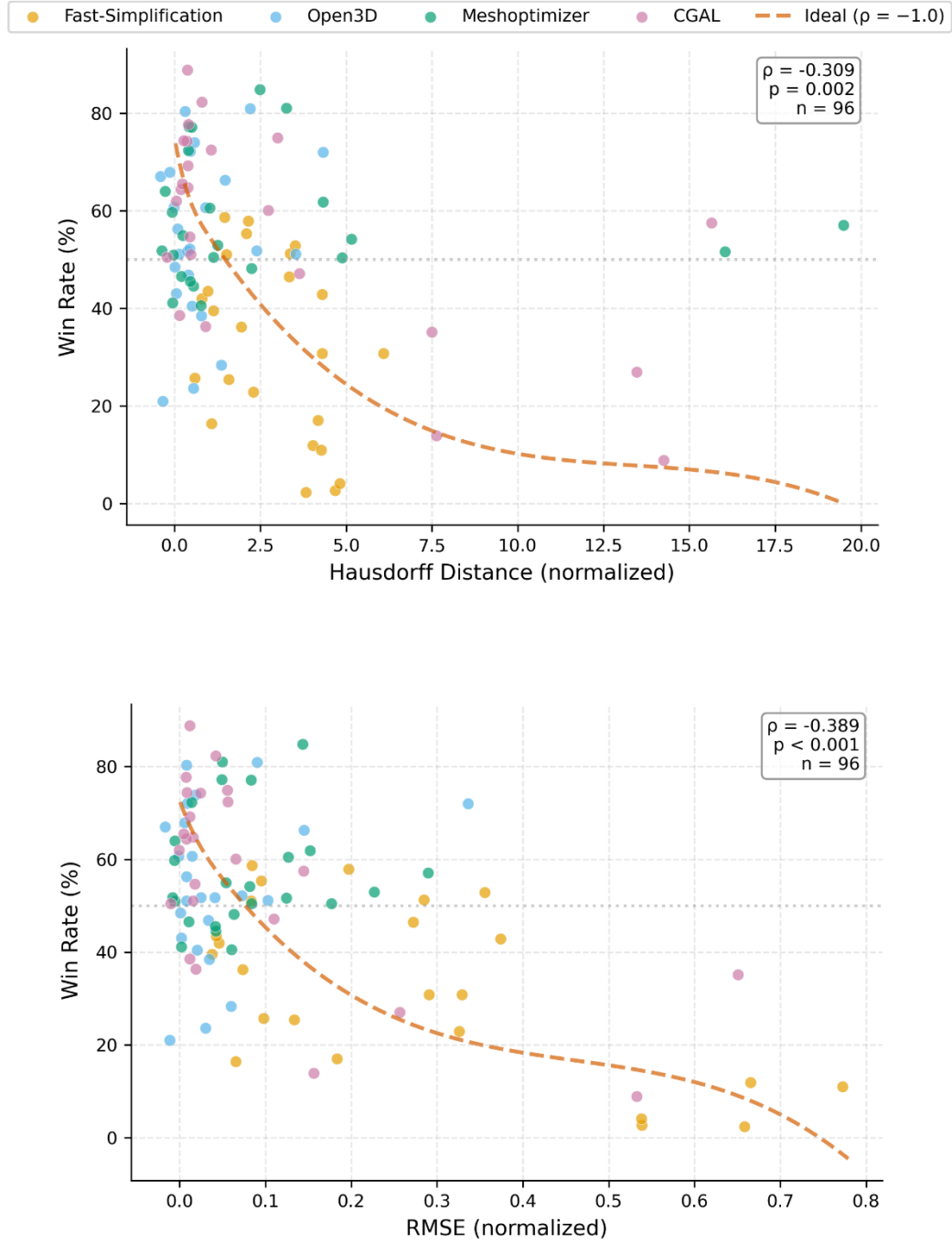


Figure 4.9. Perceptual win rate versus normalized geometric error for all method-asset-reduction combinations at $n = 96$. Dashed curve: ideal trend representing perfect negative rank correlation at $\rho = -1.0$ derived from the observed data distribution. Top: Hausdorff distance. Bottom: RMSE. Each point represents one method applied to one asset at one reduction level. Color indicates method. Dashed horizontal line at 50% marks chance-level preference.

Figure 4.9 visualizes the relationship between perceptual preference and geometric error. The dashed curve represents perfect negative rank correlation at $\rho = -1.0$ derived from the observed data distribution. The dispersion of data points around this reference reflects the weak observed correlation. Fast-simplification forms a distinct cluster at higher error and lower win rates. At low geometric error values below approximately 1% Hausdorff or 0.05% RMSE, win rates span the full range from approximately 20% to 85%, indicating that geometric metrics poorly discriminate perceptual quality among geometrically accurate results. The three remaining methods Open3D, meshoptimizer, and CGAL overlap substantially in both geometric error and perceptual win rate, with method identity providing limited predictive power within this cluster.

4.7 Summary of Findings

The data presented in this chapter provided a factual basis for answering the research question: *How do accessible mesh simplification methods compare in geometric accuracy, processing performance, visual perceptual quality, and practical usability when evaluated within Blender-based game asset workflows targeting resource-constrained developers?*

The answer is mixed. No single method dominates across all evaluation dimensions. Processing performance spans two orders of magnitude, with meshoptimizer completing in 23.82 ms mean compared to CGAL at 1,479.60 ms, while all four methods operated within consumer hardware memory limits for production-scale assets. Geometric accuracy favors Open3D, which achieved the lowest error on both Hausdorff distance and RMSE, while fast-simplification consistently produced the highest geometric error. Perceptual evaluation tells a partially different story: meshoptimizer achieved the highest win rate at 57.6%, with CGAL at 55.9% and Open3D at 54.2% close behind, while fast-simplification was consistently least preferred at 32.3%. The top three methods are separated by only 3.4 percentage points in perceptual preference, despite larger differences in geometric accuracy and substantially larger differences in processing time.

The relationship between geometric metrics and perceptual quality is weak with Spearman ρ of -0.31 to -0.39 and $R^2 \approx 0.10$ -0.15. Both metrics correlate in the expected direction, confirming that lower geometric error is associated with higher perceptual preference, but neither metric accounts for more than 15% of variance in participant choices. Asset geometry influences method-specific outcomes: no method achieves the highest perceptual win rate across all eight assets.

All four methods achieved 100% success rate across 288 production-scale operations and 48 scalability operations, confirming robustness on consumer hardware. Scalability testing demonstrated that all methods process meshes up to 28.1 million faces without failure, though processing times at this scale range from 21 seconds for meshoptimizer to 13.4 minutes for CGAL.

These findings address the three expectations stated at the close of the literature review in Section 2.5. The hypothesis that geometric metrics would fail to predict perceptual quality was confirmed: Spearman correlations of $\rho \approx -0.31$ to -0.39 account for only 10-15% of perceptual variance. The expectation that the four QEM implementations would exhibit different quality-performance characteristics was also confirmed, though the nature of those differences, specifically asset-class-dependent rankings rather than a single quality hierarchy, was not anticipated. The pilot hypothesis regarding neural method accessibility barriers was confirmed during the investigation documented in Appendix A, leading to their exclusion before quantitative comparison. Chapter 5 discusses the implications of these findings for method selection, workflow integration, and the role of geometric metrics in production quality assessment.

Chapter 5: Discussion of Data and Results

Throughout Chapter 4, it was established that no single method dominates across all evaluation dimensions. Processing performance spans two orders of magnitude, geometric accuracy favours Open3D, perceptual preference favours meshoptimizer, and the correlation between geometric metrics and perceptual judgments is weak at $\rho \approx -0.31$ to -0.39 . These results answer the research question's "how do they compare" component, but they do not explain why the rankings disagree across dimensions, or what the disagreements mean for practitioners selecting a method.

This chapter interprets the Chapter 4 data through three lenses. It connects each method's observable behavior to specific design decisions in the twenty-eight-year development history of quadric error metrics, places the geometric-perceptual disconnect in the context of prior perceptual quality research, and grounds the analysis in visual inspection of simplified outputs rather than aggregate statistics alone.

5.1 Method Behavior: Visual Artifacts and Algorithmic Lineage

The per-method performance patterns reported in Chapter 4 do not describe four interchangeable algorithms competing on a single quality axis. They describe four different implementations of the quadric error metric family, each reflecting specific design decisions that produce distinct artifact patterns visible in the simplified outputs. These patterns trace directly to algorithmic design choices and explain why geometric accuracy rankings and perceptual quality rankings do not agree.

5.1.1 Fast-simplification: threshold escalation and catastrophic failure

Fast-simplification produces the highest mean geometric error of the four methods, with Hausdorff distance of 2.88% and RMSE of 0.27%, and the lowest perceptual win rate at 32.3%. On most assets this manifests as consistently elevated error that still preserves the overall shape and silhouette. On the Church asset at 80% and 90% reduction, however, the behavior changes qualitatively: the tower spire deforms into ragged protrusions, the roof fragments into irregular polygons, and the actual reduction stalls at 79.5% rather than reaching the requested target. Figure 5.1 shows the result alongside the three other methods at identical reduction.

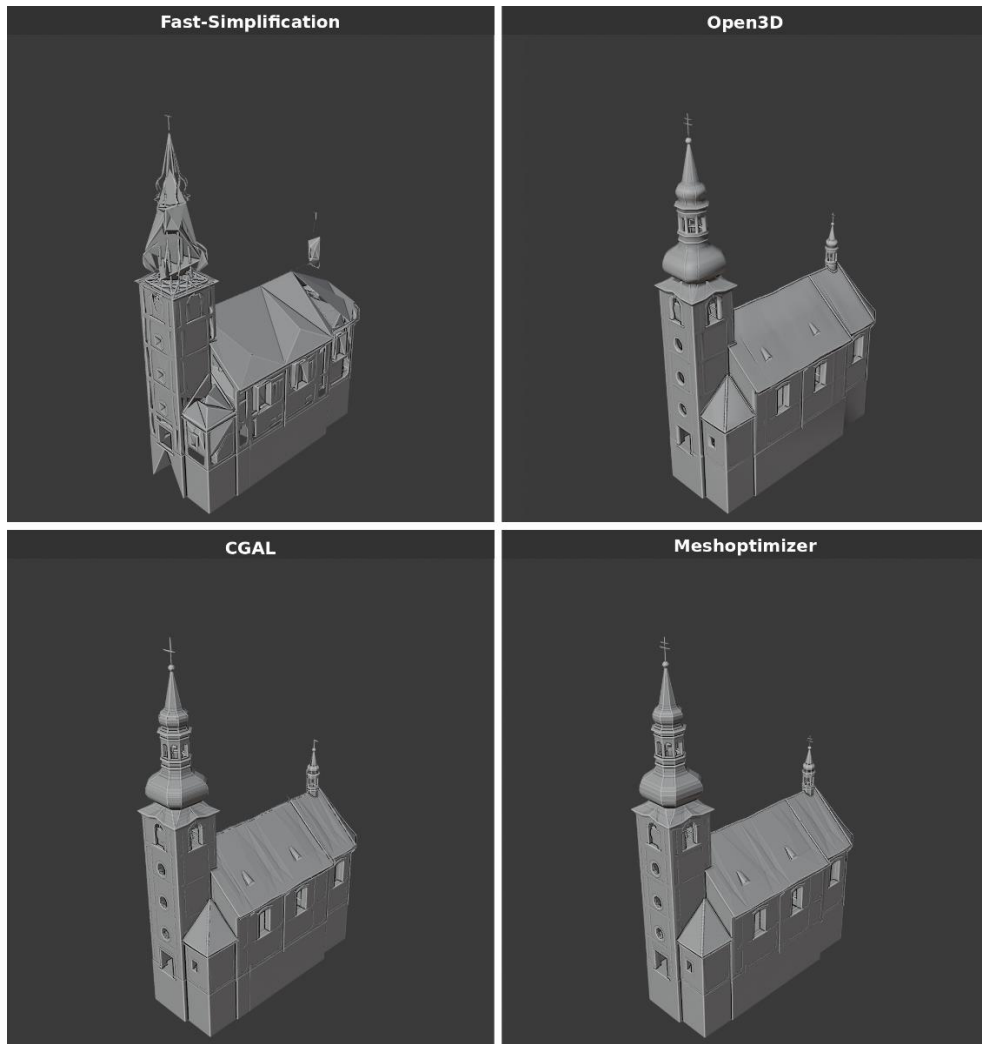


Figure 5.1. Church at 80% reduction.

The fast-simplification implementation (Forstmann, 2014) uses an iterative threshold scheme where the error tolerance grows aggressively with each iteration, controlled by an exponent parameter defaulting to 7. The algorithm avoids maintaining a sorted priority queue of candidate collapses, trading quality for speed. Early iterations remove low-cost collapses. When the target face count cannot be reached through low-cost operations, the threshold escalates aggressively, and subsequent iterations accept collapses that would have been rejected as destructive earlier in the process. The algorithm's own documentation states directly that it "works well for watertight volumes without thin sections" (Forstmann, 2014). The Church asset, with its tower spires, window mullions, and dense architectural detailing, is precisely the class of geometry the author identifies as outside the algorithm's intended domain.

The simplification ceiling observed on the Church, reaching 79.5% actual against a 90% target, is a well-documented failure mode of unconstrained QEM. Hoppe (1999) identifies the mechanism: repeated edge collapses on fine features create error conditions that lock vertices in place and prevent further simplification. When the algorithm encounters this locked state, fast-simplification escalates the threshold until a collapse becomes possible, producing the destructive output visible in Figure 5.1. The approximately 1.25-second processing-time spike reflects the additional iterations required. (Bahirat et al., 2018) report the same pattern in their own evaluation: QEM error curves terminate partway through the reduction process on meshes with similar geometry.

The Church represents the most extreme instance, but it is not the only asset where fast-simplification produces structurally damaging simplification. On the Watermill, the method removes paddle fins that other methods preserve. On the Jeep, it fills the windshield opening, collapsing a void that meshoptimizer, Open3D, and CGAL retain. These are less catastrophic than the Church's structural collapse but follow the same pattern: the threshold-escalation scheme accepts damaging collapses that the other methods avoid. The single asset where fast-simplification achieves the highest perceptual win rate, the LAV at 55.8%, is a compact armoured volume without thin sections or voids, precisely the geometry class the developer identifies as the method's intended domain.

One could argue that the failure mode is disclosed rather than hidden, and that developers working exclusively with watertight, compact geometry can exploit the method's mean 159 ms speed advantage, the second fastest, without encountering destructive behavior. However, production pipelines process heterogeneous asset libraries by definition. A workflow that must pre-screen each asset's topology before selecting a simplification method reintroduces the manual intervention the automation was designed to eliminate. The Church is not an exotic edge case. Dense architectural detailing is a routine asset class in the military simulation context from which the test suite originates.

Fast-simplification's mean performance statistics therefore understate its risk in production. A pipeline that processes heterogeneous assets will likely encounter an asset whose topology triggers this structural collapse. For comparison, meshoptimizer's single largest Hausdorff outlier in the entire dataset, 19.48% on the LAV at 90% reduction, reflects distant thin features being preserved rather than central geometry being destroyed, which is less severe than the structural collapse observable on the Church.

5.1.2 Open3D: geometric fidelity and shading degradation

Open3D achieves the lowest mean geometric error on both metrics at Hausdorff 0.88% and RMSE 0.045% yet ranks third in perceptual preference at 54.2%. The mechanism becomes apparent on inspection of the Watermill asset, shown in Figure 5.2. Meshoptimizer and CGAL both produce a Watermill whose paddles read as thin flat plates attached to a faceted wheel rim, and where the stone blocks at the base retain their cubic edges. Fast-simplification removes several paddle fins entirely, as discussed in Section 5.1.1. The Open3D output preserves the same structural elements as meshoptimizer and CGAL, but the smooth shading makes the flat paddles appear rounded, the faceted wheel rim appear curved, and the cubic stone blocks appear pillowed. The vertex positions are in fact closer to the original than the other methods achieve. What has changed is not the geometry but how it is shaded.

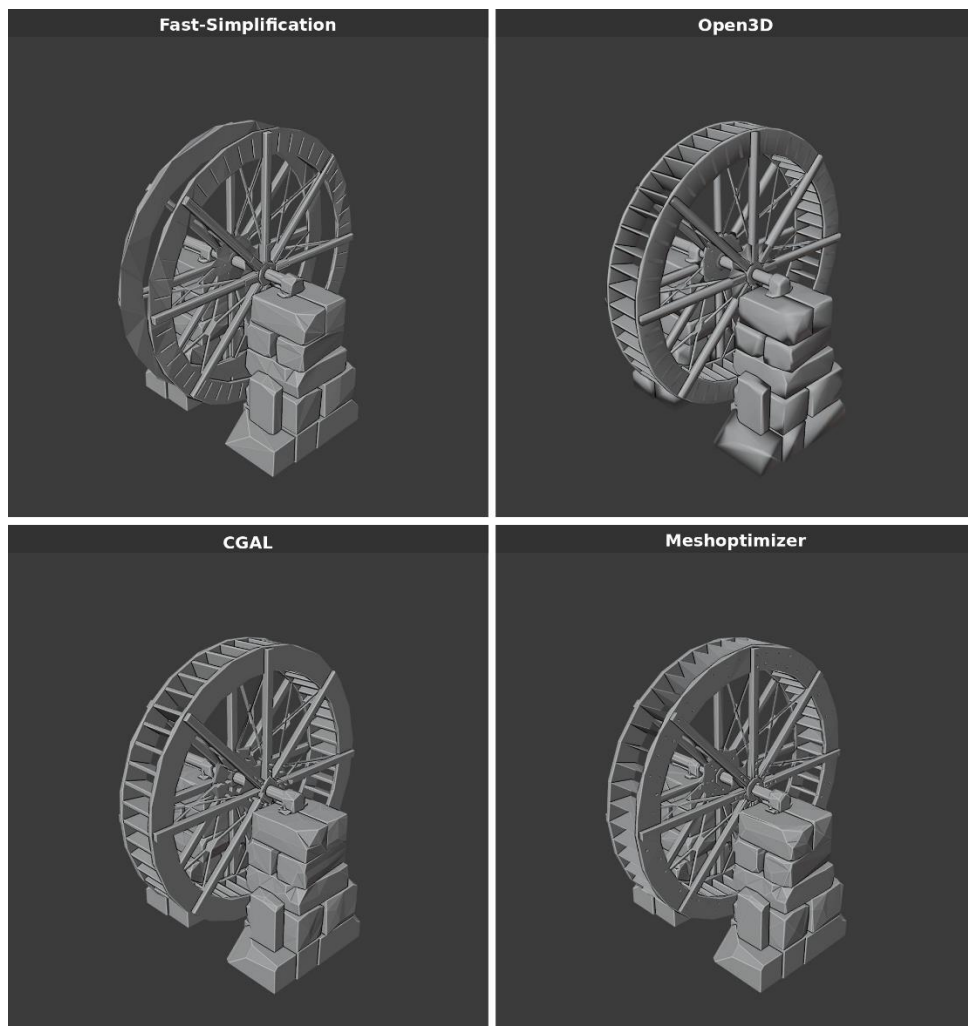


Figure 5.2. Watermill at 80% reduction.

Open3D implements the 1997 geometric form of QEM (Garland & Heckbert, 1997) without subsequent extensions for appearance preservation. Neither Hoppe's (1999) extension for preserving attribute boundaries across sharp edges nor the 1998 attribute-extended quadric (Garland & Heckbert, 1998) is present. (Riegler, 2019) states directly in the pull request that introduced the function that "the color and normal information is averaged in all implementations and not considered in the error minimization." When two vertices are collapsed, their normals are averaged together, losing the sharp transitions between faces, and the error minimization optimizes vertex position alone.

On meshes where artist-authored creases produce intentional hard edges, such as the Watermill and Church, this averaging fuses sharp transitions into smooth gradients. The asset geometry survives, while the asset identity does not. On meshes with smoother, more curved surfaces, such as the Bunker sandbag stack or the Mi-8 fuselage, the same averaging behaves as a useful smoothing effect. This predicts exactly the asset-class dependence observed in Chapter 4: Open3D leads perceptually on the Bunker at 71.3% and Mi-8 at 72.7%, but places below meshoptimizer on the Watermill where meshoptimizer reaches 79.9% compared to Open3D's 51.1%, and on the Church at 70.8% compared to 64.8%.

The shading loss is not necessarily permanent. Production pipelines routinely recompute vertex normals after mesh operations. Blender's Auto Smooth is a common step in even simple workflows, and if normals are regenerated downstream, Open3D's geometric accuracy advantage survives while the shading degradation is corrected. The data also shows that the shading behavior is not universally detrimental. On assets with smoother, more curved surfaces such as the Bunker and Mi-8, normal averaging appears less disruptive, suggesting asset-class dependence rather than a uniform defect, though the test suite's hard-surface focus limits generalization to truly organic geometry. However, this argument assumes the artist recognizes that normals need repair in the first place. The thesis evaluates methods as end-to-end tools for resource-constrained developers, and a tool that degrades shading identity without warning while reporting the lowest geometric error creates a mismatch between what the metrics promise and what the output delivers. The perceptual data confirms this: participants evaluated Open3D output as delivered, without post-processing, and ranked it third overall despite its geometric accuracy lead. (Cohen et al., 1998) argued in the original appearance-preserving simplification paper that vertex positions alone do not determine how the final pixels look on screen. The Open3D data supports this argument empirically: a method whose output requires the artist to know that correction is

needed is, for the target audience of this research, functionally equivalent to a method that does not have that advantage.

This is not a criticism of the Open3D implementation on its own terms. The library documents the method it implements, and the implementation is faithful to its cited source. It is a warning against the inference "low Hausdorff distance means good simplification" when that inference is applied uncritically to game production contexts where assets carry meaningful shading information.

5.1.3 CGAL: memoryless drift and local spikes

CGAL produces strong mean perceptual preference at 55.9% overall, leading at 50% reduction with 63.5% as the highest single-reduction win rate of any method, but exhibits two systematic deviations. First, it consistently over-reduces: every CGAL configuration in the test matrix exceeds its requested reduction target, with the Mi-8 helicopter exhibiting the largest overshoot at +3.5 percentage points. Second, at 90% reduction it produces extreme Hausdorff outliers of 15.98% on the LAV and 13.24% on the Bunker, an order of magnitude above its own median.

Both behaviors reflect CGAL's use of the Lindstrom-Turk strategy (Lindstrom & Turk, 1998, 1999) rather than standard Garland-Heckbert QEM. The method is "memoryless" by design: each edge collapse is evaluated against the current state of the simplified mesh, with no reference to the original geometry. This makes the algorithm memory-efficient but allows small placement errors to compound across collapses. Lindstrom and Turk's (1999) own evaluation reports strong mean geometric error but weaker worst-case bounds than methods that track the original surface, the exact pattern observed in this evaluation. The visible spike artifacts on the Bunker shown in Figure 5.3 are corroborated externally: a maintainer-acknowledged issue in the CGAL repository under Issue #8213, opened May 2024, which reports identical behavior, with the reporter confirming that Garland-Heckbert strategies on the same inputs do not produce spikes. Pull request #8237 addressing the issue is linked but unmerged at the time of writing. The CGAL documentation itself notes that Lindstrom-Turk "does not guarantee that the resulting surface has no self-intersections" (CGAL Project, 2025).

The over-reduction reflects a different Lindstrom-Turk property: the algorithm is driven by edge-collapse cost, and the stopping criterion is evaluated after each priority-queue pass. When a target is reached mid-pass, residual low-cost collapses already queued continue to execute, producing the observed 0.2 to 3.5 percentage-point overshoots. The overshoot is consistent and predictable, meaning developers can compensate by requesting a slightly conservative target. CGAL also leads perceptually

on three assets, namely AK-74 at 64.9%, M9 Pistol at 73.6%, and Jeep at 69.7%, all compact hard-surface objects where the strategy's mean accuracy translates into visible quality, and the spike issue has a pending maintainer fix under PR #8237, suggesting the problem might be transient.

However, production LOD chains require aggressive reduction levels precisely where CGAL's reliability degrades. Distant LODs at 80% and 90% reduction are not optional tiers. They determine rendering performance on lower-end hardware, the core concern for resource-constrained developers. CGAL's drop from 63.5% preference at 50% to 47.9% at 90%, falling below chance level, means the method loses its perceptual advantage at the reduction levels where LOD systems spend most of their screen time. The predictability of the over-reduction helps, but it does not address the spike artifacts: a developer can request 87% to land at 90% actual, but cannot predict or prevent a 15.98% Hausdorff outlier. A method that excels at moderate reduction but becomes unreliable at aggressive reduction serves only the upper tiers of a LOD chain.

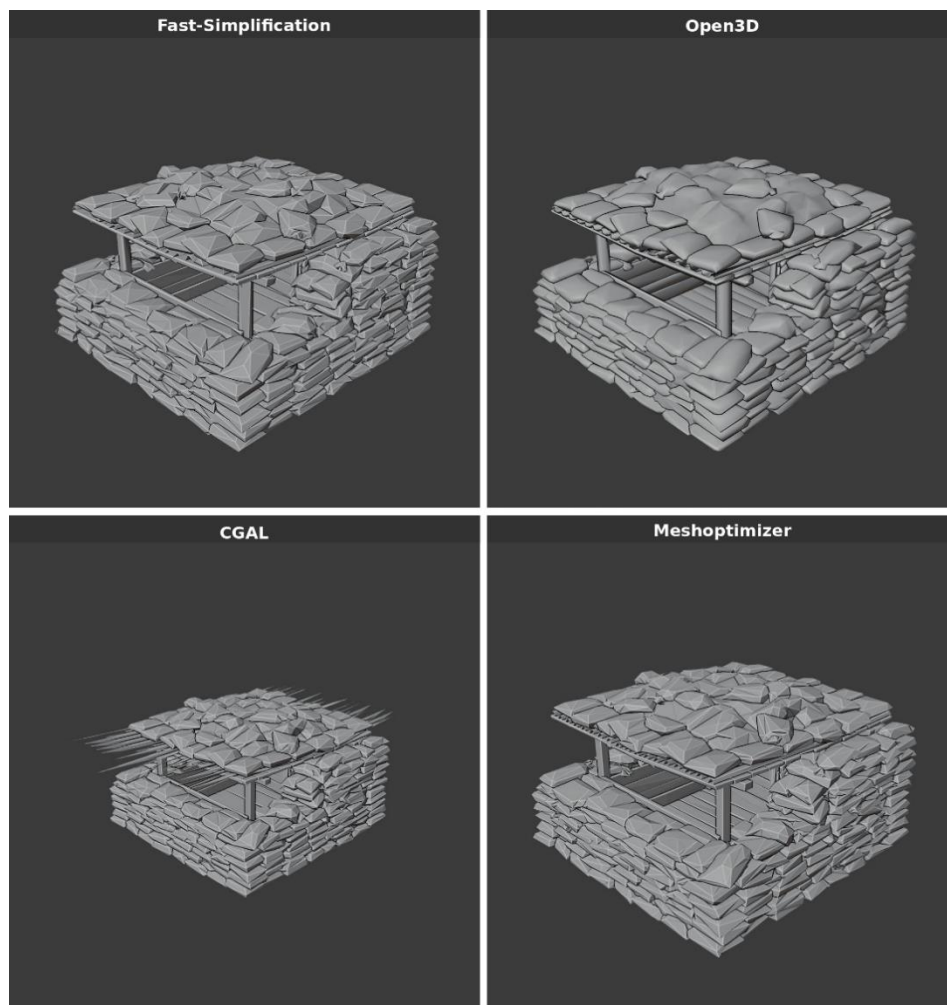


Figure 5.3. Bunker at 80% reduction.

5.1.4 Meshoptimizer: consistency over specialization

Meshoptimizer ranks first in perceptual preference at 57.6% without leading on any single geometric metric. The finding is not that meshoptimizer produces exceptional output on any individual asset, but that it produces consistently reliable output across all of them. It does not catastrophically fail on dense architectural topology, systematically alter shading, or produce spike outliers at aggressive reductions.

The design choices that produce this behavior are deliberate and documented. The meshoptimizer simplifier, under continuous development since 2016 (Kapoulkine, 2025), classifies vertices by kind, specifically Manifold, Border, Seam, Complex, and Locked, before the collapse loop begins, determining which collapses are legal. Its attribute handling infers discontinuities from the supplied index buffer automatically. The documentation states that the simplifier "is aware of attribute discontinuities by default, inferring them through the supplied index buffer, and tries to preserve them" (Kapoulkine, 2025). Borders and seams receive dedicated error weights rather than being discarded. A small regularizing term prevents the numerical instabilities in the error calculation that (Trettner & Kobbelt, 2020) identify as the theoretical source of QEM fragility.

The case for meshoptimizer as the strongest overall method is not unchallenged. It produces the single largest Hausdorff outlier in the entire dataset, 19.48% on the LAV at 90% reduction, larger than any individual value from Open3D or fast-simplification. Its perceptual lead over CGAL is 1.7 percentage points at 57.6% versus 55.9%, a margin that is directional but narrow enough that it could shift with a different asset selection or participant pool. Meshoptimizer does not lead on geometric accuracy by either metric. On three assets, the AK-74, M9 Pistol, and Jeep, CGAL outperforms it perceptually, and on two the Bunker and Mi-8, Open3D leads. Meshoptimizer's perceptual advantage is concentrated on the Church and Watermill, meaning its overall ranking depends on two assets.

What distinguishes meshoptimizer is the absence of a failure mode that disqualifies it on any asset. Open3D's geometric lead does not transfer to perceptual preference on hard-surface assets. CGAL's strong moderate-reduction performance collapses at aggressive levels. Fast-simplification's speed advantage is negated by catastrophic failure on dense architecture. Meshoptimizer produces no comparable disqualifying pattern. Its 19.48% LAV outlier reflects conservative preservation of a distant thin feature such as an antenna or gun barrel rather than structural destruction, occurring on the single asset class where retaining distant geometry arguably preserves silhouette identity. The LAV is accordingly the only asset where fast-simplification wins

the perceptual comparison at 55.8%. The narrow perceptual margin over CGAL is the point: meshoptimizer wins the aggregate by never losing badly, not by excelling on any single asset.

These twenty-eight years of QEM refinement since Garland and Heckbert's original paper have produced concrete, implementable improvements: topology preservation, attribute-aware cost functions, seam inference, numerical regularization. The question for practitioners is not whether these refinements matter, the perceptual data answers that, but which of the accessible methods have absorbed them. Meshoptimizer has, while Open3D and fast-simplification have not.

5.1.5 Summary

The four methods exhibit characteristic artifact signatures that reflect specific algorithmic design choices, explaining why geometric accuracy and perceptual preference produce different rankings. Fast-simplification's threshold-escalation design produces structurally damaging simplification on assets with thin features, voids, or dense architectural topology. Open3D's omission of attribute-preserving extensions produces asset-class-dependent shading loss that appears to benefit smoother geometry but degrades hard-surface identity. CGAL's memoryless Lindstrom-Turk strategy excels at moderate reduction but loses reliability at aggressive levels where production LOD chains need it most. Meshoptimizer's integration of border, seam, and vertex-kind classification produces balanced output without method-specific failure modes, winning the perceptual aggregate through consistency rather than peak performance. None of these behaviors is a defect in the sense of a bug. Each is the predictable consequence of the algorithmic lineage the implementation draws from.

5.2 Geometric-Perceptual Disconnect

The Spearman correlation between geometric error metrics and perceptual win rates is weak across all conditions: $\rho = -0.31$ for Hausdorff distance and $\rho = -0.39$ for RMSE, both at $p < 0.01$ with $n = 96$. The negative direction confirms the expected relationship, where lower geometric error is associated with higher perceptual preference, but the magnitude means these metrics account for roughly 10-15% of the variance in participant choices. The correlation remains consistent between viewing conditions despite 33.4% of paired judgments producing different selections at distant versus close viewing, suggesting that whatever factors beyond geometric error drive preference, they operate similarly regardless of viewing distance.

This finding extends a body of literature documenting the limitations of geometric metrics as perceptual predictors. Lavoué et al. (2006) reported Spearman correlations between Hausdorff distance and subjective quality scores that varied dramatically by object: from 0.892 on the Armadillo to 0.006 on Venus for Hausdorff maximum. This instability motivated their curvature-based Mesh Structural Distortion Measure (MSDM) metric, which achieved more stable correlations ranging from 0.735 to 0.867. Corsini et al. (2013) surveyed the field and confirmed that the gap between geometric and perceptual metrics remained an open problem. Karis, Stubbe, and Wihlidal (2021) confirm this limitation from the industry side, noting that geometric error at simplification time cannot capture the perceptual impact of normal or UV error because the material that will be applied to the mesh is unknown. The present study's correlations at $|\rho| = 0.31\text{-}0.39$ fall at the lower end of Lavoué's observed range, though the comparison is not exact. Lavoué computed per-object correlations across synthetic distortions such as noise and smoothing on three organic meshes, while the present study pools 96 data points across four real simplification algorithms and eight production assets. The weaker correlations here likely reflect this difference: real algorithms produce qualitatively different artifact types as described in Section 5.1, and the factors that drive perceptual preference, such as attribute preservation and failure mode avoidance, are precisely what geometric metrics do not measure.

The Open3D result illustrates this directly. Open3D achieves the lowest geometric error by a wide margin, with Hausdorff distance of 0.88% versus the next best at 2.70% yet ranks third perceptually, likely because its normal averaging destroys shading identity on hard-surface assets while preserving vertex positions as discussed in Section 5.1.2. Hausdorff distance and RMSE register this as the most faithful simplification in the dataset, but participants disagree.

One could argue that the correlation, while weak, retains value as a rough filter. That argument is partially supported: Fast-simplification clusters in the high-error, low-win-rate region of Figure 4.9, and excluding methods above 3% Hausdorff error would successfully avoid the worst performer. However, within the cluster of geometrically accurate results below approximately 1% Hausdorff, win rates span from roughly 20% to 85%, meaning the metrics provide no useful discrimination among the three competitive methods. A developer choosing between Open3D, meshoptimizer, and CGAL on geometric accuracy alone would select Open3D, the method that ranks third perceptually and loses decisively to meshoptimizer on the assets where shading preservation matters most. Geometric metrics can detect catastrophic failure, but they

cannot rank competitive methods against each other. For production tool selection, perceptual evaluation provides information that geometric metrics structurally cannot.

5.3 Production Assets as Evaluation Ground

The decision to evaluate on production game assets rather than exclusively on standard academic models shaped the findings in ways that a conventional benchmark study would not have produced. The asset-class dependence documented in Section 5.1 only emerges because the test suite contains structurally diverse production geometry. Patterns such as Open3D appearing to benefit organic geometry while degrading hard-surface identity, meshoptimizer tending to lead on architectural and mechanical assets, and CGAL showing strength on compact weapons are based on one to three assets per geometry type and should be treated as indicative rather than definitive. A study conducted exclusively on Stanford 3D Scanning Repository models, all of which are watertight, manifold, single-material laser scans, would likely have ranked Open3D higher and missed the shading preservation problem entirely.

Greg Turk, who scanned the original Stanford Bunny in 1994, has stated directly that he considers the bunny "too good as a test model" because it is "so smooth that there is a lot of redundant information in the original collection of triangles, so simplification and compression algorithms give wonderful results" (Turk, 2000). This observation from the creator of the field's most widely used benchmark supports the methodological choice to test on geometry that is not structurally forgiving. (Liu et al., 2024) provide quantitative reinforcement: 98.9% of ShapeNet meshes contain non-manifold geometry, indicating that even large-scale academic datasets differ structurally from the clean topology that simplification algorithms are typically validated against. Production game assets, with their multi-material boundaries, thin architectural features, and mixed organic and mechanical components, occupy a complexity regime that standard benchmarks do not represent.

The findings support the conclusion that this complexity matters. Fast-simplification's clear failure on the Church described in Section 5.1.1 is triggered by dense architectural detailing that no Stanford model contains. CGAL's spike artifacts on the Bunker described in Section 5.1.3 appear on a sandbag-and-timber structure whose irregular topology differs substantially from a smooth laser scan. The 28.8 percentage-point perceptual gap between meshoptimizer and Open3D on the Watermill as discussed in Section 5.1.2 depends on the asset having intentional hard edges that the normal-averaging mechanism can destroy. None of these findings would surface on a corpus of smooth organic scans.

The cost of this choice is reproducibility. The production assets are provided under non-disclosure agreement, preventing external researchers from replicating the geometric accuracy and perceptual findings on identical geometry. The Stanford models included for scalability testing as described in Section 3.4 provide a partial reproducibility bridge, but they cover only processing time and success rate, not the core perceptual and artifact analyses. This is a genuine limitation. However, the alternative, restricting evaluation to publicly available models to maximise reproducibility, would have produced a study that validates algorithms in precisely the structurally forgiving regime that Turk identifies as unrepresentative. The test suite prioritizes hard-surface game assets, which means the findings may not generalize to organic character models or foliage. Section 5.4 addresses this scope constraint directly.

5.4 Methodological Reflection

The findings reported in this chapter are shaped by several methodological choices and constraints that affect their generalizability, reliability, and replicability. This section identifies the most consequential limitations and, where appropriate, suggests how a future study could address them.

The most significant constraint is the hard-surface focus of the production test suite. All eight Bohemia Interactive assets are mechanical, architectural, or industrial objects with well-defined edges and flat surfaces. The asset-class dependence documented in Section 5.1, particularly Open3D's advantage on organic geometry, was visible only because the Bunker and Mi-8 contain organic-like elements such as sandbags and curved fuselage panels within an otherwise hard-surface corpus. A test suite that included dedicated organic assets, such as character models, vegetation, or terrain, could shift the rankings. Open3D's normal averaging, which degrades hard-surface identity, may prove consistently beneficial on organic geometry, potentially elevating it above meshoptimizer in that domain. The findings should therefore be read as specific to hard-surface game asset workflows, with organic asset performance remaining an open question.

All perceptual stimuli were rendered without textures, using only flat shading on neutral geometry. In production, textures and material properties can both mask and amplify simplification artifacts. A normal-map-heavy asset might hide the shading loss that Open3D produces, while a clean metal surface with sharp reflections might make it more visible. The untextured evaluation isolates geometric and shading quality from material interaction, which is methodologically clean but does not fully represent in-

game visual conditions. A future study incorporating textured rendering would test whether the method rankings hold under more realistic visual conditions.

The controlled rendering conditions represent a deliberate methodological trade-off. Untextured, uniformly lit presentation isolates simplification quality from confounding variables, meaning the measured perceptual differences represent a conservative upper bound on what players would notice in-game, where textures, normal maps, and dynamic lighting further mask artifacts. However, this framing also means the study cannot determine which differences remain visible under production rendering and which disappear entirely.

A separate limitation is that the evaluation assessed each reduction level independently rather than as part of an integrated LOD chain. In production, a geometrically imperfect final LOD may be acceptable if the player arrives at it through gradual, imperceivable steps, while visible jumps in silhouette or shading between levels can be more disruptive than the absolute quality of any single level. This distinction is relevant to the per-method findings: fast-simplification's threshold escalation on the Church would produce exactly the kind of abrupt LOD transition that players notice, while meshoptimizer's consistent degradation may translate into smoother LOD chains, though this remains untested.

Memory profiling produced measurement artifacts on fast-executing methods. The RSS-based approach recorded negative deltas for Open3D on several configurations and low measurement stability for methods completing in under 100 ms. These artifacts are documented in Section 4.1.2 and reflect polling granularity rather than genuine method behavior. A more robust approach would wrap each method invocation in a dedicated subprocess and measure peak RSS of the entire process, eliminating interference from background memory management. The memory findings should be interpreted with this limitation in mind, though the relative ordering from meshoptimizer lowest to CGAL highest is consistent with algorithmic expectations.

Several additional constraints need acknowledgment. All benchmarks were conducted on a single hardware configuration of an RTX 3060, Ryzen 7 5800H, and 32 GB RAM. Relative method rankings are likely stable across hardware, but absolute processing times and memory values are specific to this system.

The Spearman p-values reported in Chapter 4 treat the 150 participants as a sample from a larger population of possible evaluators. This framing is standard in perceptual research (Kim et al., 2010; Liu et al., 2024), but should be noted, and the correlation magnitudes rather than the significance values carry the practical meaning.

The distant viewing condition used CSS scaling to simulate LOD distances rather than calibrated physical distance on a fixed display. The 33.4% judgment flip rate between conditions suggests the simulation was sufficient to produce measurable viewing-distance effects, though a controlled laboratory setup would provide more precise distance calibration.

5.5 Ethical Considerations

The methodology chapter in Section 3.8 addressed ethical considerations anticipated before data collection: asset licensing, participant privacy, environmental impact, and LLM usage. The findings introduce three additional considerations that warrant acknowledgment.

First, the strongest findings in this thesis depend on production assets that cannot be shared due to non-disclosure agreement. Publishing conclusions that readers cannot independently verify on the same geometry creates a transparency tension. To mitigate this, the Stanford 3D Scanning Repository models included for scalability testing described in Section 3.4 are publicly available and cover processing time, memory usage, and success rate, providing a partial replication path. The complete scripting repository, benchmark framework, and perceptual study platform will be published on GitHub following thesis defense, enabling replication on any geometry a future researcher supplies.

Second, the finding that fast-simplification and CGAL exhibit specific failure modes carries a reputational dimension for open-source maintainers who provide these tools freely to the community. The thesis is careful to frame these as algorithmic design trade-offs rather than implementation defects. Both limitations are documented by their own developers: fast-simplification's README explicitly warns that the method works best on "watertight volumes without thin sections" (Forstmann, 2014), and CGAL's spike behavior is the subject of a maintainer-acknowledged Issue #8213 with an active pull request. Identifying where a tool's documented limitations manifest on production geometry is a contribution to the community that maintains it, not a criticism of the developers who built it.

Third, the thesis provides production scenario recommendations in Chapter 6 that could influence which open-source libraries receive adoption and community investment. This responsibility is mitigated by framing recommendations as context-dependent rather than as a single endorsement: different methods suit different asset types, reduction levels, and workflow constraints. No method is declared universally superior, and the recommendation table presents trade-offs rather than rankings.

Chapter 6: Conclusion and Future Directions

This chapter draws conclusions from the interpretations developed in Chapter 5 to answer the research question: How do accessible mesh simplification methods compare in geometric accuracy, processing performance, visual perceptual quality, and practical usability when evaluated within Blender-based game asset workflows targeting resource-constrained developers? The answer is explicitly inconclusive: no single method proved superior across all four evaluation dimensions, and the asset-class dependence documented in Chapter 5 means that the best method for a given workflow depends on the geometry being processed, the reduction levels required, and the trade-offs a developer is willing to accept.

6.1 Research Question Answer

The research question asks how four accessible mesh simplification methods compare across geometric accuracy, processing performance, visual perceptual quality, and practical usability. The evidence collected across 288 benchmark operations, 48 scalability tests, and 2,769 perceptual judgments produces a clear answer on each dimension individually, but no coherent answer when the dimensions are combined. Each method occupies a different position in the trade-off space, and the relative importance of each dimension depends on the production context.

On geometric accuracy, Open3D achieved the lowest error by a substantial margin at Hausdorff 0.88% and RMSE 0.045%, outperforming the next closest method by a factor of three on Hausdorff distance. However, as Chapter 5 established, this geometric advantage does not transfer to perceptual preference on hard-surface assets because Open3D's implementation averages vertex normals during edge collapse, degrading shading identity while preserving vertex positions. Geometric accuracy measured by Hausdorff distance and RMSE is therefore a necessary but insufficient indicator of simplification quality: it can detect catastrophic failure but cannot rank competitive methods against each other.

On processing performance, meshoptimizer completed simplification in 23.82 ms mean, roughly 7 times faster than fast-simplification, 20 times faster than Open3D, and 62 times faster than CGAL. All four methods operated within consumer hardware memory constraints for production-scale assets and all four scaled successfully to 28.1 million faces in scalability testing. Processing time is therefore not a barrier to adoption for any of the four tested methods at production-relevant complexity, though the two-

order-of-magnitude spread becomes consequential for batch processing of large asset libraries where CGAL's per-asset cost accumulates.

On visual perceptual quality, meshoptimizer achieved the highest overall win rate at 57.6%, followed by CGAL at 55.9% and Open3D at 54.2%, with fast-simplification substantially behind at 32.3%. The top three methods are separated by only 3.4 percentage points, a margin that is directional but narrow. The more consequential finding is asset-class dependence: no method achieved the highest win rate across all eight assets. CGAL led on compact hard-surface objects such as the AK-74, M9, and Jeep. Open3D led on assets with organic-like geometry such as the Bunker and Mi-8, and meshoptimizer led on architecturally complex assets such as the Church and Watermill. Meshoptimizer's aggregate advantage comes from not losing badly on any tested asset rather than from excelling on any single one.

On practical usability, three methods, fast-simplification, Open3D, and meshoptimizer, install via `pip` in under three minutes with minimal dependencies. CGAL requires C++ compilation through `vcpkg` with 109 dependencies and 62 minutes of build time, placing it beyond casual adoption for developers without C++ experience. All four methods achieved 100% success rate across all 336 operations, confirming robustness on consumer hardware. Fast-simplification's simplification ceiling on the Church asset at 79.5% actual versus a 90% target, and its structurally damaging behavior on assets with thin features and voids, represent a usability concern that success rate alone does not capture. The method completes without error but produces output that requires manual inspection and potential rejection.

Taken together, these findings do not support a single recommendation. They support a context-dependent selection framework. Meshoptimizer provides the strongest default choice for heterogeneous asset pipelines because it combines the highest perceptual preference, the fastest processing, `pip`-level accessibility, and the absence of a disqualifying failure mode on any tested asset class. Open3D is the strongest choice when geometric accuracy is the primary criterion or, potentially, when the asset library is predominantly organic. CGAL is the strongest choice for moderate reduction levels on compact hard-surface assets, provided the developer can accommodate the installation complexity and accepts its reliability degradation at aggressive reduction. Fast-simplification offers a speed advantage on geometry that is confirmed watertight and compact, but its threshold-escalation failure mode on dense or thin-featured geometry limits its reliability as a general-purpose pipeline tool.

6.2 Production Scenario Recommendations

The context-dependent answer in Section 6.1 can be operationalized as a set of production scenarios derived from the asset-class dependence, failure mode analysis, and performance characteristics documented in Chapters 4 and 5. Table 6.1 maps six common production scenarios to the method best supported by the evidence. Because the top three methods are separated by only 3.4 percentage points in perceptual preference, most scenarios have a competitive alternative that may suit a developer's specific constraints.

Production Scenario	Recommended	Alternative	Primary Justification	Key Limitation
Heterogeneous pipeline (mixed asset types)	Meshoptimizer	CGAL (55.9% overall win rate, leads on three assets)	Highest perceptual win rate (57.6%), no disqualifying failure mode on any asset class	Narrow perceptual margin over CGAL (1.7pp); does not lead on geometric accuracy
Hard-surface final bake (weapons, vehicles, architecture)	Meshoptimizer	CGAL (leads perceptually on AK-74, M9, Jeep)	Leads perceptually on Church (70.8%) and Watermill (79.9%); preserves shading discontinuities	CGAL outperforms on compact hard-surface objects (AK-74, M9, Jeep)
Organic or curved asset pipeline	Open3D	Meshoptimizer (above-chance win rate on all assets, no shading risk)	Leads perceptually on Bunker (71.3%) and Mi-8 (72.7%); lowest geometric error overall	Normal averaging degrades hard-surface shading identity; tested on limited organic-like geometry
Moderate reduction only (LOD1, 50%)	CGAL	Meshoptimizer (55.4% at 50%, pip-installable without C++ compilation)	Highest single-reduction win rate at 50% (63.5%)	Drops below chance level at 90% reduction (47.9%); requires C++ compilation
Speed-critical batch processing with confirmed watertight geometry	Fast-Simplification	-	Second fastest method (159 ms mean); pip-installable	Catastrophic failure on thin features and voids; simplification ceiling on dense topology; not suitable for heterogeneous asset libraries
Full LOD chain (50% through 90%)	Meshoptimizer	Open3D (57.1% at 90% reduction, strongest at aggressive levels after meshoptimizer)	Only method maintaining above-chance perceptual preference at all three reduction levels	19.48% Hausdorff outlier on LAV at 90% indicates imperfect aggressive-reduction behaviour

Table 6.1. Production scenario recommendations derived from Chapters 4 and 5 findings. Each recommendation includes a competitive alternative where the data supports one, the primary evidence-based justification, and the most relevant limitation.

The alternative column reflects the narrow margins between competitive methods. A developer who cannot accommodate CGAL's installation complexity can select meshoptimizer for moderate reduction with a modest perceptual trade-off, 55.4% for meshoptimizer versus CGAL's 63.5%. A developer concerned about Open3D's

shading behavior on mixed-geometry pipelines can use meshoptimizer as a safe fallback. Fast-simplification is the only method without a listed alternative, because its niche of fast processing on confirmed-watertight geometry is narrow enough that no other method competes on exactly those terms, and its failure modes make it unsuitable outside that niche.

These recommendations prioritize asset-class routing because the data showed the strongest method differentiation along geometry type. Production constraints beyond asset type, such as iteration speed, quality acceptance criteria, and team technical skill, also influence method selection but did not produce comparably clear separations in the evaluation data. The recommendations are further specific to hard-surface game assets evaluated without textures on a single hardware configuration. The asset-class dependence documented in Chapter 5 suggests that a pipeline routing assets to different methods based on geometry type could outperform any single-method recommendation, but such routing would need to be validated empirically and weighed against the added pipeline complexity. Section 6.4 identifies this as a future research direction.

6.3 Contributions

This research makes contributions across three areas: empirical findings, methodology, and reusable research infrastructure.

The primary empirical contribution is the first systematic multi-metric comparison of accessible mesh simplification methods evaluated on production game assets. Prior comparative studies evaluate methods in isolation, use exclusively academic benchmark models, or measure only geometric accuracy. This study combined geometric accuracy, processing performance, and perceptual quality assessment on eight production assets from an active game title, producing findings that would not emerge from standard benchmarks alone. The asset-class dependence documented in Chapter 5, the geometric-perceptual disconnect at $\rho \approx -0.31$ to -0.39 , and the identification of method-specific failure modes on production geometry all depend on this multi-metric, production-grounded evaluation design. The finding that geometric metrics account for only 10-15% of variance in perceptual preference provides concrete empirical support for a limitation that prior literature has discussed but rarely quantified in a production context.

The methodological contribution is the paired distant-close perceptual evaluation design, which tested the same method comparison at both simulated gameplay viewing distance and close inspection within a single session. The 33.4%

judgment flip rate between viewing conditions demonstrates that viewing distance meaningfully influences perceptual preference, a finding relevant to any future study evaluating LOD quality. The two-level balanced adaptive sampling system maintains coverage across an asymmetric comparison space of 8 assets, 3 reduction levels, 6 method pairs, without requiring complete participation from every participant, providing a practical template for perceptual studies operating under similar constraints.

The infrastructure contribution consists of the complete scripting repository, benchmark automation framework, and perceptual study platform, all of which will be published on GitHub following thesis defense. This includes the batch testing pipeline `mesh_simplifier_batch.py` for batch simplification, geometric accuracy analysis via `analyze_benchmarks.py`, perceptual data analysis via `analyze_perceptual.py`, scalability analysis via `analyze_scalability.py`, and the Flask-based perceptual evaluation application with adaptive sampling logic. Publishing these tools addresses a documented barrier in computational graphics research: (Bonneel et al., 2020) found that only a small portion of published graphics methods include working code, and this study's own experience with Neural Mesh Simplification documented in Appendix A.2.2 demonstrated firsthand how absent or incomplete research code prevents replication. The published repository enables future researchers to reproduce the benchmark methodology on any geometry they supply, extending the evaluation to asset types, reduction levels, or methods not covered by this study.

6.4 Future Research

The limitations documented in Section 5.4 and the open questions surfaced by the findings suggest six directions for future research, ordered by their potential to expand the boundaries of the current evaluation.

The most consequential extension would be evaluating organic and character asset performance. The production test suite consists entirely of hard-surface game assets, and the asset-class dependence identified in Chapter 5 suggests that method rankings may shift substantially on organic geometry. Open3D's normal averaging, which degrades hard-surface shading identity, may prove consistently beneficial on character models, vegetation, and terrain where smooth shading is desirable. A study using a mixed corpus of hard-surface and organic assets would test whether the recommendation table in Section 6.2 holds across the full range of production geometry, or whether separate recommendations are needed for each asset class. Character models would additionally require preserving skinning weight boundaries during

simplification to avoid animation deformation artifacts, a constraint outside the scope of the static mesh methods evaluated here.

Second, textured rendering evaluation would test whether the perceptual rankings observed under untextured conditions survive the addition of materials. The current evaluation isolates geometric and shading quality from material interaction, which is methodologically clean but does not represent in-game visual conditions. Normal maps can mask the shading loss that Open3D produces on hard-surface assets, while clean metallic surfaces with sharp reflections could amplify it. A study rendering simplified assets with their production materials and evaluating perceptual quality in an interactive viewport rather than static image pairs would provide stronger ecological validity for production recommendations.

Third, asset-class routing deserves empirical validation. The asset-class dependence documented in Section 5.1 implies that directing organic assets to Open3D, compact hard-surface assets to CGAL at moderate reduction, and architecturally complex assets to meshoptimizer could outperform any single-method pipeline. However, this routing logic has not been tested as an integrated system. A study implementing and evaluating an automated routing pipeline, including the classifier that assigns assets to methods and the overall quality of the combined output, would determine whether the added complexity produces meaningful improvement over the single-method default of meshoptimizer.

Fourth, a qualitative perceptual follow-up would address a gap in the current study's design. The pairwise forced-choice methodology captures which method participants prefer but not why. Connecting participant reasoning to the artifact types documented in Section 5.1, such as whether participants penalise shading loss more than geometric distortion, or whether silhouette preservation drives preference on specific assets, would deepen the interpretive framework and inform which artifact types simplification algorithms should prioritize avoiding. A think-aloud protocol or post-comparison questionnaire added to the existing evaluation platform could collect this data without requiring a new study infrastructure.

Fifth, evaluating full LOD chains under production rendering conditions would address two limitations this study could not. Testing complete LOD chains would reveal whether transition smoothness between levels matters more than the absolute quality of any single level, and rendering those chains with production materials and lighting would test whether the perceptual differences measured under controlled conditions persist in practice. The per-method patterns generate a testable prediction:

meshoptimizer's consistent degradation should produce smoother transitions than fast-simplification's threshold escalation.

Sixth, neural mesh simplification methods warrant re-evaluation as the field matures. This study excluded MeshCNN and Neural Mesh Simplification based on empirical accessibility barriers documented in Appendix A.2, including architectural input limits of 750 edges, memory scaling at 267x beyond available hardware, and topologically invalid output. The neural simplification literature from 2023 to 2026 may have produced methods that overcome these specific barriers. A future study could apply the same multi-metric evaluation framework published with this thesis to any neural method that achieves pip-level installation, consumer-hardware processing, and production-scale input, directly testing whether neural approaches have crossed the accessibility threshold that this study found they had not reached.

6.5 Final Reflection

Mesh simplification is one of the oldest problems in real-time rendering, yet developers who rely on freely available tools have had limited guidance on which ones actually work in their pipelines. This research does not resolve that problem completely. It does not declare a winner, and it cannot generalize beyond hard-surface assets rendered without textures on a single hardware configuration. What it provides is a structured, evidence-based framework for making the decision: which method, for which geometry, at which reduction level, with which trade-offs. The finding that geometric metrics explain only 10-15% of perceptual preference is not a disappointing result. It is an actionable one, because it tells practitioners and researchers that selecting or evaluating a simplification method based on Hausdorff distance alone will mislead them, and that perceptual evaluation, even at a modest scale, reveals information that automated metrics cannot. The benchmark framework, perceptual study platform, and analysis pipeline published alongside this thesis are designed to outlive it. Any researcher or developer can extend the evaluation to new methods, new asset types, or new metrics without rebuilding the infrastructure. If this work succeeds in its ambition, the next developer choosing between free simplification tools can make a more informed decision rather than experimenting without guidance.

Bibliography

343 Industries. (2021). *Halo Infinite [Video game]*. Xbox Game Studios.

Activision GDC. (2018). *GDC Vault - Automated Testing and Profiling for "Call of Duty."*
<https://www.gdcvault.com/play/1025324/Automated-Testing-and-Profiling-for>

Bahirat, K., Lai, C., McMahan, R. P., & Prabhakaran, B. (2018). Designing and evaluating a mesh simplification algorithm for virtual reality. *ACM Transactions on Multimedia Computing, Communications and Applications*, 14(3s).
<https://doi.org/10.1145/3209661>

Blender Foundation. (2024). *Blender Foundation Annual Report 2024 — Blender*.
<https://www.blender.org/news/blender-foundation-annual-report-2024/>

Blender Foundation. (2025). *Decimate Modifier - Blender 4.5 LTS Manual*.
<https://docs.blender.org/manual/en/latest/modeling/modifiers/generate/decimate.html>

Bonneel, N., Coeurjolly, D., Digne, J., & Mellado, N. (2020). Code replicability in computer graphics. *ACM Transactions on Graphics*, 39(4).
<https://doi.org/10.1145/3386569.3392413>

CD Projekt RED. (2015). *The Witcher 3: Wild Hunt [Video game]*. CD Projekt.

CGAL Project. (2025). *GitHub - CGAL/cgal: The public CGAL repository, see the README below*. <https://github.com/CGAL/cgal>

Chen, H. K., Fahn, C. S., Tsai, J. J. P., Chen, R. M., & Lin, M. B. (2006). Generating high-quality discrete LOD meshes for 3D computer games in linear time. *Multimedia Systems*, 11(5), 480–494. <https://doi.org/10.1007/s00530-006-0017-1>

Cignoni, P., Scopigno, R., & Rocchini, C. (1998). *Metro: measuring error on simplified surfaces*.

- Clark, J. H. (1976). *Hierarchical Geometric Models for Visible Surface Algorithms*.
- Cohen, J., Olano, M., & Manocha, D. (1998). Appearance-Preserving Simplification. 0897919998. <https://doi.org/10.1145/280814.280832>
- Corsini, M., Larabi, M. C., Lavoué, G., Petřík, O., Váša, L., & Wang, K. (2013). Perceptual metrics for static and dynamic triangle meshes. In *Computer Graphics Forum* (Vol. 32, Number 1, pp. 101–125). Blackwell Publishing Ltd. <https://doi.org/10.1111/cgf.12001>
- Donya Labs AB. (2025). *Simplygon 10.4.232.0*. https://documentation.simplygon.com/SimplygonSDK_10.4.232.0/
- Epic Games. (2025). *Creating and Using LODs in Unreal Engine | Unreal Engine 5.6 Documentation | Epic Developer Community*. <https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-and-using-lods-in-unreal-engine>
- Forstmann, S. (2014). *GitHub - sp4cerat/Fast-Quadric-Mesh-Simplification: Mesh triangle reduction using quadrics*. <https://github.com/sp4cerat/Fast-Quadric-Mesh-Simplification>
- Garland, M., & Heckbert, P. S. (1997). *Surface Simplification Using Quadric Error Metrics*. <http://www.cs.cmu.edu/>
- Garland, M., & Heckbert, P. S. (1998). *Simplifying Surfaces with Color and Texture using Quadric Error Metrics*. <http://www.cs.cmu.edu/>
- GDC. (2024). *GDC 2024 State Of The Game Industry: game developers on layoffs, generative AI, and more*. <https://www.gamedeveloper.com/business/gdc-2024-state-of-the-game-industry-devs-discuss-layoffs-generative-ai-and-more>
- Hanocka, R., Hertz, A., Fish, N., Giryas, R., Fleishman, S., & Cohen-or, D. (2019). MeshCNN: A network with an edge. *ACM Transactions on Graphics*, 38(4). <https://doi.org/10.1145/3306346.3322959>

- Hattori, S. (2023). *GitHub - astaka-pe/mesh_simplification: Python script for “Quadric Edge Collapse Decimation.”* https://github.com/astaka-pe/mesh_simplification
- Hernández, J. A., & Colom, M. (2023). *Repeatability, Reproducibility, Replicability, Reusability (4R) in Journals’ Policies and Software/Data Management in Scientific Publications: A Survey, Discussion, and Perspectives.*
<http://arxiv.org/abs/2312.11028>
- Hoppe, H. (1996). *Progressive Meshes.*
<http://www.research.microsoft.com/research/graphics/hoppe/>
- Hoppe, H. (1999). *New Quadric Metric for Simplifying Meshes with Appearance Attributes.*
<http://research.microsoft.com/hoppe/>
- InstaLOD. (2025). *SEGA: InstaLOD automates the Asset Pipeline for Sonic Frontiers – InstaLOD.* <https://instalod.com/2025/08/20/instalod-sega-automating-the-asset-pipeline-for-sonic-frontiers/>
- InstaLOD GmbH. (2024). *InstaLOD – The leading solution for 3D optimization, workflow automation and CAD to 3D.* <https://instalod.com/>
- InstaLOD GmbH. (2025). *Licensing – InstaLOD.* <https://instalod.com/licensing/>
- JetBrains. (2021). *Python Developers Survey 2021 Results.*
<https://lp.jetbrains.com/python-developers-survey-2021/>
- Kapoulkine, A. (2025). *GitHub - zeux/meshoptimizer: Mesh optimization library that makes meshes smaller and faster to render.*
<https://github.com/zeux/meshoptimizer>
- Karis, B., Stubbe, R., & Wihlidal, G. (2021). *Nanite: A deep dive.* In ACM SIGGRAPH 2021 Courses (SIGGRAPH ’21). Association for Computing Machinery.
https://Advances.RealtimeRendering.Com/S2021/Karis_Nanite_SIGGRAPH_Advances_2021_final.Pdf.

- Kaszynski, A. (2025). *fast-simplification: Python wrapper around the Fast-Quadric-Mesh-Simplification library [Computer software]*. GitHub.
<https://github.com/pyvista/fast-simplification>
- Kim, Y., Varshney, A., Jacobs, D. W., & Guimbretière, F. (2010). Mesh saliency and human eye fixations. *ACM Transactions on Applied Perception*, 7(2), 1–13.
<https://doi.org/10.1145/1670671.1670676>
- Kitware, Inc. (2025). *GitHub - Kitware/VTK: Mirror of Visualization Toolkit repository*.
<https://github.com/Kitware/VTK>
- Knodt, J. (2025). Single Edge Collapse Quad-Dominant Mesh Reduction. *ACM Transactions on Graphics*, 44(4). <https://doi.org/10.1145/3731143>
- Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Evgeny, B., Alexa, M., Zorin, D., & Panozzo, D. (2018). *ABC: A Big CAD Model Dataset For Geometric Deep Learning*.
<https://deep-geometry.github.io/abc-dataset>
- Lavoué, G., Drelie Gelasca, E., Dupont, F., Baskurt, A., & Ebrahimi, T. (2006). *Perceptually driven 3D distance metrics with application to watermarking*.
- Lee, C. H., Varshney, A., & Jacobs, D. W. (2005). *Mesh Saliency*.
- Lemley, M. A., & Casey, B. (2020). *Fair Learning | Texas Law Review*.
<https://texaslawreview.org/fair-learning/>
- Lin, F., Yue, Y., Hou, S., Yu, X., Xu, Y., Yamada, K. D., & Zhang, Z. (2023). *Hyperbolic Chamfer Distance for Point Cloud Completion*. <https://github.com/Zhang-VISLab>.
- Lindstrom, P., & Turk, G. (1998). *Fast and Memory Efficient Polygonal Simplification*.
- Lindstrom, P., & Turk, G. (1999). Evaluation of Memoryless Simplification. In *IEEE Transactions on Visualization and Computer Graphics* (Vol. 5, Number 2).
- Liu, H.-T. D., Zhang, X., & Yuksel, C. (2024). *Simplifying Triangle Meshes in the Wild*.
<http://arxiv.org/abs/2409.15458>

- Luebke, D. P. (2001). *A Developer's Survey of Polygonal Simplification Algorithms*.
- Mantiuk, R. K., Tomaszewska, A., & Mantiuk, R. (2012). *Comparison of four subjective methods for image quality assessment* (Vol. 0). www.interscience.wiley.com
- Microsoft Game Dev. (2024). *The State of Simplygon in 2024*.
<https://developer.microsoft.com/en-us/games/articles/2024/08/the-state-of-simplygon-in-2024/>
- Okeremi, M. (2025). *GDC 2025: How Build Insights Reduced Call of Duty: Modern Warfare II's Build Times by 50%*. <https://developer.microsoft.com/en-us/games/articles/2025/03/gdc-2025-build-insights-call-of-duty-modern-warfare/>
- Open3D. (2023). *Build from source - Open3D 0.19.0 documentation*.
<https://www.open3d.org/docs/release/compilation.html>
- Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.-M., Rothchild, D., So, D., Texier, M., & Dean, J. (2021). *Carbon Emissions and Large Neural Network Training*.
- Peng, C., Park, S. I., Cao, Y., & Tian, J. (2011). A real-time system for crowd rendering: Parallel LOD and texture-preserving approach on GPU. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 7060 LNCS, 27–38. https://doi.org/10.1007/978-3-642-25090-3_3
- Potamias, R. A., Ploumpis, S., & Zafeiriou, S. (2022). *Neural Mesh Simplification*.
- Riegler, G. (2019). *Mesh simplification [Pull request #945]*. Open3D, GitHub.
<https://github.com/isl-org/Open3D/pull/945>
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., & Fei-Fei, L. (2015). ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115(3), 211–252. <https://doi.org/10.1007/S11263-015-0816-Y>

- Shapiro, P. (2013). *Teaching the open source creative tool, Blender, to high school students* / *Opensource.com*. <https://opensource.com/education/13/5/teaching-blender-students>
- Sharp, N., & Ovsjanikov, M. (2020). *PointTriNet: Learned Triangulation of 3D Point Sets*.
- Simplygon. (2019). *Simplygon Batch*.
https://documentation.simplygon.com/SimplygonSDK_8.3.33500.0/articles/tools/simplygonbatchutility.html
- Sonic Team. (2022). *Sonic Frontiers [Video game]*. SEGA.
- Sorgente, T., Biasotti, S., Manzini, G., & Spagnuolo, M. (2023). A Survey of Indicators for Mesh Quality Assessment. *Computer Graphics Forum*, 42(2), 461–483.
<https://doi.org/10.1111/cgf.14779>
- Stack Overflow. (2024). *Technology | 2024 Stack Overflow Developer Survey*.
<https://survey.stackoverflow.co/2024/technology>
- Stanford University. (n.d.). *The Stanford 3D Scanning Repository*. Retrieved January 15, 2026, from <https://graphics.stanford.edu/data/3Dscanrep/>
- Strubell, E., Ganesh, A., & McCallum, A. (2019). *Energy and Policy Considerations for Deep Learning in NLP*. <https://bit.ly/2JTbGnI>
- Sullivan, C., & Kaszynski, A. (2019). PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK). *Journal of Open Source Software*, 4(37), 1450. <https://doi.org/10.21105/JOSS.01450>
- Symposium on Computational Geometry (SoCG). (2023). *SoCG Test of Time Award 2023*.
<https://apps.utdallas.edu/SOCG23/awards.html>
- Trettner, P., & Kobbelt, L. (2020). Fast and Robust QEF Minimization using Probabilistic Quadrics. *Computer Graphics Forum*, 39(2), 325–334.
<https://doi.org/10.1111/CGF.13933>

- Turk, G. (2000). *The Stanford Bunny*.
<https://www.cc.gatech.edu/~turk/bunny/bunny.html>
- Uccheddu, F., Servi, M., Furferi, R., & Governi, L. (2018). Comparison of mesh simplification tools in a 3D watermarking framework. *Smart Innovation, Systems and Technologies*, 76, 60–69. https://doi.org/10.1007/978-3-319-59480-4_7
- Unreal Wiki. (2025). *Unreal (video game)/Release Notes | Unreal Wiki | Fandom*.
[https://unreal.fandom.com/wiki/Unreal_\(video_game\)/Release_Notes#cite_note-1](https://unreal.fandom.com/wiki/Unreal_(video_game)/Release_Notes#cite_note-1)
- Valve Corporation. (2025). *Steam Hardware & Software Survey*.
<https://store.steampowered.com/hwsurvey/Steam-Hardware-Software-Survey-Welcome-to-Steam>
- Yildiz, Z. C., & Capin, T. (2017). A perceptual quality metric for dynamic triangle meshes. *Eurasip Journal on Image and Video Processing*, 2017(1).
<https://doi.org/10.1186/s13640-016-0157-y>
- Zhang, Y., Tiño, P., Leonardis, A., & Tang, K. (2021). *A Survey on Neural Network Interpretability*. <https://www.acm.org/media-center/2019/march/turing-award-2018>

Appendix A: Method Selection and Implementation Documentation

This appendix provides detailed technical documentation supporting the method selection process described in Chapter 3, including exclusion rationale for rejected candidates, implementation specifications, and empirical accessibility barrier measurements.

A.1 Excluded Methods: Justification Summary

Table A.1 provides the full exclusion rationale for all 11 filtered methods.

Method	Type	Primary Exclusion Rationale
Probabilistic Quadrics	Traditional (C++)	Incomplete implementation - primitives only, no full algorithm
libigl	Traditional (C++)	Functional redundancy - third Garland-Heckbert variant without unique value
VTK	Traditional (C++)	Poor performance - 4-5x slower than existing alternatives
MeshLab	Traditional (GUI)	Incompatible architecture - GUI-only, no programmatic API for batch testing
simplifyjs	Traditional (JavaScript)	Platform incompatibility - requires Node.js subprocess for Python workflows
naver/mesh-simplifier	Traditional (TypeScript)	Platform incompatibility - web-focused, no Python integration
astaka-pe/mesh_simplification	Traditional (Python)	Lack of validation - no peer review, production deployment, or benchmarks
sp4cerat/Fast-Quadric	Traditional (C++)	Component library - wrapped by fast-simplification (functional redundancy)
MeshCNN	Neural (Python)	Fixed 750-edge limit - requires 267x memory scaling for production assets
Neural Mesh Simplification	Neural (Python)	Topologically invalid output - holes, disconnected components, training instability

Table A.1. Overview of excluded methods with primary exclusion rationale. Exclusion categories include incomplete implementation, platform incompatibility with the Python-based test pipeline, functional redundancy with a selected method, and scaling limitations preventing production-asset processing.

A.1.1 Probabilistic Quadrics

The method represented a theoretical advancement in quadric error formulation achieving 50× speedup over SVD-based approaches as published in EUROGRAPHICS 2020. The reference implementation provided C++17 header-only library containing quadric primitive operations including data structures and error calculation functions, but without complete simplification algorithm. Code inspection identified absent components: mesh I/O utilities, edge collapse priority queue, topology preservation logic, iterative simplification loop, and termination criteria handling. Implementing complete algorithm requires estimated 40+ hours of development covering half-edge mesh structure, priority queue, edge collapse with manifold preservation, algorithm integration, and Python bindings. The repository constitutes research contribution rather than production tool. Integration effort exceeded Block B timeline allocation.

A.1.2 libigl

The library provided comprehensive C++ geometry processing functionality with 100MB+ installation footprint. QSlim implementation represented Garland-Heckbert QEM variant already covered by fast-simplification and Open3D selections. Recent Python bindings via `nanobind` remain untested for production stability. The library's broad scope spanning parameterization, boolean operations, deformation, and visualization exceeded focused simplification comparison requirements. Inclusion would have introduced third Garland-Heckbert implementation without unique algorithmic evaluation dimension.

A.1.3 VTK

The Visualization Toolkit (Kitware, 2025) implemented `vtkQuadricDecimation` using Garland-Heckbert QEM. PyVista benchmark documentation demonstrated performance 4-5× slower than Fast-Quadric alternatives on equivalent hardware. Installation exceeded 2GB, designed primarily for scientific visualization workflows. Functional redundancy existed with fast-simplification, which wraps identical algorithm with superior performance through Cython optimization. Including both evaluates implementation efficiency rather than algorithmic differences, beyond research scope.

A.1.4 MeshLab

The desktop GUI application architecture proved incompatible with automated batch testing methodology. Command-line interface documentation focuses on GUI

usage with programmatic API poorly specified. GUI-based parameter selection complicates precise methodology documentation and reproducibility. The VCGLib backend requires C++ compilation similar to CGAL complexity without Python accessibility advantages. Excluded in favor of programmatic APIs enabling systematic testing.

A.1.5 Platform-Incompatible Methods

JavaScript via `simplifyjs` and TypeScript via `naver/mesh-simplifier` implementations exhibit platform incompatibility with Python-based Blender workflows. Integration requires `Node.js` subprocess infrastructure contradicting accessibility objectives. Target audience mismatch exists between web developers and game development research focus. Implementations constitute ports of existing C++ algorithms already represented by native Python alternatives.

A.1.6 Unvalidated Personal Projects

`astaka-pe/mesh_simplification` (Hattori, 2023) is Pure Python implementation that lacks academic validation through peer review, production deployment, or published benchmarks. Baseline quality and performance characteristics cannot be established without validation study. Personal repository status with single maintainer raises long-term support concerns. Academic research integrity requires validated method comparisons.

A.1.7 Component Libraries

`sp4cerat/Fast-Quadric-Mesh-Simplification` (Forstmann, 2014) is C++ header-only library that serves as foundation for `pyfqmr` and `fast-simplification` wrapper implementations. Direct usage requires custom Python binding development duplicating existing wrapper functionality. Functional redundancy exists as no justification supports including both underlying library and accessible wrapper. Header-only format necessitates C++ compilation in target applications, contradicting accessibility premise addressed by wrapper libraries.

A.2 Neural Methods Scalability Analysis

A.2.1 MeshCNN

The architecture imposed a fixed 750-edge maximum input size determined by the pooling schedule design of $750 \rightarrow 600 \rightarrow 450 \rightarrow 300 \rightarrow 180$ edges. Production game assets contain 100,000-500,000 edges, representing $133\text{-}667\times$ larger complexity than the supported input range. Memory scaling exhibited linear relationship based on training observations:

$$\text{GPU_memory} = (\text{input_edges} / 750) \times 500 \text{ MB}$$

The 750-edge SHREC-16 training meshes require approximately 500 MB GPU memory.

Asset Type	Edge Count	Required Memory	Hardware Requirement
Simple prop	10,000	6.7 GB	Exceeds RTX 3060 6 GB
Weapon	25,000	16.7 GB	RTX 3090 24 GB
Vehicle	100,000	66.7 GB	Multi-GPU cluster
Hero character	500,000	333 GB	5-8x H100 GPUs (\$60,000+)

Table A.2. Projected GPU memory requirements for MeshCNN processing of production game assets based on measured linear scaling of 500 MB per 750 edges.

Architecture extension to production scale would require fundamental network redesign incorporating hierarchical or progressive processing, custom training dataset creation using high-polygon game assets with ground truth labels where no existing suitable dataset exists, multi-GPU distributed training infrastructure with 80GB+ VRAM per device, and development timeline of 6-12 months. The resource requirements include $267\times$ memory scaling factor relative to available hardware and \$50,000+ infrastructure investment, constituting scope beyond thesis constraints. The architectural limitation creates circular dependency where neural method application requires traditional simplification preprocessing from 100,000 down to 750 edges before neural processing, defeating the research purpose of comparing methods as independent alternatives.

A.2.2 Neural Mesh Simplification

Neural Mesh Simplification (Potamias et al., 2022) decomposes simplification into three learned stages: a point sampler selects vertices via multinomial sampling, an

edge predictor establishes connectivity between sampled vertices using attention networks, and a face classifier assigns inclusion probabilities to candidate triangles. The method claims 10x speedup over QEM (Potamias et al., 2022). No official implementation accompanied the paper. Investigation used the only publicly available third-party implementation by Martin Normark on GitHub under `martinnormark/neural-mesh-simplification`, which the author describes as work-in-progress. The investigation spanned approximately 40-50 hours of active researcher effort across two hardware configurations and multiple training approaches.

Initial testing on an RTX 3060 with 6 GB VRAM, establishing the method's memory scaling characteristics on consumer hardware. Systematic testing with meshes of increasing vertex counts demonstrated successful processing at 5,000 vertices with 3.2 GB peak memory and 8,498 vertices with 5.0 GB peak, with failures at 8,639 vertices due to memory fragmentation despite 5.2 GB usage remaining below the 6 GB capacity. Linear regression on the successful measurements yielded a scaling relationship of approximately 0.57-0.6 GB VRAM per 1,000 vertices. Production game assets containing 20,000-100,000 vertices would require 12-60 GB VRAM based on this relationship, placing them beyond consumer GPU capabilities. The primary memory bottleneck was identified through source code examination: the `generate_candidate_triangles` function creates a dense adjacency matrix of dimensions $N \times N$ vertices, requiring $O(n^2)$ memory. For a 48,000-vertex mesh this allocates 9.2 GB for the adjacency matrix alone, compared to approximately 1.2 MB for a sparse edge list representation of the same connectivity.

The investigation continued on an RTX 4080 with 16 GB VRAM to determine whether the memory barriers reflected fundamental architectural constraints or merely insufficient VRAM. The training dataset comprised 57 meshes from the ABC Dataset (Koch et al., 2018), filtered to 12,000 vertices or fewer, split into 45 training and 11 validation samples.

Four iterations of the candidate triangle generation function were developed, each addressing limitations identified in the previous version. The original implementation's dense adjacency matrix was first replaced with a PyTorch sparse COO tensor, which reduced peak allocation but introduced a directional edge lookup bug: triangle enumeration checked only source-to-destination edges, missing triangles whose closing edge was stored in the reverse direction. The second iteration added bidirectional edge lookup through a helper function checking both directions, resolving the missed-triangle problem. The third iteration, which remained the active code for all subsequent training, implemented a full sparse edge dictionary with bidirectional

probability lookup, canonical triangle representation to eliminate duplicates, and a `torch.no_grad()` wrapper to exclude the enumeration from gradient computation. These optimizations reduced memory consumption for the triangle enumeration phase from $O(n^2)$ to $O(E)$, where E is the number of edges.

Despite the sparse refactoring, out-of-memory failures persisted during training. Configuration included batch size 1, mixed precision FP16 training, gradient clipping at 0.5, per-batch CUDA cache clearing, and gradient accumulation over 8 steps. Training ran for 20 epochs before early stopping triggered. Out-of-memory errors occurred on 12 of 45 batches per epoch at a 27% skip rate, consistently affecting meshes exceeding approximately 5,000 vertices. The OOM errors originated from pipeline stages beyond triangle generation, including the face classifier's k-nearest-neighbor graph construction via `torch.cdist` and the loss computation over candidate triangles. A representative error message at batch 17 with 13,907 nodes reported an attempted allocation of 7.95 GB with 2.14 GB reserved but unallocated due to fragmentation, confirming that PyTorch's caching allocator behavior on 16 GB consumer hardware provided insufficient headroom for the method's variable-sized allocations.

Training loss remained effectively flat across all 20 epochs, decreasing from 5.59 at epoch 1 to 5.53 at epoch 20. Loss component analysis at epoch 20 revealed that the overlapping triangles loss dominated at a mean of 159.6, clamped to the configured maximum of 200.0, indicating the model consistently produced self-overlapping geometry. The chamfer distance averaged 1.42, surface distance 2.02, and collision loss 4.93, contributing to the total but showed no convergent trajectory.

Source code examination identified a gradient flow obstruction in the face selection mechanism. The forward pass selects faces via hard boolean thresholding:

```
threshold = torch.quantile(face_probs, 1 - self.target_ratio)
initial_mask = face_probs >= threshold
```

This boolean operation has zero gradient, blocking backpropagation from the reconstruction quality losses to the face classifier. The paper's architecture requires a differentiable selection mechanism, such as Gumbel-Softmax or a straight-through estimator, for the face classifier to learn which triangles to include based on the quality of the resulting mesh. The implementation's hard selection means the classifier receives no training signal about whether its selections produced good or poor geometry. Additionally, the manifold enforcement post-processing step, which selects the two highest-probability faces per edge as specified in the paper,

calls `.detach().cpu().numpy()`, explicitly severing the computation graph at that point.

Inference testing on meshes within the training distribution produced topologically invalid output. Mesh topology validation identified extensive surface discontinuities including holes throughout the surface, non-manifold edge conditions, and multiple disconnected components where single watertight surfaces were expected. Visual inspection revealed fragmented geometry rather than continuous simplified surfaces. These output characteristics are consistent with the gradient flow obstruction: without a training signal for face selection quality, the classifier has no mechanism to learn coherent triangle inclusion patterns.

Independent corroboration was obtained through direct correspondence with Martin Normark, the author of the third-party implementation, initiated in January 2026. Normark confirmed experiencing identical training instability and reported no successful inference from his implementation, stating: "I had the same issues as you did, I tried lots of stuff to tweak the training loop but never succeeded. The loss curve has very erratic behavior" (M. Normark, personal communication, January 2026). He further reported that the original paper's authors had communicated plans to release official code and model weights, which never materialized, leading him to conclude: "I'm beginning to doubt the reproducibility of the model" (M. Normark, personal communication, January 2026). Normark confirmed he used an Apple M1 chip for his experiments and had no trained checkpoints available. Permission to cite this correspondence was explicitly granted.

Separately, a detailed email was sent to Dr. Rafail Potamias at Imperial College London requesting implementation details, trained checkpoints, hardware requirements, training dataset specifications, and awareness of the third-party implementation's divergence from the published method. A follow-up message was sent via LinkedIn. No response was received as of the thesis submission date.

The method was excluded based on converging evidence across five dimensions. Topologically invalid inference output prevented geometric accuracy metric computation, as Hausdorff distance and RMSE measurements require manifold mesh inputs for meaningful results. Hardware scalability limitations restrict processing to approximately 73% of the training dataset on a 16 GB consumer GPU at a 27% batch OOM rate, while production game assets require 12-60x available VRAM based on measured scaling. A gradient flow obstruction in the face selection mechanism prevented the face classifier from receiving training signal about reconstruction quality. An independent implementer confirmed identical failures and expressed doubts about

reproducibility. The original authors neither released the promised official implementation nor responded to requests for implementation details. The combined investigation, totaling approximately 40-50 hours of active effort across two GPU configurations, provides empirical evidence validating the thesis premise that neural mesh simplification methods remain inaccessible to resource-constrained developers due to compounding barriers in hardware requirements, implementation maturity, and reproducibility.

A.3 Traditional Method Implementation: Key Technical Challenges

A.3.1 CGAL

Official Python bindings in the `cgal-swig-bindings` repository exclude the Surface Mesh Simplification module despite coverage of other CGAL components including triangulations, Alpha shapes, and convex hull algorithms. Community alternative `seagullmesh` remains in pre-alpha development status without simplification functionality as of December 2024. Custom binding development was estimated at 20-30 hours. The selected integration architecture uses a standalone C++ executable compiled once by the researcher. The executable accepts mesh files and command-line parameters, performs simplification using CGAL's Lindstrom-Turk implementation, and outputs both simplified mesh geometry and JSON-formatted performance statistics. The Python testing framework invokes the executable via `subprocess` calls with structured data exchange through JSON parsing.

Dependency resolution via `vcpkg` package manager identified 109 required packages including approximately 90 Boost libraries covering graph algorithms, property maps, serialization, geometry, and threading, plus mathematical computation libraries including GMP for arbitrary precision arithmetic and MPFR for multi-precision floating-point, plus build infrastructure components including CMake, PowerShell Core, and compression utilities, and CGAL core library with Surface Mesh Simplification module. Compilation from source required 62 minutes on standard hardware with total disk space consumption of 2.5 GB. Code developed from CGAL 5.x documentation failed compilation with CGAL 6.1 installed by `vcpkg` due to API breaking changes. The changes included deprecation of `Count_ratio_stop_predicate` class replaced by `Edge_count_ratio_stop_predicate`, and removal of manual index map specification as `Surface_mesh` type manages indices automatically in version 6.1. Migration required consultation of CGAL changelogs and updated API documentation. Template compilation

errors exceeded 140 lines requiring C++ template metaprogramming expertise for diagnosis and resolution.

Format compatibility testing revealed systematic PLY I/O failures across both binary and ASCII PLY variants, consistently producing "mesh is empty" errors despite file validity confirmed through successful loading in PyVista. Root cause remains undetermined after investigation including format specification review, absolute path testing, and minimal test file generation. OBJ format succeeded immediately without issues. The finding established OBJ as standardized format across all implementations, ensuring testing consistency and eliminating per-method conversion complexity. Bohemia Interactive production assets stored in FBX format undergo batch conversion to OBJ via Blender scripting prior to testing, with conversion validation confirming triangle count preservation, UV coordinate maintenance, and material boundary integrity.

A.3.2 meshoptimizer API Discovery

Pre-release Python bindings at version 0.2.20a5 exhibit API divergence from C++ library documentation. Expected keyword argument interface produced type errors during invocation. Systematic API introspection via `inspect.signature()` and `help()` functions revealed buffer-based calling convention requiring pre-allocated destination array with integer return count, diverging from Python convention of returning new objects. The corrected implementation allocates destination buffer sized to input index array length, invokes simplification function with buffer and mesh data arrays, and extracts valid simplified indices from the portion of buffer indicated by return count. The API discovery methodology through systematic introspection provided reproducible debugging approach applicable to other undocumented pre-release package bindings.

A.3.3 fast-simplification API Compatibility

PyVista library evolution deprecated the `n_faces` property in favor of `n_cells` for strict cell counting semantics between version updates. Backward-compatible accessor implementation uses conditional attribute checking to select appropriate property based on PyVista version availability, maintaining functionality across library versions without requiring dependency pinning to specific releases. The implementation demonstrates defensive programming practices necessary for managing actively developed dependencies in production workflows.

A.4 Empirical Accessibility Barrier Measurements

A.4.1 Installation Complexity Quantification

Installation time, dependency count, and storage requirements were measured systematically across all traditional methods to establish quantifiable accessibility baselines.

Method	Installation	Time	Dependencies	Disk Space
fast-simplification	pip install	1.5 min	3 packages	45 MB
Open3D	pip install	2 min	5 packages	52 MB
meshoptimizer	pip install	1.5 min	4 packages	18 MB
CGAL	vcpkg + CMake	62 min	109 packages	2.5 GB

Table A.3. Installation time, dependency count, and disk space measured across all four selected methods.

A.4.2 Barrier Multipliers

Installation time comparison between CGAL and pip-installable alternatives yielded consistent multiplier patterns. CGAL installation required 62 minutes compared to fast-simplification's 1.5 minutes, a 41× time barrier. Open3D installed in 2.0 minutes for a 31× barrier, and meshoptimizer in 1.5 minutes for a 41× barrier. Mean installation time multiplier: 38×.

Dependency complexity comparison shows CGAL requiring 109 packages compared to fast-simplification's 3, a 36× multiplier. Open3D requires 5 packages for a 22× multiplier, and meshoptimizer 4 packages for a 27× multiplier. Mean dependency count multiplier: 28×.

Storage requirements show CGAL consuming 2,500 MB compared to fast-simplification's 45 MB, a 55× multiplier. Open3D requires 52 MB for a 48× multiplier, and meshoptimizer 18 MB for a 139× multiplier. Mean disk space multiplier: 81×.

These empirical measurements demonstrate quantifiable accessibility gaps in academic tools despite identical zero licensing costs. CGAL exhibited 38× time requirement, 28× dependency complexity, and 81× storage overhead versus pip-installable alternatives. Barriers reflect organizational and packaging decisions rather

than algorithmic necessities, supporting thesis premise that professional-grade tools remain effectively inaccessible to resource-constrained developers.

A.4.3 Skill Escalation Analysis

Pip-based methods, namely fast-simplification, Open3D, and meshoptimizer, require Python virtual environment management and basic package installation. Stack Overflow Developer Survey 2024 reported Python usage among 51% of developers, though only 9% of Python developers work in game development (JetBrains, 2021). These requirements represent junior-to-intermediate proficiency, constituting the most accessible tier.

CGAL requires C++17 syntax, CMake configuration, vcpkg operation, and template metaprogramming for debugging compilation errors exceeding 140 lines. While general C++ usage reaches 23% of developers (Stack Overflow, 2024), template metaprogramming represents advanced specialization. No surveys quantify this expertise, though barriers documented in Section A.3.1 requiring 62-minute compilation and 109 dependencies indicate accessibility limited to senior-level developers with computational geometry background.

Neural methods require PyTorch expertise, CUDA programming, GPU memory management, and professional hardware requiring 24+ GB VRAM costing \$5,000-60,000. While 76% of developers use AI assistive tools like ChatGPT (GDC, 2024), custom neural network implementation represents specialized expertise combined with substantial hardware investment, likely excluding most indie developers on consumer hardware. These tiers demonstrate categorical accessibility differences: pip-installable tools accessible to basic-intermediate Python users with the broadest coverage, C++ compilation requiring advanced expertise with substantially narrower reach, and neural methods requiring ML specialization plus professional hardware with the narrowest coverage. While exact percentages remain unquantified by industry surveys, documented barriers validate that accessibility constraints extend beyond licensing costs to technical implementation feasibility.

A.4.4 Barrier Compounding Analysis

Neural methods exhibit simultaneous barriers across all five accessibility dimensions: time at 40 hours of setup investment, cost of \$5,000-60,000 for hardware, skills in machine learning domain expertise, complexity of environment debugging requirements, and hardware specifications requiring professional-grade VRAM. These barriers combine multiplicatively rather than additively. Developers possessing

machine learning expertise still encounter \$5,000-60,000 hardware cost barriers prohibitive for indie studio budgets. Developers possessing professional GPU hardware still require 40 hours setup time investment and machine learning training methodology knowledge. Combined requirements create near-total exclusion of target population from independent adoption.

CGAL exhibited three barrier types covering time, complexity, and skills, without cost barrier beyond standard computing hardware ownership. Methods installable via `pip` exhibit only minor time barrier of 5 minutes for setup, accessible to developers with basic Python proficiency. Barrier structure analysis demonstrated that accessibility constraints compound through multiplication of individual barrier types rather than simple addition.

Appendix B: Detailed Benchmark and Perceptual Data

This appendix provides per-reduction and per-asset breakdowns supporting the aggregate results presented in Chapter 4. Tables and figures are organized by measurement dimension: processing performance in Section B.1, geometric accuracy in B.2, per-asset detail in B.3, perceptual quality in B.4, and scalability in B.5. All values use the same normalization and units described in the methodology in Chapter 3 and the main results in Chapter 4.

B.1 Processing Performance by Reduction Level

Tables B.1 through B.3 present method comparison statistics at each reduction level. Figures B.1 through B.3 show processing time distributions at each level, with zoomed panels excluding CGAL for scale clarity.

	Time Mean (ms)	Time Std (ms)	Mem Mean (MB)	Hausdorff Mean (%)	Hausdorff Std (%)	RMSE Mean (%)	RMSE Std (%)
Fast-Simplification	57.43	51.6	5.43	2.782	1.933	0.21	0.215
Open3D	336.45	300.26	1.56	0.115	0.081	0.006	0.007
Meshoptimizer	21.47	17.19	1.54	0.209	0.102	0.017	0.015
CGAL	1212.35	864.09	16.39	0.145	0.072	0.005	0.004

Table B.1. Method comparison at 50% reduction level. Time and memory values represent means across all eight production assets. Geometric accuracy values are normalized to bounding box diagonal.

	Time Mean (ms)	Time Std (ms)	Mem Mean (MB)	Hausdorff Mean (%)	Hausdorff Std (%)	RMSE Mean (%)	RMSE Std (%)
Fast-Simplification	204.64	404.45	2.58	2.811	1.447	0.266	0.229
Open3D	505.73	438.31	1.88	0.868	0.882	0.03	0.026
Meshoptimizer	24.31	20.0	1.74	3.301	5.482	0.073	0.04
CGAL	1558.45	1120.78	16.33	2.528	4.721	0.086	0.178

Table B.2. Method comparison at 80% reduction level.

	Time Mean (ms)	Time Std (ms)	Mem Mean (MB)	Hausdorff Mean (%)	Hausdorff Std (%)	RMSE Mean (%)	RMSE Std (%)
Fast-Simplification	215.06	402.77	2.37	3.059	1.457	0.347	0.229
Open3D	565.87	493.34	2.9	1.664	1.484	0.099	0.107
Meshoptimizer	25.68	20.4	1.43	4.585	6.223	0.145	0.083
CGAL	1667.99	1225.05	16.43	6.547	5.673	0.185	0.206

Table B.3. Method comparison at 90% reduction level.

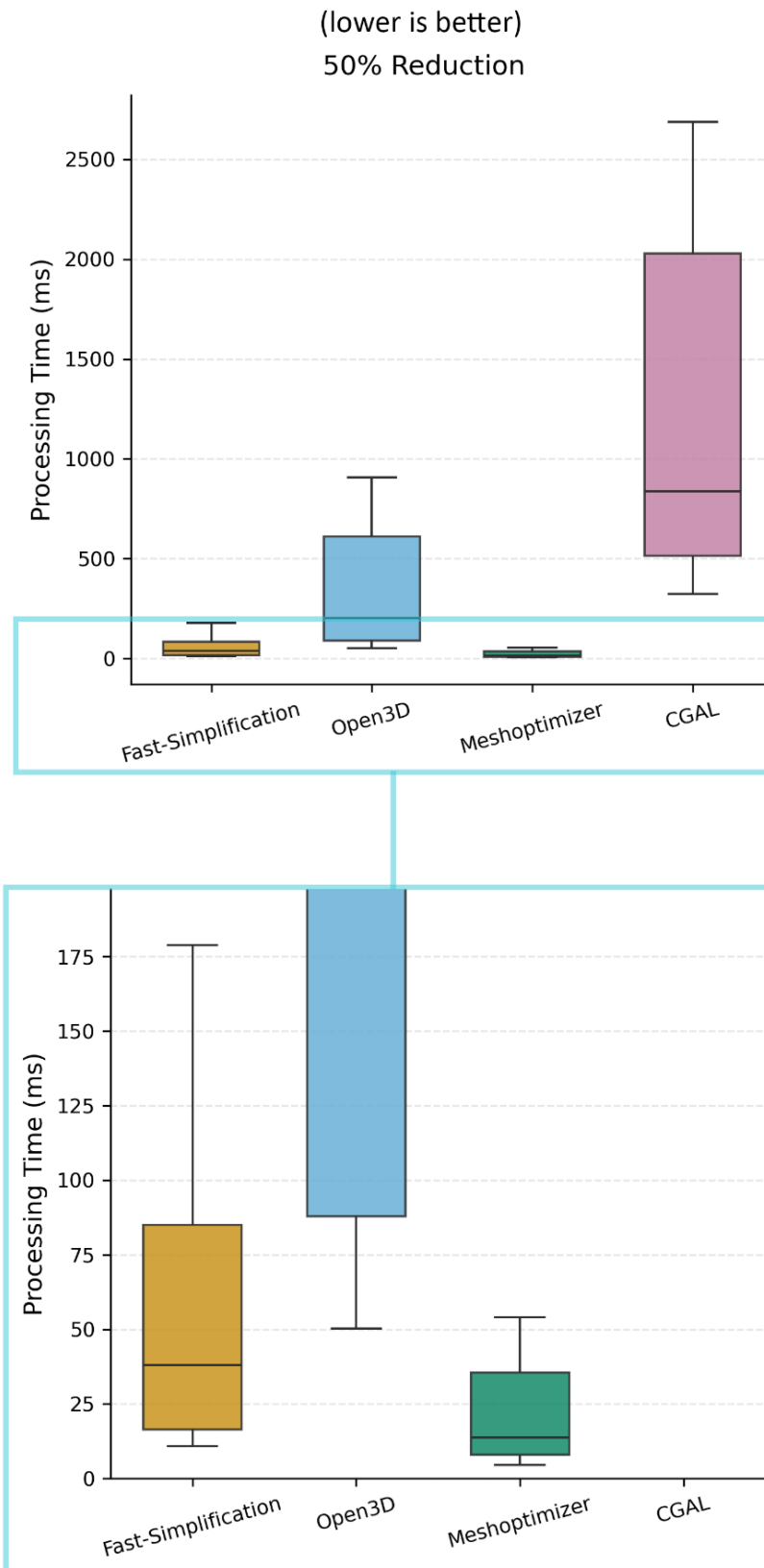


Figure B.1. Processing time distribution by method at 50% reduction. Top: full range including CGAL. Bottom: zoomed view excluding CGAL.

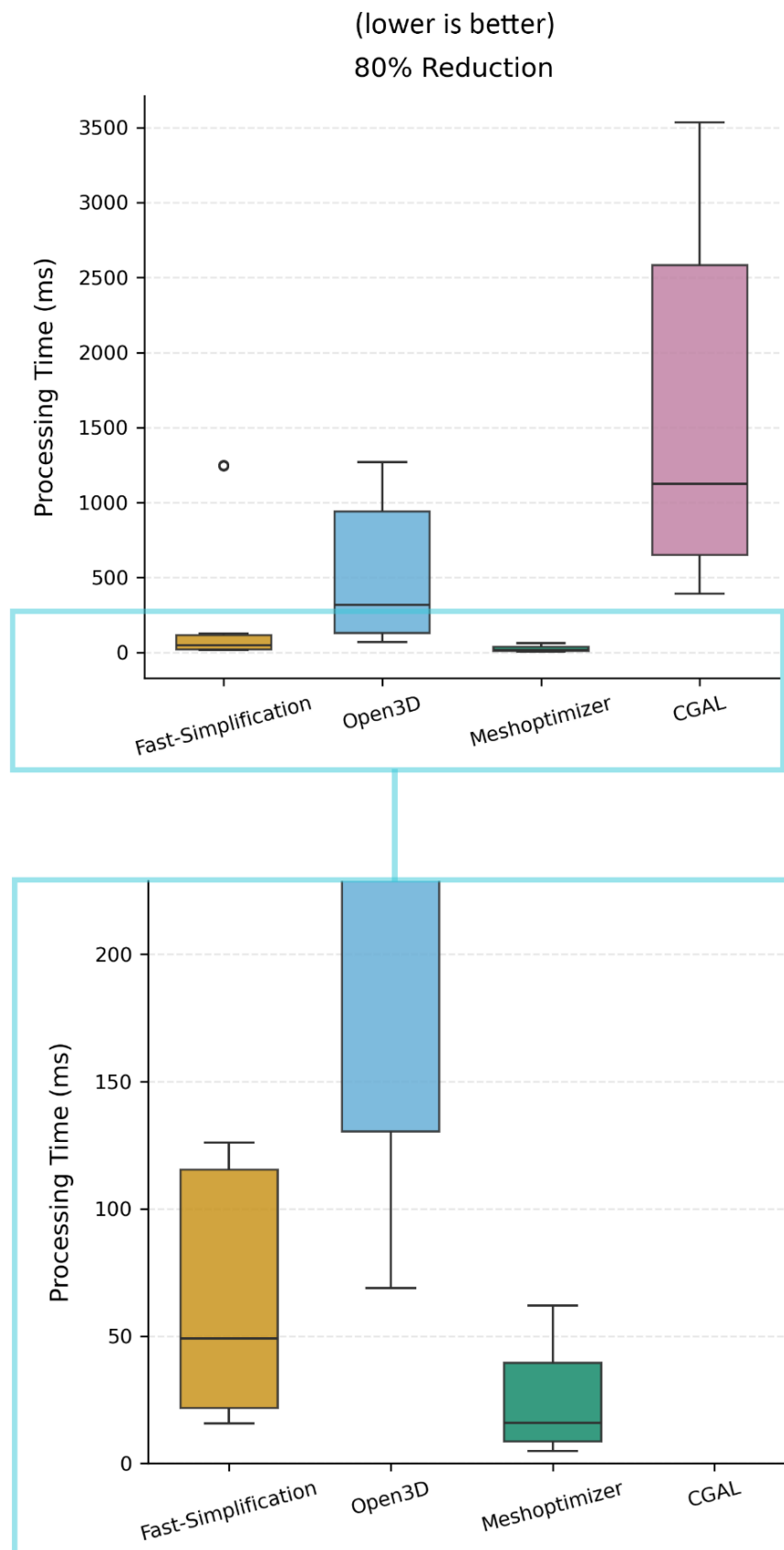


Figure B.2. Processing time distribution by method at 80% reduction.

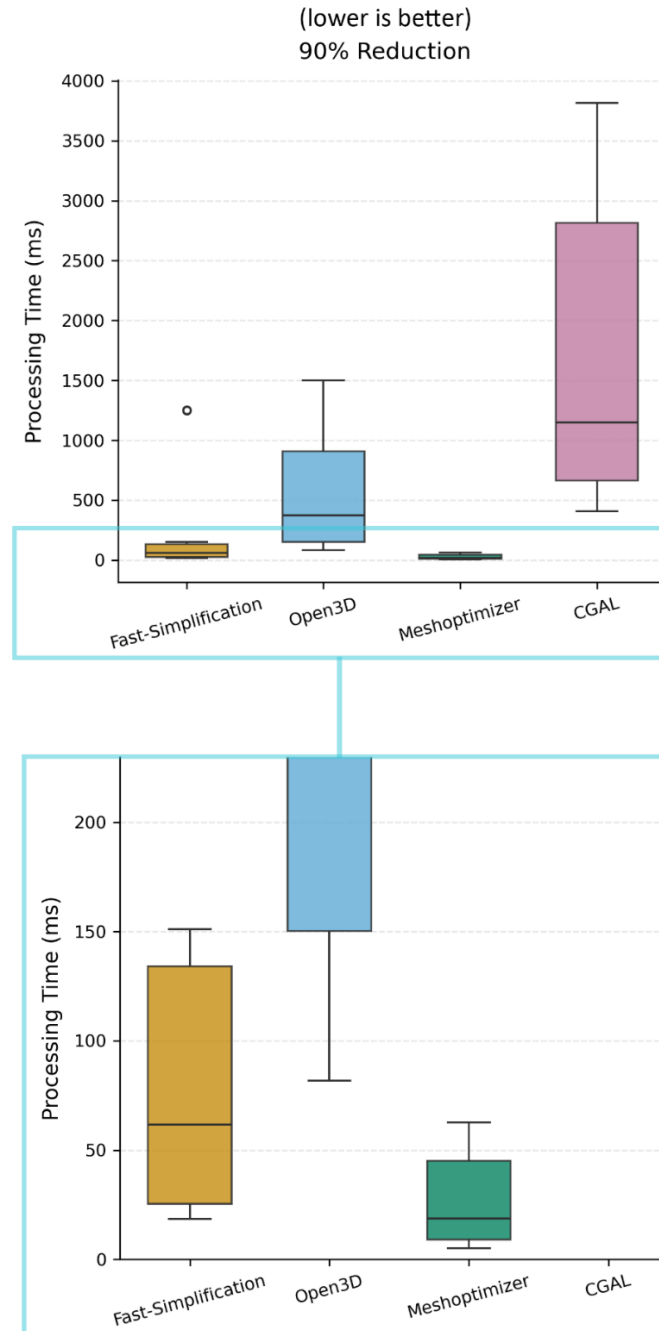


Figure B.3. Processing time distribution by method at 90% reduction.

B.2 Geometric Accuracy by Reduction Level

Figures B.4 and B.5 present the geometric accuracy comparison at 50% and 80% reduction levels. The equivalent 90% reduction figure appears in Chapter 4 as Figure 4.5. Each figure shows normalized Hausdorff distance as maximum local deviation and normalized RMSE as mean surface deviation per method and asset, with values normalized to bounding box diagonal to enable cross-asset comparison.

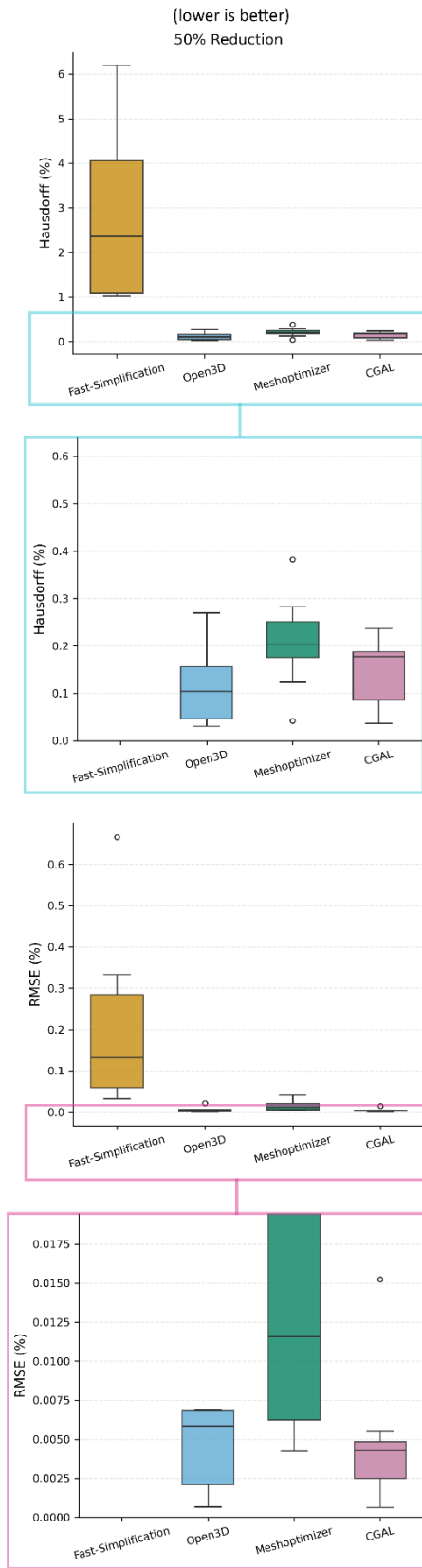


Figure B.4. Geometric accuracy at 50% reduction. Top: normalized Hausdorff distance. Bottom: normalized RMSE. Values represent bounding-box-normalized error.

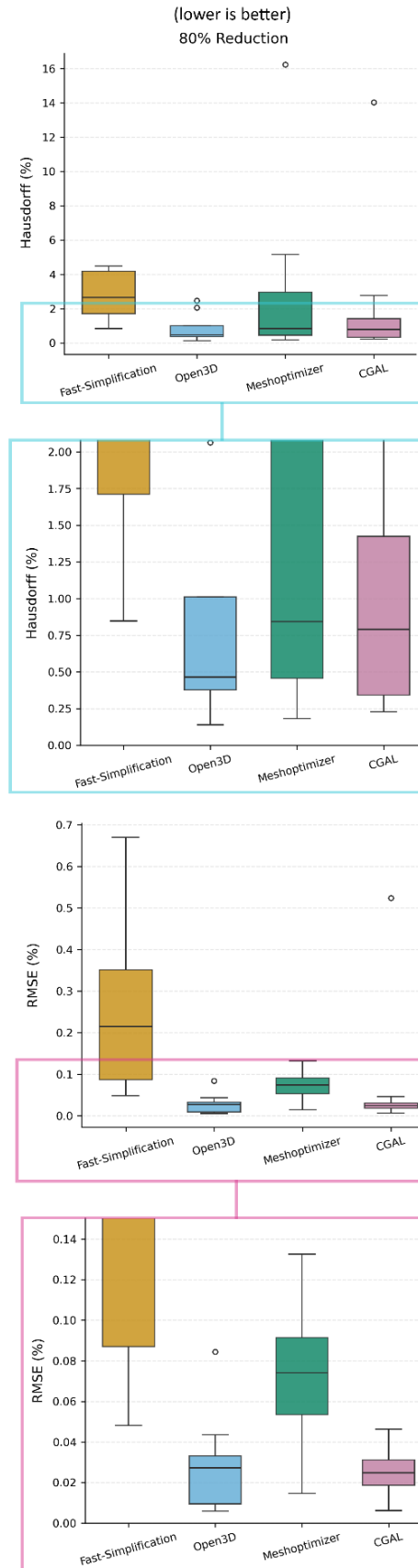


Figure B.5. Geometric accuracy at 80% reduction.

B.3 Per-Asset Detail Tables

Tables B.4 through B.11 present complete benchmark results for each production asset at all three reduction levels. Assets are ordered by ascending face count: AK-74 at 11,194 faces, M9 pistol at 14,932, Bunker at 23,508, LAV at 41,886, Mi-8 helicopter at 118,944, Watermill at 129,124, Jeep at 141,282, and Church at 169,087. Each table reports processing time, peak memory, normalized Hausdorff distance, and normalized RMSE for all four methods.

Method	Reduction	Time (ms)	Memory (MB)	Hausdorff (%)	RMSE (%)	Actual Red.
Fast-Simplification	50%	36.3	15.3	1.023	0.0330	50.0%
Fast-Simplification	80%	43.0	2.8	0.848	0.0482	80.0%
Fast-Simplification	90%	58.0	2.6	0.697	0.1021	90.0%
Open3D	50%	175.1	-6.4	0.040	0.0023	50.0%
Open3D	80%	286.4	-8.3	0.141	0.0101	80.0%
Open3D	90%	327.4	1.2	0.274	0.0281	90.0%
Meshoptimizer	50%	11.0	2.1	0.042	0.0047	50.0%
Meshoptimizer	80%	11.1	0.0	0.182	0.0270	80.0%
Meshoptimizer	90%	17.3	1.6	0.352	0.0583	90.0%
CGAL	50%	682.0	11.1	0.037	0.0015	50.3%
CGAL	80%	898.8	11.2	0.262	0.0103	80.3%
CGAL	90%	978.0	11.6	0.474	0.0283	90.3%

Table B.4. AK-74, 11,194 faces: per-method benchmark results at 50%, 80%, and 90% reduction.

Method	Reduction	Time (ms)	Memory (MB)	Hausdorff (%)	RMSE (%)	Actual Red.
Fast-Simplification	50%	12.0	0.0	1.097	0.0477	50.0%
Fast-Simplification	80%	16.8	0.0	1.717	0.1394	80.0%
Fast-Simplification	90%	22.1	0.0	2.301	0.3262	90.0%
Open3D	50%	57.9	0.0	0.108	0.0069	50.0%
Open3D	80%	85.2	0.0	0.450	0.0276	80.0%
Open3D	90%	106.0	0.0	0.696	0.0825	90.0%
Meshoptimizer	50%	5.1	0.0	0.210	0.0125	50.0%
Meshoptimizer	80%	5.1	0.0	0.906	0.0659	80.0%
Meshoptimizer	90%	5.3	0.0	0.938	0.1689	90.0%
CGAL	50%	342.0	8.3	0.096	0.0046	50.2%
CGAL	80%	428.5	8.2	0.369	0.0248	80.3%
CGAL	90%	463.6	8.2	1.347	0.0677	90.3%

Table B.5. M9 pistol, 14,932 faces: per-method benchmark results at 50%, 80%, and 90% reduction.

Method	Reduction	Time (ms)	Memory (MB)	Hausdorff (%)	RMSE (%)	Actual Red.
Fast-Simplification	50%	61.4	9.4	3.261	0.2691	50.0%
Fast-Simplification	80%	56.1	2.9	3.526	0.2910	80.0%
Fast-Simplification	90%	61.7	2.1	3.711	0.3636	90.0%
Open3D	50%	252.1	0.5	0.270	0.0224	50.0%
Open3D	80%	367.4	14.2	2.065	0.0845	80.0%
Open3D	90%	420.5	-1.1	1.551	0.1480	90.0%
Meshoptimizer	50%	16.4	0.7	0.383	0.0386	50.0%
Meshoptimizer	80%	19.0	1.6	0.780	0.1171	80.0%
Meshoptimizer	90%	20.4	0.0	1.182	0.2237	89.7%
CGAL	50%	994.7	14.2	0.237	0.0153	51.9%
CGAL	80%	1271.2	13.8	14.034	0.5240	81.7%
CGAL	90%	1326.2	14.0	13.240	0.2474	91.1%

Table B.6. Bunker, 23,508 faces: per-method benchmark results at 50%, 80%, and 90% reduction.

Method	Reduction	Time (ms)	Memory (MB)	Hausdorff (%)	RMSE (%)	Actual Red.
Fast-Simplification	50%	87.4	5.4	1.450	0.0807	50.0%
Fast-Simplification	80%	117.6	4.5	1.692	0.0940	80.0%
Fast-Simplification	90%	126.8	3.9	1.806	0.1830	90.0%
Open3D	50%	502.9	6.5	0.101	0.0061	50.0%
Open3D	80%	1011.8	3.8	0.480	0.0269	80.0%
Open3D	90%	872.2	3.9	1.415	0.0618	90.0%
Meshoptimizer	50%	42.2	3.0	0.123	0.0107	50.0%
Meshoptimizer	80%	36.3	2.9	16.247	0.1327	80.3%
Meshoptimizer	90%	50.8	2.9	19.476	0.2888	90.2%
CGAL	50%	2025.6	22.8	0.179	0.0046	50.9%
CGAL	80%	2557.1	22.4	0.719	0.0249	81.1%
CGAL	90%	2854.8	22.3	15.978	0.1579	91.0%

Table B.7. LAV, 41,886 faces: per-method benchmark results at 50%, 80%, and 90% reduction.

Method	Reduction	Time (ms)	Memory (MB)	Hausdorff (%)	RMSE (%)	Actual Red.
Fast-Simplification	50%	87.6	5.0	1.031	0.0634	50.0%
Fast-Simplification	80%	125.9	4.2	1.779	0.0663	80.0%
Fast-Simplification	90%	146.2	3.6	2.443	0.1087	90.0%
Open3D	50%	642.5	3.4	0.048	0.0016	50.0%
Open3D	80%	937.7	3.8	0.416	0.0075	80.0%
Open3D	90%	1111.3	5.6	0.469	0.0146	90.0%
Meshoptimizer	50%	34.0	2.8	0.283	0.0068	50.0%
Meshoptimizer	80%	47.8	2.7	5.176	0.0829	80.0%
Meshoptimizer	90%	42.8	2.7	4.990	0.0878	90.0%
CGAL	50%	2048.5	23.3	0.175	0.0028	53.2%
CGAL	80%	2580.5	23.4	0.971	0.0214	83.5%
CGAL	90%	2697.1	24.5	7.379	0.1463	93.0%

Table B.8. Mi-8 helicopter, 118,944 faces: per-method benchmark results at 50%, 80%, and 90% reduction.

Method	Reduction	Time (ms)	Memory (MB)	Hausdorff (%)	RMSE (%)	Actual Red.
Fast-Simplification	50%	12.7	-2.2	4.016	0.6661	50.0%
Fast-Simplification	80%	15.7	0.0	4.155	0.6703	80.0%
Fast-Simplification	90%	21.0	0.0	4.434	0.7793	90.0%
Open3D	50%	50.5	0.0	0.168	0.0068	50.0%
Open3D	80%	70.0	0.0	0.660	0.0296	80.0%
Open3D	90%	84.0	0.0	4.399	0.3392	90.0%
Meshoptimizer	50%	4.8	0.0	0.198	0.0408	50.0%
Meshoptimizer	80%	5.4	0.0	0.469	0.0823	80.1%
Meshoptimizer	90%	5.4	0.0	2.581	0.1470	90.0%
CGAL	50%	328.3	9.0	0.187	0.0055	51.4%
CGAL	80%	413.1	9.0	2.780	0.0463	81.8%
CGAL	90%	418.1	9.0	7.836	0.6646	91.9%

Table B.9. Watermill, 129,124 faces: per-method benchmark results at 50%, 80%, and 90% reduction.

Method	Reduction	Time (ms)	Memory (MB)	Hausdorff (%)	RMSE (%)	Actual Red.
Fast-Simplification	50%	20.0	1.1	4.187	0.1837	50.0%
Fast-Simplification	80%	23.7	1.1	4.281	0.2898	80.0%
Fast-Simplification	90%	28.4	1.1	4.351	0.3761	90.0%
Open3D	50%	97.9	0.1	0.152	0.0056	50.0%
Open3D	80%	142.2	-0.1	2.466	0.0436	80.0%
Open3D	90%	170.5	-0.1	3.449	0.0991	90.0%
Meshoptimizer	50%	9.1	1.1	0.241	0.0162	50.0%
Meshoptimizer	80%	9.9	0.0	2.225	0.0623	80.4%
Meshoptimizer	90%	10.2	0.0	3.985	0.1383	90.5%
CGAL	50%	565.0	12.0	0.190	0.0039	50.7%
CGAL	80%	757.5	12.0	0.860	0.0261	81.3%
CGAL	90%	730.8	12.0	3.756	0.1144	91.3%

Table B.10. Jeep, 141,282 faces: per-method benchmark results at 50%, 80%, and 90% reduction.

Method	Reduction	Time (ms)	Memory (MB)	Hausdorff (%)	RMSE (%)	Actual Red.
Fast-Simplification	50%	178.9	38.6	6.195	0.3333	50.0%
Fast-Simplification	80%	1244.2	7.2	4.488	0.5308	79.5%
Fast-Simplification	90%	1253.6	6.0	4.727	0.5345	79.5%
Open3D	50%	853.3	12.0	0.030	0.0007	50.0%
Open3D	80%	1262.3	2.6	0.267	0.0060	80.0%
Open3D	90%	1498.6	10.5	1.060	0.0203	90.0%
Meshoptimizer	50%	54.0	4.0	0.193	0.0042	50.1%
Meshoptimizer	80%	58.6	3.9	0.423	0.0147	80.0%
Meshoptimizer	90%	60.2	3.9	3.175	0.0460	90.0%
CGAL	50%	2665.6	29.1	0.056	0.0006	51.4%
CGAL	80%	3457.4	28.8	0.230	0.0062	81.4%
CGAL	90%	3816.5	29.8	2.366	0.0509	91.4%

Table B.11. Church, 169,087 faces: per-method benchmark results at 50%, 80%, and 90% reduction.

Readers examining these tables should note three asset-specific patterns discussed in Chapter 4. First, fast-simplification exhibits a simplification ceiling on the Church at 79.5% actual versus a 90% target. Second, CGAL over-reduces on the Mi-8 by +3.0 to +3.5 percentage points across all levels. Third, meshoptimizer produces a Hausdorff outlier on the LAV at 90% reduction with 19.48% normalized error.

B.4 Perceptual Quality Breakdowns

Table B.12 provides the full perceptual win rate summary across all pairwise comparisons. Figure B.6 presents win rates broken down by reduction level. Table B.13 provides the corresponding numerical values for each reduction level and method.

Method	Overall Win Rate	Overall Wins	Distant Win Rate	Close Win Rate	Δ (Close – Distant)
Fast-Simplification	32.3%	448/1385	34.7%	30.0%	-4.8pp
Open3D	54.2%	762/1406	53.4%	55.0%	+1.7pp
Meshoptimizer	57.6%	791/1373	58.6%	56.6%	-2.0pp
CGAL	55.9%	768/1374	53.4%	58.4%	+5.1pp

Table B.12. Perceptual win rates by method, aggregated across all assets and viewing distances.

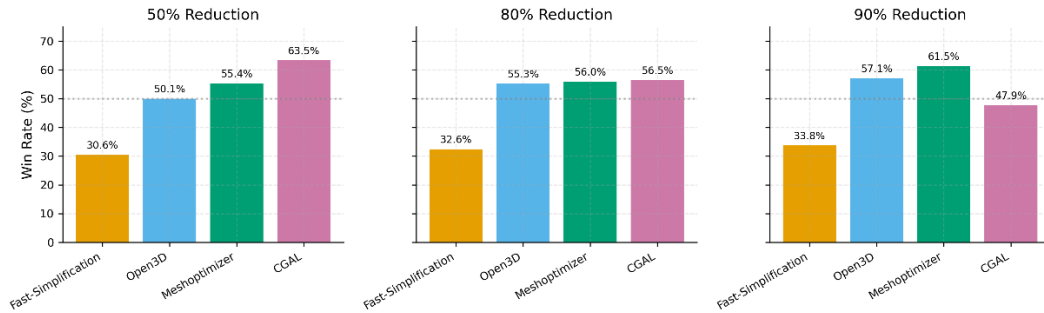


Figure B.6. Perceptual win rates by method and reduction level. Grouped bars show performance at 50%, 80%, and 90% reduction. Dashed line at 50% represents chance-level performance.

Reduction	Fast-Simplification	Open3D	Meshoptimizer	CGAL
50%	30.6%	50.1%	55.4%	63.5%
80%	32.6%	55.3%	56.0%	56.5%
90%	33.8%	57.1%	61.5%	47.9%

Table B.13. Perceptual win rates by method and reduction level.

Figure B.7 presents the per-asset win rate heatmap, showing method performance variation across individual assets and revealing asset-specific method advantages not visible in aggregate statistics.

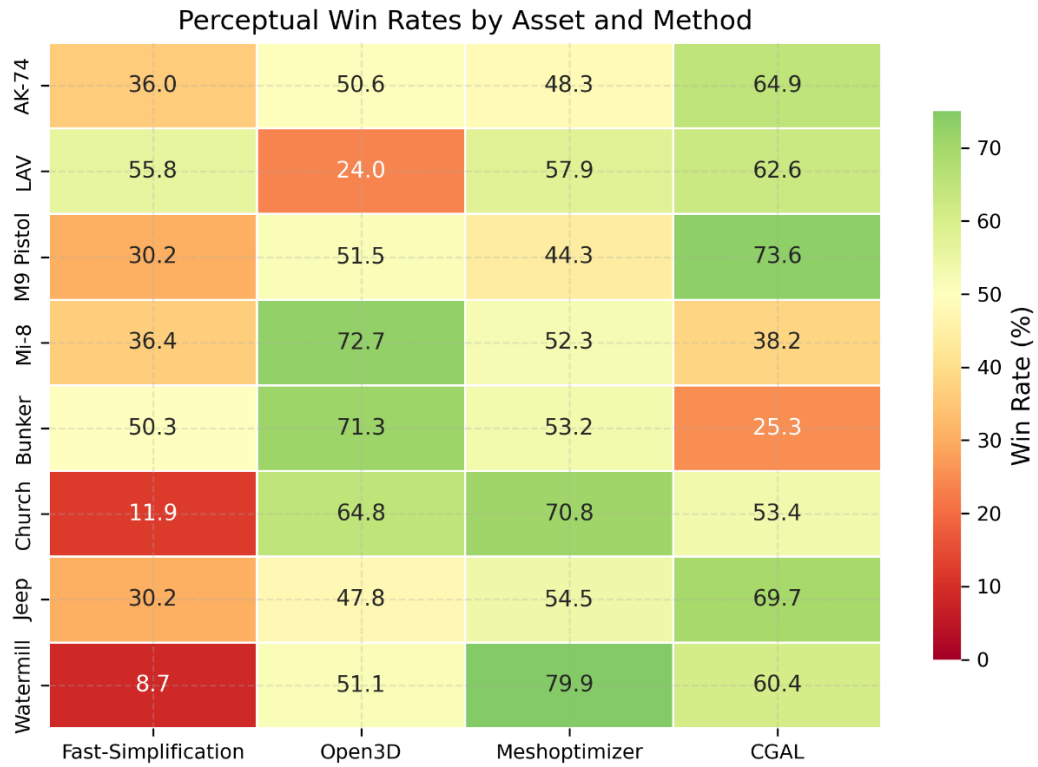


Figure B.7. Perceptual win rate heatmap by method and asset. Color intensity indicates win rate, with values above the 50% chance level indicating preferred performance in pairwise comparison.

Figure B.8 presents the relationship between geometric accuracy metrics and perceptual preference, plotting Hausdorff distance and RMSE against perceptual win rate for each method-asset-reduction combination.

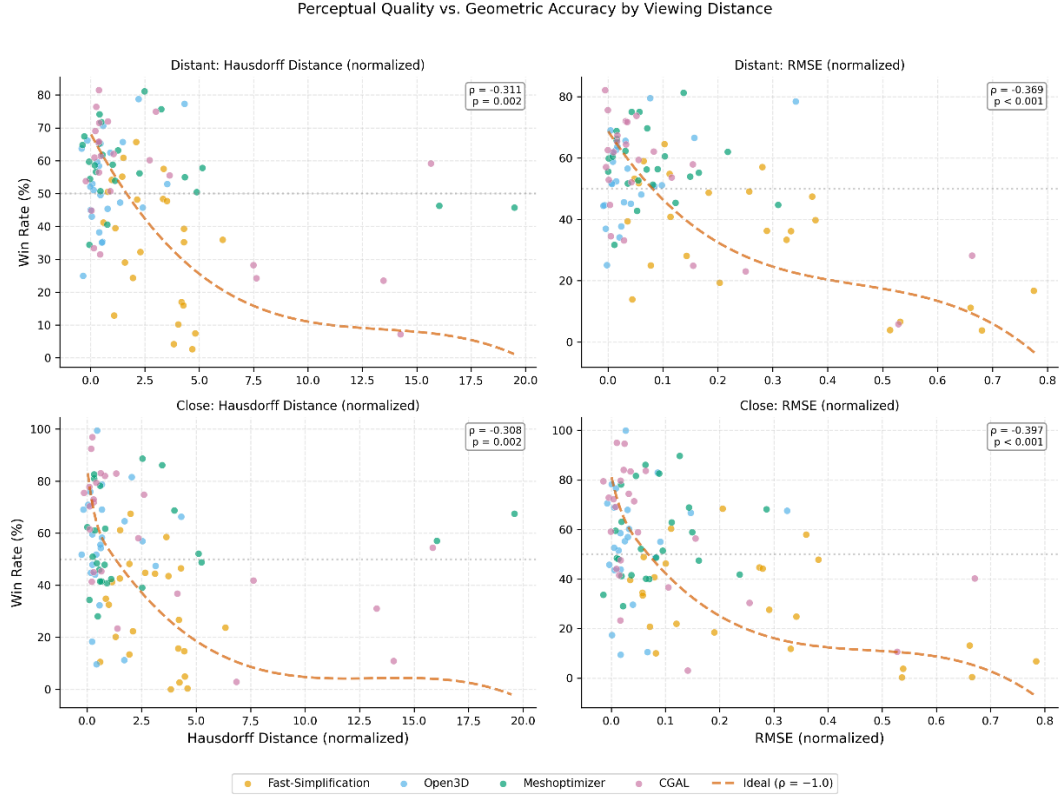


Figure B.8. Geometric-perceptual correlation. Four-panel scatter plot showing Hausdorff distance on the left and RMSE on the right against perceptual win rate at distant viewing in the top panels and close viewing in the bottom panels. Each point represents one method-asset-reduction combination.

B.5 Scalability Data

Table B.14 provides the complete scalability results across all four Stanford 3D Scanning Repository models, namely Bunny, Armadillo, Dragon, and Lucy, at all three reduction levels for all four methods. This 48-row table supports replicability of the scaling analysis presented in Section 4.4 and provides the exact values underlying Figure 4.7.

Asset	Faces	Method	Reduction	Time	Peak Mem.	Actual Red.
Bunny	69,451	Fast-Simplification	50%	46ms	0MB *	50.0%
Bunny	69,451	Fast-Simplification	80%	54ms	2MB	80.0%
Bunny	69,451	Fast-Simplification	90%	61ms	2MB	90.0%
Bunny	69,451	Open3D	50%	267ms	0MB *	50.0%
Bunny	69,451	Open3D	80%	378ms	2MB	80.0%
Bunny	69,451	Open3D	90%	459ms	3MB	90.0%
Bunny	69,451	Meshoptimizer	50%	17ms	2MB	50.0%
Bunny	69,451	Meshoptimizer	80%	20ms	7MB	80.0%
Bunny	69,451	Meshoptimizer	90%	22ms	7MB	90.0%
Bunny	69,451	CGAL	50%	1.1s	15MB	50.0%
Bunny	69,451	CGAL	80%	1.4s	15MB	80.1%
Bunny	69,451	CGAL	90%	1.5s	15MB	90.1%
Armadillo	345,944	Fast-Simplification	50%	414ms	113MB	50.0%
Armadillo	345,944	Fast-Simplification	80%	457ms	12MB	80.0%
Armadillo	345,944	Fast-Simplification	90%	497ms	14MB	90.0%
Armadillo	345,944	Open3D	50%	2.4s	11MB	50.0%
Armadillo	345,944	Open3D	80%	4.5s	9MB	80.0%
Armadillo	345,944	Open3D	90%	4.4s	10MB	90.0%
Armadillo	345,944	Meshoptimizer	50%	130ms	37MB	50.0%
Armadillo	345,944	Meshoptimizer	80%	163ms	37MB	80.0%
Armadillo	345,944	Meshoptimizer	90%	163ms	37MB	90.0%
Armadillo	345,944	CGAL	50%	5.7s	50MB	50.0%
Armadillo	345,944	CGAL	80%	7.7s	51MB	80.0%
Armadillo	345,944	CGAL	90%	8.4s	50MB	90.0%
Dragon	871,414	Fast-Simplification	50%	687ms	199MB	50.1%
Dragon	871,414	Fast-Simplification	80%	812ms	53MB	80.0%
Dragon	871,414	Fast-Simplification	90%	1.2s	28MB	90.0%
Dragon	871,414	Open3D	50%	6.0s	39MB	50.0%
Dragon	871,414	Open3D	80%	8.4s	30MB	80.0%
Dragon	871,414	Open3D	90%	9.6s	29MB	90.0%
Dragon	871,414	Meshoptimizer	50%	282ms	94MB	50.1%
Dragon	871,414	Meshoptimizer	80%	387ms	93MB	80.0%
Dragon	871,414	Meshoptimizer	90%	399ms	93MB	90.0%
Dragon	871,414	CGAL	50%	14.5s	127MB	50.0%
Dragon	871,414	CGAL	80%	20.5s	152MB	80.0%
Dragon	871,414	CGAL	90%	22.2s	121MB	90.1%
Lucy	28,055,742	Fast-Simplification	50%	26.3s	9.1GB	50.0%
Lucy	28,055,742	Fast-Simplification	80%	28.0s	1.6GB	80.0%
Lucy	28,055,742	Fast-Simplification	90%	30.6s	1.2GB	90.0%
Lucy	28,055,742	Open3D	50%	4.7min	0MB *	50.0%
Lucy	28,055,742	Open3D	80%	8.8min	0MB *	80.0%
Lucy	28,055,742	Open3D	90%	10.8min	0MB *	90.0%
Lucy	28,055,742	Meshoptimizer	50%	15.8s	3.0GB	50.0%
Lucy	28,055,742	Meshoptimizer	80%	19.2s	3.0GB	80.0%
Lucy	28,055,742	Meshoptimizer	90%	21.2s	3.0GB	90.0%
Lucy	28,055,742	CGAL	50%	8.3min	4.2GB	50.0%
Lucy	28,055,742	CGAL	80%	12.0min	4.2GB	80.0%
Lucy	28,055,742	CGAL	90%	13.4min	4.2GB	90.0%

Table B.14. Complete scalability results for all Stanford model-method-reduction combinations. Processing time, peak memory, and actual reduction percentage for each of 48 configurations.

Figure B.9 presents reduction accuracy for the Stanford models, plotting actual reduction percentage against target reduction. This complements the production asset reduction accuracy shown in Figure 4.6 and confirms that all methods maintain target accuracy across the full complexity range.

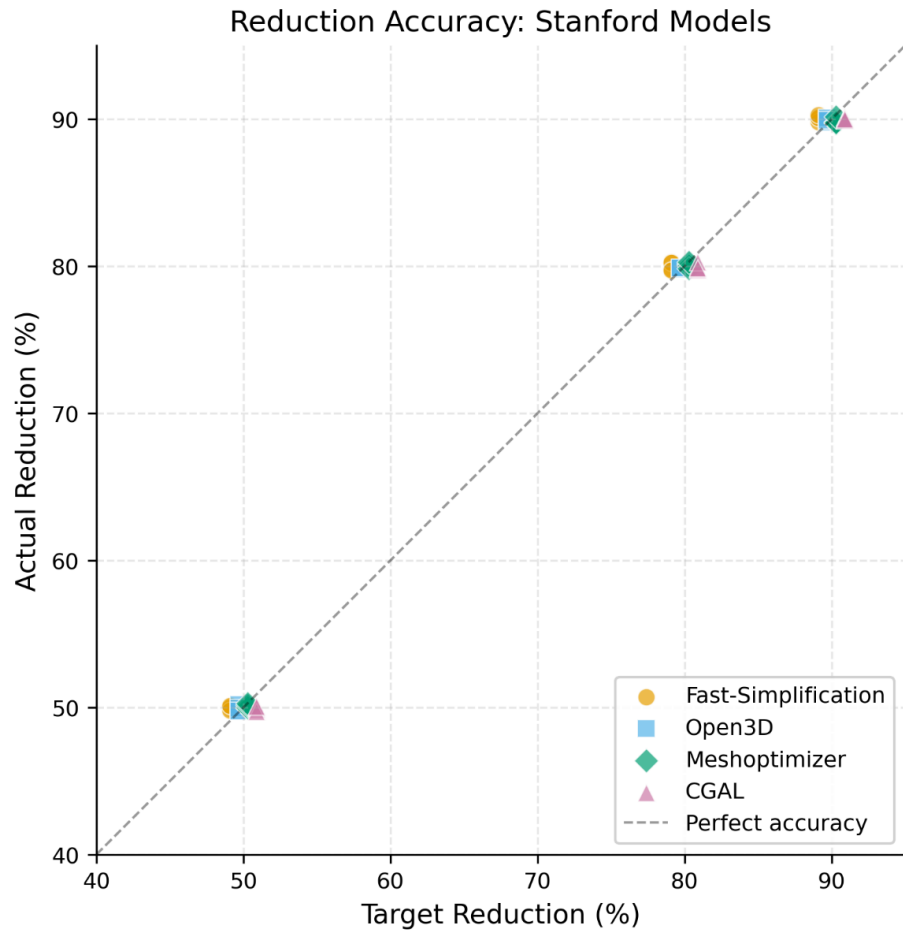


Figure B.9. Reduction accuracy for Stanford scalability models: target versus actual reduction percentage. Dashed diagonal represents perfect accuracy.

Table B.15 presents processing time scaling factors between consecutive complexity tiers, quantifying how processing time increases as input complexity grows.

	Bunny → Armadillo (5.0× faces)	Armadillo → Dragon (2.5× faces)	Dragon → Lucy (32.2× faces)
Fast-Simplification	9.0×	1.7×	38.3×
Open3D	8.9×	2.5×	46.7×
Meshoptimizer	7.6×	2.2×	55.8×
CGAL	5.3×	2.6×	34.3×

Table B.15. Processing time scaling factors between consecutive Stanford models. Each value represents the ratio of processing time at the higher complexity to the lower complexity for a given method and reduction level.

This template is based on a template by:
Steve Gunn (<http://users.ecs.soton.ac.uk/srg/softwaretools/document/templates/>)
Sunil Patel (<http://www.sunilpatel.co.uk/thesis-template/>)

Template license:
CC BY-NC-SA 3.0 (<http://creativecommons.org/licenses/by-nc-sa/3.0/>)